NUGET AT SCALE

An enterprise guide to managing NuGet package approvals, handling vulnerabilities, versioning and license compliance, setting up a solid CI/CD pipeline, and everything else growing teams need to scale with confidence.

inedo

NuGet at Scale

An enterprise guide to managing NuGet package approvals, handling vulnerabilities, versioning and license compliance, setting up a solid CI/CD pipeline, and everything else growing teams need to scale with confidence.

Table of Contents

NuGet At Scale

Using NuGet in development starts out easy, just a few projects, a small team, grabbing packages from NuGet.org. But as your org grows, things slowly get messy. More teams, more repos, more tools, all adding to the chaos. Before you know it, you're dealing with version conflicts, missing packages, broken CI pipelines, and no one really knows who's in charge of licenses or security. The problem isn't NuGet itself, it's the lack of structure around it. And honestly, most teams don't even realize it's a problem until everything starts breaking.

It comes down to a lack of governance. Tossing in another CLI, script, or policy won't fix it if everyone's making random decisions that don't scale. New projects keep adding tools and dependencies, and suddenly managing NuGet feels like a total guessing game with no clear owner. The real fix? Take a centralized, opinionated approach. Curated feeds for internal and external packages, automated approvals, security and license checks, and consistent rules. All built in from the start and made to scale.

In this chapter, we're diving into why NuGet tends to fall apart as things scale. We'll look at the sneaky problems that even "best practices" don't catch and what it really takes to get things under control. From handling transitive dependencies to sketchy licenses, we'll show how a solid, central package strategy can give you way more than control.

What Does "At Scale" Mean and Why is it Challenging?

As teams and apps grow, NuGet gets harder. Not because anyone's doing anything wrong, but because growth introduces hidden complexity that traditional tooling can't handle.

⛔ **Too Many Projects → Too Many Tools:** When you've got tons of projects, it's easy for the same package to show up in all kinds of versions. Plus, every new project brings its own tools, scripts, and random conventions. That version sprawl turns into a compatibility mess and makes updates way harder.

⛔ **Too Many People → Too Little Visibility:** The more devs and teams you have, the easier it is for random packages to slip in. Without a central way to track things, no one really knows what's being used, where it's from, or if it's even safe. Governance becomes a total guessing game.

⛔ **Too Many Updates → Too Little Control:** New package versions are constantly dropping. When developers install packages ad hoc, especially via CLI, it bypasses any shared standards. Even a small update can break builds or introduce vulnerabilities.

⛔ **Security & Compliance → Invisible Risk:** Security teams often don't have great visibility into what packages are being used, or where. That makes it tough to manage risk, enforce license policies, or build a clean SBOM when you need one.

⛔ **Environment Issues → No Guarantees:** Just because a package works in dev doesn't mean it'll behave the same in staging or production. Without tight control, these inconsistencies lead to bugs that are hard to trace.

⛔ **Pipeline Problems → Fragile by Default:** Relying on NuGet.org for every restore creates a single point of failure. If it's slow or down, your builds are blocked. And when every pipeline is configured differently, diagnosing issues gets even harder.

This isn't edge-case stuff, it's what scaling actually looks like. But it's not just about the number of packages. It's the sprawl of tools, behaviors, and assumptions that multiplies the risk:

- *Who's using which package?*
- *In which project?*
- *At what version?*
- *Under what license?*
- *With what upstream dependencies?*
- *And when do those need to change?*

If you're still trying to manage all this with tribal knowledge or scattered tools, it's already way out of hand. You need something built to handle the complexity, not hide it behind even more tools.

Why NuGet Gets Especially Tricky at Scale

We've talked about how things get messy as you scale—but the thing is, NuGet was built for simplicity, not scale. It works fine for one team, but once you add more people, projects, and processes, stuff starts breaking in sneaky (and frustrating) ways. You might not notice it on a small team or a single app, but once your org grows? That's when the real problems begin.

Transitive Dependencies Sneak In: NuGet automatically resolves transitive dependencies. That's convenient, until one of those indirect packages introduces a vulnerability, license conflict, or breaks your build. And since no one explicitly adds them, they often bypass reviews and policy checks altogether.

> 💡 **Solution:** Use packages.lock.json to lock down exact versions so you know exactly what's being used.
> ⛔ **But at scale:** You now have hundreds of lockfiles across dozens of repos. A single policy or package update requires coordinating updates across all of them. Keeping them aligned becomes a full-time job.

Inconsistent Restore Behavior Across Projects: Different project types (.NET Framework, .NET Core, SDK-style projects) restore dependencies differently. That inconsistency causes hard-to-debug issues across environments.

> 💡 **Solution:** Standardize on SDK-style projects and isolate restore steps in CI/CD pipelines.
> ⛔ **But at scale:** That might work for one team, but alignment across CI pipelines becomes political, not just technical.

Too Many Ways to Manage Versions: NuGet supports `packages.config`, `PackageReference`, `global.json`, and `Directory.Packages.props`. In large orgs, teams often mix and match. NuGet gives teams options, maybe too many. And when every team picks a different pattern, consistency becomes impossible.

> 💡 **Solution:** Use `PackageReference` + `Directory.Packages.props` for centralized version management.
> ⛔ **But at scale:** Migrating legacy projects takes time. And enforcing consistency across hundreds of repos? That requires tooling, documentation, and cultural alignment.

Unlisted Packages Still Get Restored: NuGet.org doesn't delete packages, it unlists them. If your builds rely on a specific version that later gets unlisted, your CI may break without warning.

> 💡 **Solution:** Cache packages in a private NuGet feed.

⛔ **But at scale:** Some teams stash packages in siloed repos or ad-hoc folders. That's brittle and non-reproducible. Without a central feed, you can't guarantee stability.

Missing or Incomplete License Metadata: Many NuGet packages don't include reliable license data. That makes audits a nightmare and opens you to legal risk.

> 💡 **Solution:** Automate license scanning with tools like ProGet, and block packages with unacceptable licenses (like GPL).
> ⛔ **But at scale:** Relying on manual review or leaving it to developers doesn't scale. You need policy enforcement and automation built into your approval workflow.

Vulnerability Scanning Doesn't Tell the Full Story: Running dotnet list package --vulnerable is a good start, but it often lacks context. You get a list of issues but no prioritization or remediation guidance.

> 💡 **Solution:** Integrate vulnerability scanning into a centralized repository like ProGet, where packages are scanned and classified automatically.
> ⛔ **But at scale:** Without a central hub, the same vulnerability might appear in five different feeds, or go unpatched in forgotten projects. You need visibility and enforcement.

NuGet.org Isn't Always Reliable: NuGet.org is a shared public resource. Outages, throttling, or rate limits can bring your CI/CD pipelines to a halt, especially when multiple builds are happening simultaneously.

> 💡 **Solution:** Cache your dependencies in a local repository.
> ⛔ **But at scale:** If every team is caching their own packages separately, you miss out on deduplication, visibility, and policy enforcement. You need a central caching proxy.

Publishing Internal Packages Is Messy: Internal NuGet packages often lack versioning standards, changelogs, or clear promotion processes. Teams publish directly to shared feeds or drop packages into folders.

> 💡 **Solution:** Treat internal packages like products. Add metadata, automate builds and promotion.
> ⛔ **But at scale:** Without a structured promotion pipeline (e.g., Dev → Test → Prod), internal packages become just another source of chaos and unreproducible builds.

Debugging NuGet Packages Is Frustrating: Packages rarely include debugging symbols. That makes it nearly impossible to step into third-party or internal code when debugging issues.

> 💡 **Solution:** Use .snupkg symbol packages and include source files during packaging.
> ⛔ **But at scale:** Developers don't consistently generate symbols. You need an enforced, automated publishing workflow, backed by a central repository that stores binaries and symbols.

Keeping in Control at Scale

You can't fix these problems by just adding more tools or scripts—especially when every team does their own thing. What you really need is a shift in how you manage packages. Think of how your organization moved from ad hoc CI scripts to standardized pipelines with GitHub Actions or Azure DevOps. That change wasn't just about automation, it gave you control and visibility. The same approach applies to NuGet: a centralized repo that's more than storage, with built-in governance and automation to make package management actually work at scale:

- **Central Package Feed:** Use a private repository like ProGet to act as the single source of truth. Proxy NuGet.org, cache what you need, and prevent direct internet restores in production.
- **Approval Workflow:** Automatically block unvetted packages from being restored in CI builds, deployed to production, or accessed by developers. Let developers request packages, but require security/license review before promotion into the "approved" feed.
- **Promotion Pipelines:** Move packages through environments—Dev → Test → Prod—just like you do with application builds. This ensures consistency and reproducibility across your lifecycle.
- **Security and License Controls:** Automate scanning for known vulnerabilities and enforce license policies (e.g., block GPL, allow MIT). Catch issues before they reach production. Treat Internal Packages Like Products. Automate versioning, changelogs, and publication with CI tools like BuildMaster. Store both binaries and symbols to improve debugging and traceability.
- **Make Everything Visible:** Generate SBOMs, show who's using which packages, and give your dev and security teams dashboards to track usage, risk, and compliance.

NuGet at Scale, Without the Pain

As your organization scales, NuGet isn't a problem, it just wasn't built for the scale and complexity modern teams deal with. Without structure, every team makes it work a different way—and that's what breaks things. When every team does things a little differently, and packages flow freely with no oversight, it's only a matter of time before you hit broken builds, mystery bugs, or a surprise audit that no one's ready for.

You don't need more effort. You need a foundation. With ProGet as your centralized package repository, you get consistency, visibility, and control across your entire org. Fast builds, fewer surprises, and a NuGet setup that actually scales with your organization.

Best Practices for Authoring Private NuGet Packages in the Enterprise

Non-standardized **NuGet package authoring** is like a messy toolbox with mislabeled tools. Unclear names, versions, and metadata leave developers guessing about a package's purpose. Extra files or missing dependencies can lead to broken builds, runtime errors, and unexpected failures.

Few talk about this—even Microsoft—because it's not "cool", and much advice doesn't fit **private packages** on private NuGet servers like ProGet. Proper package authoring often gets overlooked, but it's essential for keeping your projects dependable and secure.

From naming packages to using NuGet Audit in .NET, this chapter outlines the best practices you need to keep your private NuGet packages secure and under control.

Create a NuGet Package in the Enterprise

These days, the easiest way to create a private NuGet package is using an SDK-style project. This newer project format includes the package metadata in the project file itself.

While Microsoft has a pretty good step-by-step tutorial on how to create and publish a NuGet package with Visual Studio, the publishing steps are for nuget.org, and it doesn't give any guidance for how to manage your project once it's created. In general, your NuGet project should follow the same conventions that other projects at your organization use:

- **Single Project.** This generally means having its own source code (git) repository and issue tracker… even if it's a small library.
- **Wiki Page.** A wiki page will be especially important, even if it's just a readme file inside your repository. This will help other developers use the library, and allow them to build upon the documentation you start. At a minimum, it should have a brief description of the library, how to use/maintain the project, and links to the issue tracker. Later on, as you make major new versions of the package, you can add upgrade notes to the wiki page.
- **CI/CD Automation.** Especially with tools like BuildMaster, it's easy to create a CI/CD Pipeline for NuGet from the start so that even 1.0.0 is delivered as consistently as others.

Package Metadata in the Enterprise

Metadata is an important aspect of all NuGet packages. Standardizing which properties to use (and how) is important: the wrong metadata can lead to problems like confusion and wasted time for developers, or even deploying the wrong package to production.

You can set package metadata in Visual Studio (Project > [Project Name] Properties > Package), or directly in the project file. These will be added to the .nuspec file once the package is created. Our general advice for Package Metadata in the enterprise is to keep it simple and use only as few fields

as necessary. Unlike many package authors on nuget.org, you're not trying to "sell" people on using your private NuGet package or win a popularity contest with the most downloads.

☑ **Package ID: Use the Company name:** The NuGet package name is stored in the package's manifest file, which is stored within the package zip file itself. You can't just rename the file if you want to change the package name. It's like trying to change the title page of a PDF document by renaming the file – the contents of the file and the file name have no relation. Microsoft has some good advice for writing unique package identifiers:

- 💡 Search existing feeds before choosing a name to ensure uniqueness
- 💡 Use namespace-like names using dot notation and not hyphens
- 💡 If you have a sample, be explicit with a ".Sample" suffix

You're likely already using namespace-like names in .NET; that's great – continue to do that with your private NuGet packages. But unlike libraries in your applications, these packages should be unique across the entire organization and any expansion for years to come.

Start with the company name ("Kramerica") to avoid conflict with public packages. Be sure to register your company's name on nuget.org so you're protected from dependency confusion attacks.

Next, consider standardizing with a division or business name after the company name ("Kramerica.Warehouse" vs. "Kramerica.Offshore").

Keeping standards is incredibly important since packages are immutable. **They cannot be renamed, changed, or edited so standardize, standardize, standardize.** You must create a new NuGet package with the same internal contents to *"rename"* a package. You'll then have two packages: the "new" name one, and the "old" name one. And deleting those "old" packages is a bad idea because applications may be referencing that package.

☑ **Package Version: Use SemVer w/ Prerelease Labels:** Our recent blog post about best practices for versioning NuGet packages goes into detail about proper versioning. In a nutshell: Semantic Version (Version 2.0.0) using Prerelease tags is a must. You should always create a package with a pre-release tag and then repackage it later. The Prerelease tag signifies that the library within the NuGet package is untested and shouldn't be deployed to production.

☑ **Authors: Only use Company name:** Author is a required field, but it should be considered optional for private use in the Enterprise. Set one name as the author to keep with the standard; either the company name or a team name ("Kramerica" or "Kramerica.DevOps").

☑ **Description: Use Internal Wiki:** The Microsoft best practices come up short on description – ironic since we consider a short description key. The character limit is 4000, but it's completely unnecessary to write that much. Use the description to link to the company Wiki page so you can maintain the most up-to-date information. Be concise; reuse copy from the Wiki if necessary (likely to be pre-approved and reliable). NuGet Packages cannot be edited or updated, so hyperlinking to internal documentation is best.

☑ **Repository: Use Source Link:** A Source Control Repository gives traceability to a NuGet package. There are four fields; type, URL, branch, and commit. Source Link will automatically add repositoryURL and RepositoryType to the package metadata, saving time. This property is complex, requiring a CI/CD pipeline for your NuGet packages.

☑ **Icon: Optional, a quick visual cue:** An icon can be useful for quickly differentiating a NuGet package from others in a private NuGet repository on Visual Studio. If you have an appropriate icon file, feel free to include it, but it's not necessary.

Note: "iconURL" is deprecated so use "icon"

NuGet Audit: Optional but recommended, scan for vulnerabilities: NuGet Audit adds vulnerability scanning to your private packages using these two lines in either your `.csproj` file or a shared MSBuild file like `Directory.Build.props`:

```
xmlCopyEdit<PropertyGroup>
 <NuGetAuditMode>all</NuGetAuditMode>
 <NuGetAuditLevel>moderate</NuGetAuditLevel>
</PropertyGroup>
```

This setup flags security issues in both direct *and* transitive dependencies when you run `dotnet restore`. It's recommended to configure this at the repository level to ensure consistent and automatic coverage across all projects.

Package Metadata Properties to Avoid

✗ **Copyright: Ignore, not needed:** Your NuGet packages will be private or published on a private repository, so they don't need a copyright. If a project document doesn't need "(C) Kramerica" on them, neither does a private NuGet package.

✗ **Licensing: Ignore, not needed:** Like Copyright, your packages will be private and therefore don't need a license file, licenseURL, or requiredLicenseAcceptance.

✗ **Project URL:** Ignore, link to Internal Wiki: Following these guides, you will already include a link to the Wiki's project page in the description, making this field redundant. If you think this field should be filled, use the same project page link in the description.

✗ **Tags:** Ignore, worse than free-text search: Tags are a bad replacement for private use without a standardized system, especially compared to free-text searches. Creating tags and then maintaining their standards will also occupy a lot of time unnecessarily.

✗ **Release Notes:** Ignore, redundant to Wiki: The Wiki's project page should also link to issue trackers, give upgrade guidance, and other show release notes, making this field redundant to the description. In Visual Studio, some UI sections may display release notes instead of the description

which can cause confusion, so it's best to leave this field blank and rely on the description to do the heavy lifting.

✗ **Title:** Ignore, not needed with ID: The title field is not shown in Visual Studio or on nuget.org. Generally, it's not used for display purposes, so just use ID and leave this field blank.

✗ **Summary:** Ignore, redundant to the description: The summary field is deprecated and shouldn't be used, plus it's redundant to the description anyway. It has been replaced by the title, but as seen above it's not needed either. Leave this field blank.

✗ **Readme:** Ignore, not needed with Wiki: Readmes are not where developers typically go for documentation. Guides for using a NuGet package should be included in the Wiki anyway, so a readme will be redundant. Be sure to hyperlink to internal, up-to-date documentation that would be considered readme-worthy.

Standardizing Metadata Usage

Without standardized NuGet package authoring, your private feed can quickly turn into a mess—leading to broken builds, runtime errors, and security vulnerabilities.

Implementing best practices like consistent metadata, semantic versioning, and NuGet Audit integration transforms your package management into an organized, secure, and scalable system. Keeping property usage to a minimum will make it easier for developers to create and contribute while avoiding headaches down the line.

Best Practices for Versioning NuGet Packages in the Enterprise

Poorly versioned NuGet packages are a recipe for trouble. You'll be lucky if you're dealing with headaches from wasting time dealing with confusing and "broken" packages. At worst, you're looking at production problems like application crashes at runtime because the assemblies didn't load.

Versioning seems so simple (it's just a number, right?), but with NuGet, it's a bit of a minefield. There are **five** distinct, multi-part version numbers that can show up in a package—each with its own formatting rules, and behaviors.

In this chapter, I'll walk through how you can properly version your NuGet packages to speed up the development process, stay organized, and avoid those "wait, why did this break?" moments when deploying your NuGet packages and applications at scale in your organization.

How are Version Numbers used in NuGet Packages?

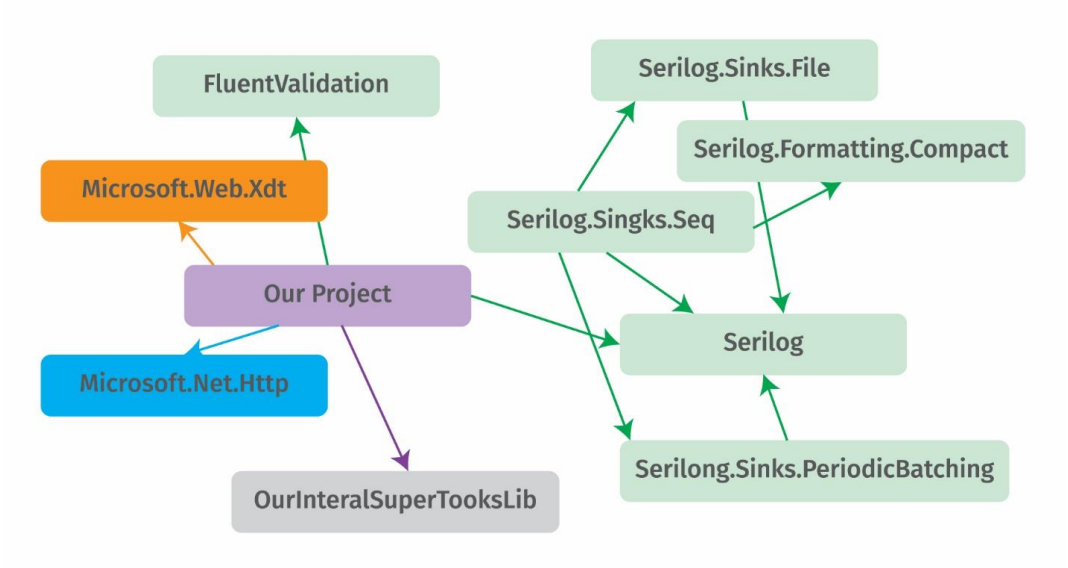
Like with all software, version numbers in NuGet are how developers tell one package apart from another. The difference between `v5.3.2` and `v5.4.0` isn't just the number—it's the bug fixes and enhancements made between those versions.

Without versioning, tracking what changed in which version gets messy fast. Eventually, the wrong version gets deployed to production, and nobody's happy. With any software, versioning isn't just about code: it's a communication tool between developers, stakeholders, and other humans.

However, version numbers are also used by NuGet/.NET in a few different ways.

1. Manage Package Dependencies

NuGet packages rarely function as a standalone unit. Most of the time, they rely on at least one other package—called a **dependency**—to be installed first. Dependencies often have dependencies, which means installing a single package can result in a "dependency tree" of required packages.



This is where things start to get a little tricky: **dependency resolution**. Essentially, only one version of a NuGet package can be used at a time. So, if two different packages (let's say A-5.4.2 and B-3.0.1) require *different* versions of another package (like C-1.2.1 and C-1.0.3), NuGet can't resolve the dependencies. This means your project won't build unless you pick different NuGet packages.

Thankfully, NuGet lets package authors use version ranges when specifying dependent packages. These can also get complex, but they essentially allow multiple package versions in different packages. Following the example above:

```
<!– Package A can specify that C-1.2.1 to C-2.0.0 is okay -->
<PackageReference Include="Package.C" Version="[1.2.1,2.0.0)" />

<!–Package B can specify that any 1.x.y is okay -->
<PackageReference Include="Package.C" Version="1.*" />
```

As a result, NuGet would likely choose C-1.2.1 because it satisfies Package A and Package B's dependency requirements.

In our earlier example, Package A might say "anything from version 1.2.1 up to (but not including) 2.0.0 is fine," while Package B might allow "any version that starts with 1." This gives NuGet enough wiggle room to pick a version that keeps everyone happy—like C-1.2.1, which fits both requirements.

2. Loading Libraries Within Your Package

NuGet packages include .NET library files—called *assemblies*—that your application loads at runtime. When that happens, .NET looks for a specific Assembly Version packed into the assembly's DLL file. If the assembly version doesn't match what your application expects, you'll get a nasty load error, and your app will crash.

Here's where things get confusing: the assembly version number is totally separate from the package version number and even follows a different format. This feels unintuitive, but there's

history there. .NET was created a decade before NuGet. Before then, assembly versions were the only way for applications to be tied to a specific version of a library.

Luckily, you can override assembly version resolution using binding redirects, and NuGet automatically does this for you in most cases:

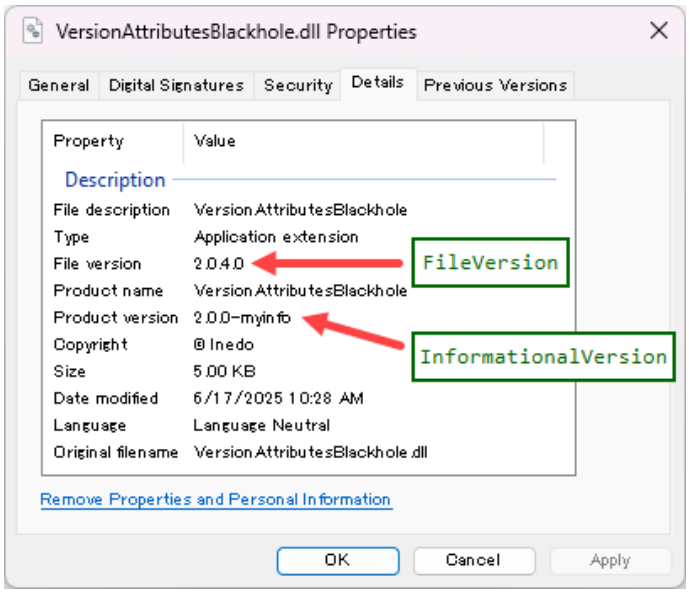
```
<!— Your application can specify that C-1.2.1.0 to C-2.0.0.0 is okay -->
<bindingRedirect oldVersion="1.2.1.0-2.0.0.0" newVersion="2.0.0.0"/>
```

That said, binding redirects, like version ranges, can get complicated fast, causing additional problems if not done right. If something goes wrong, you probably won't catch it until runtime.

3. Displaying Important Information

Additionally, DLL files can include two other version numbers: Assembly File Version and Assembly Informational Version. These version numbers are distinct from the Assembly Version that's used for loading the assembly at runtime.

They're purely informational and can be found in the Details tab in Windows Explorer:



The biggest problem this can cause is confusion when they don't match the assembly version.

Managing the Five Version Numbers

The following table lists the different version numbers, their format, and how they're used.

Version	Format	Usage/Display
Package	SemVer (e.g. 1.0.3) *or* 4-part Number (e.g. 2.3.25.1)	Used to identify one version of a package from another.
Dependency	Interval Notation (e.g. 1.2.1,2.0.0) *or* Floating Notation (e.g. 1.*)	Used during build time to resolve dependencies and find needed packages.
Assembly	4-Part Number (e.g. 2.3.25.1)	Used by .NET to load the assembly DLL at runtime.
Assembly (File)	Free Text (4-Part Recommended)	Displayed on the version tab of the Windows file properties dialog.
Assembly (Informational)	Free Text	Used for informational purposes in some WinForms libraries.

Best Practice: Package Semantic Versioning

While NuGet supports four-part version numbers, Microsoft now recommends using Semantic Versioning (SemVer) instead—and for good reason:

1. The "SemVer2 spec" is very detailed and clearly defines what each of the 3-parts mean, leaving little room for deviation across libraries.
2. The rules of semantic versioning are designed to not only help with dependency resolution, but with prerelease packages.
3. Developers and business stakeholders need no training and can understand basic SemVer numbers with minimal effort.

Semantic Versioning works by structuring each version identifier into three parts, **MAJOR**, **MINOR**, and **PATCH**. Each of these parts is managed as a number and incremented according to the following rules:

1. **Major** releases (2.0.0) indicate changes that will be incompatible with previous versions.
2. **Minor** releases (2.1.0) add functionality while still being backwards-compatible (in this example 2.1.0 will be compatible with 2.0.0).
3. **Patch** releases are minor bug fixes or security patches that should always be fully backwards compatible (2.1.4).

This structured approach makes it easier to manage versions and communicate changes effectively across teams.

Best Practice: Fixed Assembly Versions

Before NuGet, assembly version numbers were important: libraries had to be installed in the server's Global Assembly Cache before .NET applications could use them. But these days, things work differently. NuGet just copies the assembly DLL files next to your application/s executable files, so the entire application (libraries and all) gets deployed together. This makes the assembly versions a relic of the past.

For most modern .NET applications, using "fixed" numbers (like 9.9.9.9) for your library assemblies means that you'll never run into assembly load errors at runtime.

```
// Fixed version number
[assembly:AssemblyVersion("9.9.9.9")]
```

By locking the assembly version, you also reduce the need for **binding redirects** in your configuration files. Which, if you think about it, is effectively doing the same thing: after all, binding redirects just tell .NET to ignore the actual number, anyways.

Alternative: Match Assembly Version and Package Version

For some libraries, using a fixed number for your library isn't practical, so instead, use the same major, minor, and patch number as your package's semantic version number, and use "0" as the fourth part:

```
// Package Version number 5.4.2
[assembly:AssemblyVersion("5.4.2.0")]
```

Best Practice: File & Informational Versions

When using a fixed assembly version number, it's a good idea to still embed the number in the DLL file using the assembly version file attribute. This will make it easier to debug on the off chance you ever need to verify the right library was deployed with your application.

```
// Package Version number 5.4.2
[assembly:AssemblyFileVersion("5.4.2")]
[assembly:AssemblyVersion("9.9.9.9")]
```

You should avoid using the assembly informational version; it's only displayed in a few places in Windows and only gets used if the assembly file version is not present.

Best Practice: Don't Deploy Prerelease Packages

A key part of the SemVer standard, pre-release labels let developers know that a NuGet package is **unstable**. It's simple: if a package version includes a dash and descriptor after the patch number (e.g., 5.4.2-beta), it's considered **pre-release** or **unstable**. Packages without these pre-release labels are considered stable.

To help prevent untested library code from sneaking into production, a good rule of thumb is to block pre-release labelled packages from being deployed in production applications. The good news? Setting this up in your CI/CD pipeline is straightforward.

Best Practice: Build Prerelease Packages

When you build a NuGet package—whether manually with a tool like NuGet Package Explorer, or as part of a NuGet CI/CD process—you should always assign a **pre-release label**. This signals that the library within the package is unstable and should not be used in production deployments.

Packages must have unique numbers, and prerelease labels with "dot identifiers" are an easy way to manage this. Since there's no standard for what these identifiers should be, it may take some trial and error to find what works for your team. A popular method is to use "ci" along with a sequence number or `DateTime` string.

For example:

- **4.2-ci.4** uses "4" as a sequence number to identify it's the 4th build for 5.4.2.
- **4.2-ci.202103041103** uses a long Datetime string for when the package was created.

Best Practice: Repackage After Testing

If your team always builds pre-release NuGet packages, but never ships them to production, you'll need a repackaging process that turns pre-release packages into stable ones.

Repackaging creates a new package from an existing pre-release package, using the exact same contents, but changing the version number. Since NuGet packages are just .zip files, you can repackage them by:

1. **Downloading the package**.
2. **Opening the zip file**.
3. **Editing the .nuspec manifest file with the new version**.
4. **Saving the zip file**.
5. **Publishing the new package**.

By changing the manifest file, you have created a new package, **identified by a different version number.**

Repackaging ensures that what goes to production is exactly what you have tested: Your build server creates 5.4.2-ci, you test it, and upon approval, you repackage it as stable version 5.4.2. You can script your CI/CD tools to include repackaging in the pipeline (our BuildMaster can do this), or you can use a package server with built-in repackaging functionality, like ProGet.

You can write a script to tell your CI/CD tool to include repackaging in the pipeline (our BuildMaster can do this), or you can use a package server with built-in repackaging, like ProGet.

Beyond Versioning and NuGet

Learning proper NuGet package versioning means learning to handle five distinct version numbers, but putting in the time now will reduce a lot of headaches in the future and may even save you from production problems like application crashes.

Implementing the best practices for each version number, such as SemVer and fixed assembly versions, is a good way to dodge potential issues like unstable packages reaching production and assembly errors, so start using these tips now, since you'll likely be versioning many NuGet packages down the road.

How to use CI/CD pipelines for your NuGet Packages

I totally get it. CI/CD for NuGet isn't just annoying—it can feel straight-up impossible. Many teams get frustrated using CI/CD for their NuGet pipelines since it doesn't play nice with the best practices they've already implemented for their application pipelines.

CI/CD is supposed to help your team produce, test, and deliver NuGet packages faster and better. Still, a lot of teams skip it entirely. Why? Because they're flooded with a ton of builds, packages buried in packages (most of which no one will ever use), and it all just feels like it doesn't actually work.

In this chapter, we'll explore why and how to implement CI/CD with your NuGet packages and how to implement a solution that works at scale. We'll cover the commonly encountered challenges, specifically the CI/CD and NuGet disconnect, along with the non-options for handling these issues, and the secret solution to reconciling CI/CD and NuGet in your team's pipeline.

Challenges with CI/CD & NuGet

Teams new to building/using their own NuGet packages with CI/CD usually hit the same two roadblocks:

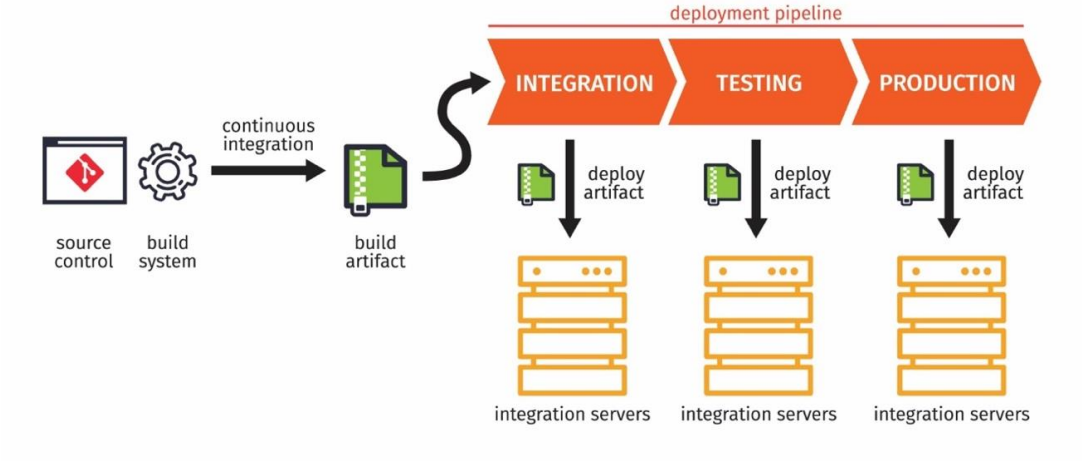
- You end up drowning in packages—most of which no one will ever actually use.
- You either can't use approved/tested pre-release packages in production... or your team just starts using pre-release packages anyway.

Without CI/CD, you'd simply create a NuGet package when it's ready to ship, but with CI/CD, suddenly you're swamped in dozens of packages, frequent library updates, more builds, and the difficulty of keeping track of all the version numbers that come with this.

The real **disconnect**? It happens when teams **try to treat NuGet pipelines like application pipelines**. Sure, the benefits and goals are the same—**more stable, reliable releases**—but applying application CI/CD best practices to NuGet will almost always lead to developer headaches.

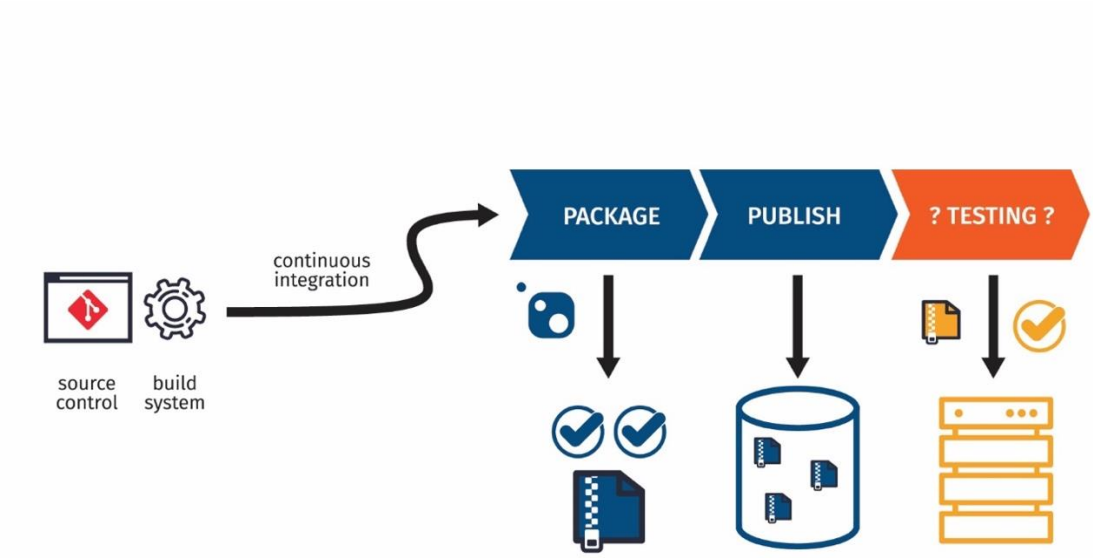
✓ CI/CD for Applications

It's pretty straightforward: a build artifact should only hit production after it's been tested in a number of environments.



⚠ CI/CD for NuGet Packages?

This is where a lot of the frustration kicks in: a NuGet package is essentially unusable until it reaches the last stage of the pipeline—*Publishing*.



The CI/CD and NuGet Disconnect

This frustration really comes down to **three best practices** your team is following:

1. **Packages are Immutable (Read-only).** Once published, a NuGet package file can't be modified. You can't "edit" the version number of a package or change its status, since the version number is part of the metadata baked into the file itself.
2. **Untested code shouldn't be deployed.** Rebuilding a NuGet package can produce different results, especially when you throw wildcard dependencies into the mix. This means you have to test the exact code you've just built, if you're planning to ship.
3. **Only deploy stable (non-pre-release) packages.** It's right there in the name—pre-release packages don't belong in a production environment. Only stable versions should ever be released in your applications.

Three Non-options for NuGet CI/CD

Once a team kicks off CI/CD for their private NuGet packages, they're in for a ton of builds and a whole bunch of packages that'll **never** actually get used. Most teams see only three solutions to choose from, and honestly, none of them are great.

✗ Use new Version Numbers at build time

How: With every new build, you create a brand-new version. That could mean using three-digit versioning (e.g. `3.4.2, 3.4.3, 3.4.112`), or four-digit versioning (e.g. `3.4.0.0, 3.4.0.1, 3.4.0.112`) for every new unstable package version.

😊 It's super clear which version is the latest.

😞 But neither approach follows **SemVer**, and neither indicates package stability. Further, taking the three-digit versioning route means sacrificing an entire number (**major**, **minor**, or **patch**) to communicate the pre-release version.

✗ Overwrite packages when you publish

How: Download your package (e.g. `3.4.2`), overwrite it every time you build, and then re-upload it.

😊 Your team avoids a landslide of packages and builds.

😞 This totally breaks the immutability rule. If multiple "versions" of "version 3.4.0" exist, how can you tell which one's stable? Plus, overwriting creates caching headaches, since Visual Studio (and CI servers) generally won't download a package that's already downloaded. That means your team members have to clear the package caches to use the most recent v3.4.0.

✗ Deploy prerelease packages

How: Add pre-release labels to the end of your package (e.g. -ci.1, -ci.2, ci.11). A package is tested in your application, and once it passes, it's good to go for production.

😊 The package's quality is clearly labeled, and you can apply all your standard CI/CD best practices to this pipeline. This is the best way to go.

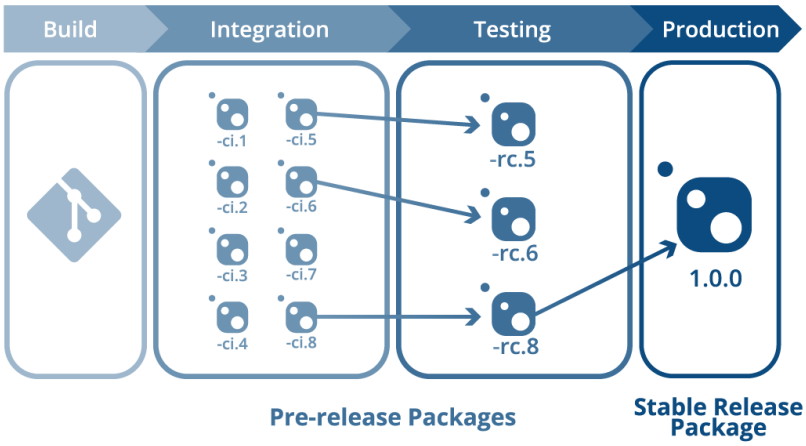
😞 Here's the catch: since your team is following CI/CD best practices, you're not using pre-release packages in production. Now, you have to create a brand new stable version of the package, sending it through the pipeline all over again. Hello, **CI/CD loop**.

It's here where many teams start to feel overwhelmed, caught between wasting time creating multiple packages that will never be consumed, or throwing SemVer out the window.

✓ The Secret to CI/CD for NuGet

Sure, it seems difficult to reconcile package immutability, SemVer, and continuous integration for your private NuGet packages, but using "repackaging" will let you use these best practices without the headache.

Repackaging creates a new package from an existing package, using the exact same content—just with a different name. For example, let's say build 8 yields 1.0.0-ci.8 for testing; once approved and repackaged, a release candidate, 1.0.0-rc.8, is created. Finally, a stable version, 1.0.0, will be created for deployment in production applications.



How to Repackage a NuGet Package

A NuGet package is really just a zip file with a `.nupkg` extension, which means repackaging takes just a few simple steps:

1. Downloading the package.

2. Opening the package as a zip file.
3. Editing the `.nuspec` manifest file and changing the version number.
4. Publishing the package file.

That said, this is *definitely* not something you want to keep doing by hand. You can script your CI/CD tool to include repackaging in the pipeline (our BuildMaster can do this), or you can use a package server with built-in repackaging functionality, like ProGet.

NuGet CI/CD Done Right

CI/CD for NuGet packages can feel nigh **impossible** when your team follows application CI/CD best practices. A tirade of builds, redundant packages, and unstable packages often leads development teams to skip CI/CD, even though it should be benefiting productivity.

However, by adapting **repackaging** into your CI/CD pipeline, you can ensure that what goes to production is exactly what you have tested, alleviating the worry of wildcard builds and unstable packages, while steering your team clear of an avalanche of unused NuGet packages.

How to Avoid Costly Lawsuits from Unexpected NuGet License Agreements

Ever feel like your dev team treats Visual Studio like an App Store—grabbing NuGet packages on a whim and breezing past the terms of service just to click "accept"? This kind of attitude toward open-source packages can land you in serious hot water. Just ask Orange S.A., who got hit with over €900,000 in damages after violating the GPL-2.0 license in the *Entr'ouvert v. Orange S.A.* case. All because they skipped some key requirements like sharing source code and proper notices.

For you and your team, this isn't just a legal cautionary tale—it's a reminder that open-source licenses matter. It's not just about avoiding legal pitfalls; it's about safeguarding your project's integrity and reputation. A little attention up front can save you a lot of pain (and money) down the line.

In this chapter I'll explain how you can avoid this kind of costly lawsuit, as well as automate some of the processes so they can be as simple and fast as possible. It all starts by adding NuGet Packages to your company's existing third-party software policy.

Third-party Software Policies

Whether or not they know the specifics of the policy, most employees do know that they cannot download free or paid software onto their company laptops without permission.

This is because an authorized person must check, among other things, the software's licensing agreement and accept its terms. Without this, an employee could unknowingly cause a lawsuit due to license violations.

NuGet Packages are just like these free/paid applications. The only difference is that most developers mindlessly download packages off NuGet.org via Visual Studio since they trust the source. It doesn't seem like it's just downloading random things from the Internet.

Whether or not they know the specifics of the policy, most employees do know that they cannot download free or paid software onto their company laptops without permission.

This is because an authorized person must check, among other things, the software's licensing agreement and accept its terms. Without this, an employee could unknowingly cause a lawsuit due to license violations.

NuGet Packages are just like these free/paid applications. The only difference is that most developers mindlessly download packages off NuGet.org via Visual Studio since they trust the source. It doesn't seem like it's just downloading random things from the Internet.

NuGet Packages are Legally Binding

Just because they are easily downloaded with a click of a button doesn't mean a NuGet package doesn't have huge, real-world ramifications *(again, see Entr'ouvert v. Orange S.A.)*.

NuGet packages are copyrighted material. When someone creates a package, they own its copyright – except in the rare case of being published in the Public Domain. The author decides how others can use their NuGet package through the license agreement.

License Agreements are legally binding contracts. When you use or otherwise incorporate freely available NuGet packages into your own applications, you are agreeing to a contract with the author of that package. By downloading a package, you—or you on behalf of your company—are agreeing to the terms. That's why it's so important to have a consistent process for reviewing and approving these packages before they make it into your codebase.

How to Expand Your Existing Policy

In theory, it's easy to add NuGet packages to your existing third-party software policy. Just do as you always do: ask for permission to use the package. As part of that process, the license agreement will be checked by an authorized person within the company, like a team lead, the legal department, or even the CEO.

Once a NuGet package is approved, it can be used by developers immediately. If they want to use one that is not "pre-approved" it must go through the authorized person. Fortunately, many NuGet packages use the exact same license agreement. This means that, if you can get approval for one type of license (ex. "MIT License Agreement"), then all other packages with that license would be acceptable.

There are two issues, however, with this manual processing; it's very time-consuming, and **the authorized person is likely not a developer**. Before the authorized person can properly vet the license, they need to be able to access the license. What's the best way to do that?

NuGet Metadata Licenses Explained for Non-Developers

Feel free to send this section to any non-developer as a quick explainer.

NuGet License Expressions

In a NuGet package's metadata (known as the .nuspec XML manifest file), there is a "license expression" property. A license can be "expressed" in three ways: a license expression, a file, or a URL.

Expression	Usually, Software Package Data Exchange (SPDX) identifier code. A code that represents an already-written open-source or free software license. Some are simple like "MIT" or "IJG," but others can be complex if an author opts for a composite license.
File	A file found within the NuGet package metadata that can be opened and read. Either a .txt or .md file. A developer would use File if their license is non-standard or proprietary.
URL	A URL that links to a website page with NuGet's license. This type is often shown in user interfaces like nuget.org.

⚠ License URLs Can Change

One problem with the URL expression is that the page linked to can change — and the license you originally agreed to will be different. Saving a copy of the page will ensure you have whatever you originally agreed to.

NuGet Package License Types

Once you've accessed the license, either by URL, file or following the SPDX expression, you can start to check its content. Anyone can write any type of license agreement they'd like, as seen in the many proprietary licenses found in paid software. Open source, and more specifically those publishing to Nuget.org, tend to use the SPDX License List, a standard list of licenses available to reference.

The SPDX List is maintained by the Linux Foundation and aims to identify and standardize open-source software licenses. The list has well over a hundred items, and each license has an associated code. This shorthand allows authors to quickly insert their preferred license via 'licensing –expression.' Some of the more popular licenses are:

SPDX Code	Full License Name
MIT	MIT License
MPL-1.1	Mozilla Public License 1.1
Apache-2.0	Apache License 2.0
BSD-3-Clause-Attribution	BSD with attribution
Beerware	Beerware License

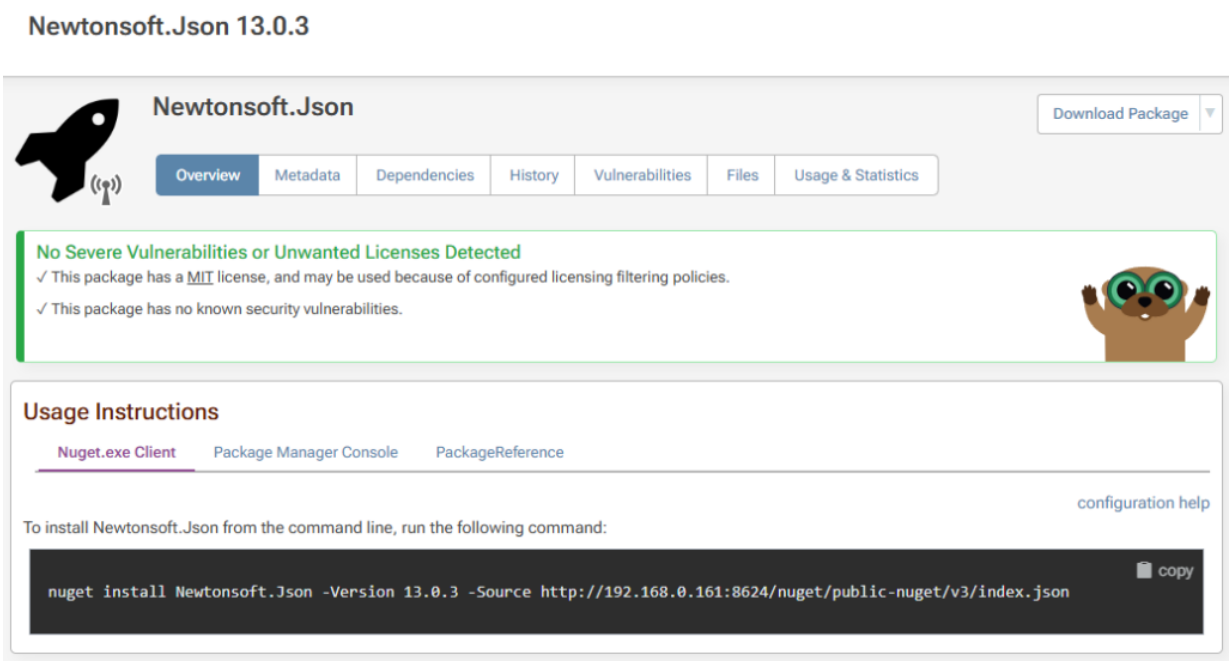
The SPDX Code is "fixed," which means the license it represents will never change. Changes will require a new "version" of the license, like MPL-2.0.

If a package is licensed as MPL-1.1, and you've already decided that MPL-1.1 is acceptable for your organization, then you won't need to read it again. This is why SPDX codes serve as a useful shorthand to license their NuGet packages.

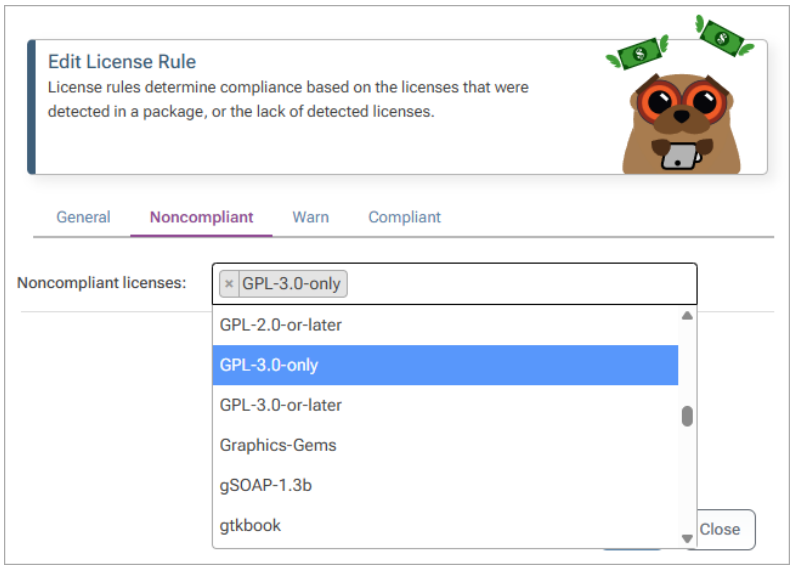
Reduce the Team Leads Workload Automatically

The above section is a great introduction for a non-developer, authorized person – e.g. CEO or legal department. A team lead assigned this task will likely understand how to access the license and know some of the more popular SPDX codes.

Still, manually vetting licensing will be incredibly time-consuming for anything. Instead, do it automatically. An automatic system can be programmed to scan a NuGet package, check what license it has and either approve or disapprove its download. Take ProGet, for example. It can immediately display a package's license on a user's feed.

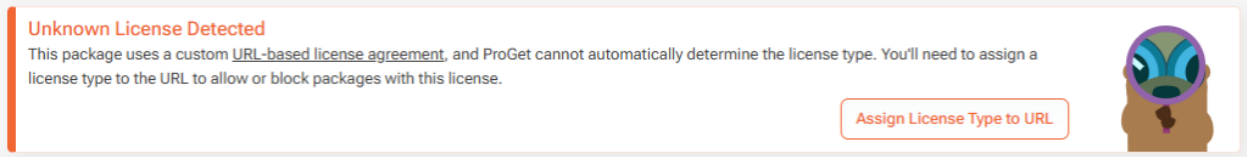


It can also be configured to filter licenses at the feed level. Licenses like GPL-3.0 are often unacceptable to businesses, so a user can configure the feed to block attempted downloads of packages with a GPL-3.0 license.



Attempting to download a NuGet package with a blocked license will be impossible, and ProGet will alert the user that the action is blocked due to license filtering rules.

An automatic system can also flag NuGet packages without a clear license expression, be it a missing SPDX identifier or an unrecognized URL. When detecting an unknown license, ProGet will display a warning and prompt a user to create a new license type.



An automatic system configured by a team lead, with approval by an authorized person, will greatly cut down on time spent manually shifting through NuGet package licenses.

If you're already using ProGet, check out our license filtering guide for yourself to see how easy it is.

Subtle Changes for the Better

By integrating NuGet packages into an organization's existing third-party policy, developers can continue to download from NuGet.org without checking – because it's already been done for them!

Team leads can rest easy as their developers click and scroll, because systems are in place to ensure they're not agreeing to suspicious licenses. This process, especially when it's automatic, can greatly decrease a company's risk of legal action while simultaneously boosting development efficiency.

Of course, this is only one factor to consider when using NuGet in enterprise environments. Filtering licenses, along with other best NuGet practices like package approval workflows or SemVer for NuGet packages, will boost your team's NuGet development.

What are NuGet Package Vulnerabilities and How to Manage Them

Have you seen vulnerability warnings when working with NuGet packages? Or read about the latest vulnerability in a NuGet package, and are concerned about your .NET application's security?

When you develop .NET applications, you most certainly will work with NuGet packages to manage libraries and dependencies. Making sure these packages are safe for production use is a crucial responsibility. Chances are you've also used the NuGet or `dotnet` CLI to scan all your packages.

Scanning NuGet.org could return several vulnerability warnings, and lead to a bunch of questions: *How do I manage them? Does "Low severity" mean it's safe? How about "High severity", just how severe is it? Can I explain what this means... or how it affects my code? What should we do about it? Do I even need to do anything at all?*

It's a lot to consider! In this chapter, I'll explain NuGet vulnerabilities, how scanning works, and how organizations can manage them effectively at scale. I'll then talk about using ProGet to handle your vulnerability management with features like assessment rules and download blocking to resolve your NuGet vulnerabilities.

What are NuGet Package Vulnerabilities and Scanning Exactly?

For example, Newtonsoft.Json has a vulnerability concerning "Improper Handling of Exceptional Conditions":

Expressions with high nesting levels that lead to Stack Overflow exceptions or high CPU and RAM usage can lead results in Denial of Service (DoS)

- *https://github.com/advisories/GHSA-5crp-9r3c-p9vr*

Vulnerabilities like these pose potential security risks. When you run `dotnet list package --vulnerable`, you'll get a list of these vulnerabilities and what package versions they affect.

```
C:\NuGet>dotnet list package --vulnerable

The following sources were used:
   https://api.nuget.org/v3/index.json
   C:\Program Files (x86)\Microsoft SDKs\NuGetPackages\

Project `NuGetAuditTest` has the following vulnerable packages
   [net8.0]:
   Top-level Package      Requested   Resolved   Severity   Advisory URL
   > Newtonsoft.Json      12.0.3      12.0.3     High       https://github.com/advisories/GHSA-5crp-9r3c-p9vr
```

Keep in mind, this just compares your NuGet packages against a database of reported vulnerabilities, not every vulnerability a package could have. This isn't some kind of "virus scanning" nor does it "eliminate all vulnerabilities". What's more, it only scans a single database (GitHub Advisory Database), so vulnerabilities reported in other databases won't show up.

But it's a lot better than nothing! However, the "severity" of these vulnerabilities doesn't reflect how you should handle them. For example, even though the Newtonsoft.Json vulnerability from earlier is a "High" severity, it probably won't be a real risk for your applications.

Quite a high nesting level (>10kk, or 9.5MB of `{a:{a:{...` input) is needed to achieve the latency over 10 seconds, depending on the hardware.

Wait... a 9.5MB malicious JSON document!? For an application sitting behind your firewall, that's probably not a real worry. The rated Severity doesn't factor in the likelihood of exploitation or the impact on your application.

Most vulnerabilities are not easily exploitable. If they require specific access rights or aren't accessible through typical user interactions, the risk of exploitation is close to zero.

So, What Can We Do About Vulnerabilities?

First things first, **don't just mindlessly upgrade everything!** Instead, you should:

✓ Regularly monitor packages you use in production for updates

✓ Routinely monitor security advisories like GHSA and NVD

✓ Use automated scanning tools to identify new vulnerabilities

✓ Assess vulnerabilities not simply based on "severity" but the real risk and impact

It's all about balance. As I mentioned, mindlessly upgrading when new updates are available isn't a great idea as not all vulnerabilities detected may pose a risk to your code. However, upgrading *could* introduce unknown vulnerabilities or cause other regressions or issues in your application.

What's more, you can't just rely on the NuGet client's built-in scanning. There's simply no guidance for resolving detected vulnerabilities, and the implied advice is to just upgrade by default. For example, if you were to blindly upgrade `NewtonSoft.Json` to the latest version -- you might end up with production errors because of how the new version deserializes objects.

💡 Assess Vulnerabilities Case-by-Case

Instead of always upgrading, first, determine if the vulnerability should be worked around or simply ignored. The reported vulnerability may simply not impact your application.

Will there even be an opportunity for a malicious actor to inject a 9.5MB JSON file? What are the consequences of Denial-of-Service attack on our application?

While the built-in NuGet vulnerability feature is helpful, it should not be your sole source of information. Combine data from various sources and conduct independent assessments of each package.

Be consistent in your Assessments

In large organizations, vulnerability assessments need to be repeatable and clearly documented. If you think about it, an "assessment" is basically someone's personal judgment and their *subjective* take on the vulnerability's severity at that time.

What one person considers "critical", another might see as less important. Even if it's one person doing multiple scans, evolving threats or personal knowledge growth can alter their judgment over time.

Should I Just Not Bother with Vulnerability Scanning?

Assessing vulnerabilities in NuGet packages is time-consuming and labor-intensive: you have to manually go through every vulnerability one-by-one, look them up, determine potential risks, and then decide how to deal with them. And then you have to somehow communicate this with the rest of your team (or your future self), so you don't have to do the same work the next time the same vulnerability comes up.

This will eventually lead to review fatigue that may result in neglecting vulnerabilities that require thorough assessment.

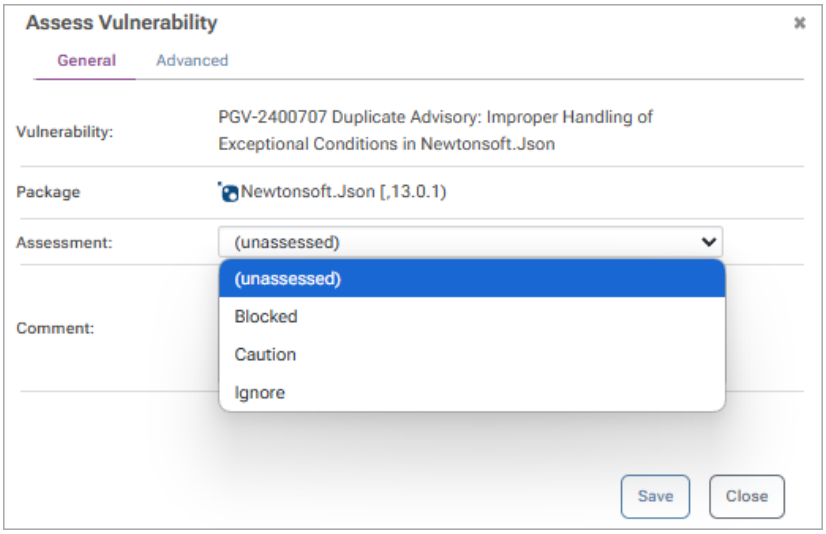
So should I just not bother at all?

No! You should still scan your packages and assess the reported vulnerabilities on a case-by-case basis. Sure, most vulnerabilities have no real risk -- but do you really want that bad package slipping through? Fortunately, ProGet's automated vulnerability management can make this very easy, making large-scale vulnerability management far more sustainable.

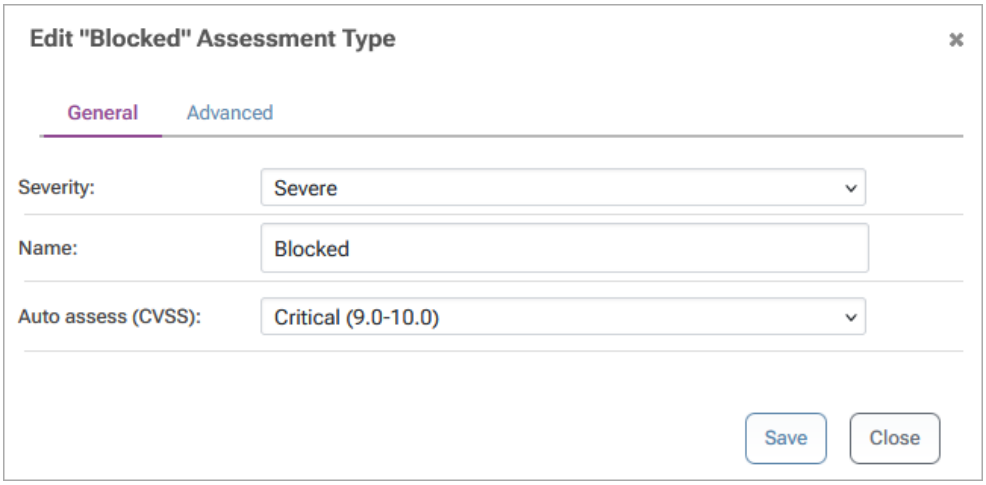
How ProGet Can Optimize Your Vulnerability Assessments

ProGet's automated vulnerability assessments and download blocking, can significantly optimize the effectiveness of your NuGet package vulnerability management.

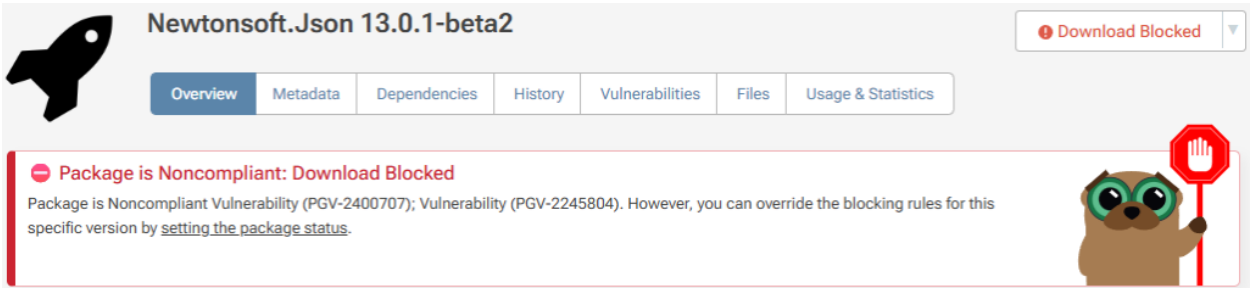
⚙ **Assessment management** allows you to configure different types of assessments (block, ignore, caution, etc.) and log comments so that everyone can follow and understand the decisions made.



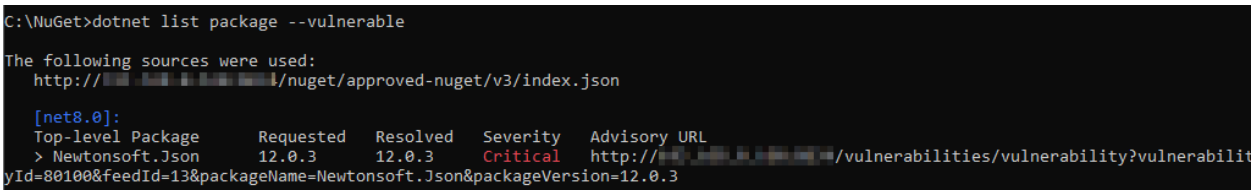
⚙ **Automated vulnerability assessment** allows you to create rules based on the reported severity (low, medium, critical, etc.), so that you don't have to react the moment a new vulnerability is reported; you can always apply judgment and edit the auto-assessment later



⚙ **Download blocking** can ensure that only approved packages are used. This reduces potential vulnerabilities and communicates a clear message to developers. It will prevent their use of specific packages, causing an error if they attempt to download them again.



⚙ **Custom Assessments** mean that audit results reflect your team's own assessments. So vulnerabilities you've assessed as "Ignore" won't show up in the list, or one's you've marked as "Blocked" will show up as "Severe" issues, even if the public advisory says it's only "Moderate" or "High". Basically, you see the severity *you* care about, not just what someone else decided.



Additionally, ProGet comes bundled with ProGet Vulnerability Center. PGVC is a constantly updated offline aggregation of leading vulnerability databases, including GSHA and the Global Vulnerability Database. No longer do you need to worry about constantly having to monitor various vulnerability databases, as everything is in one place.

Take Control of Your NuGet Vulnerability Management

As your development organization grows, vulnerabilities shouldn't be a concern to your applications as long as you know what to do:

- Learn to assess vulnerabilities not only by severity but potential risk and impact
- Make informed decisions and determine how to react to vulnerabilities identified

This will involve individual package assessment over just mindless upgrading. ProGet can help you assess vulnerabilities consistently and reliably by introducing automated assessment against an aggregated, offline database of vulnerabilities.

Why You Should Create a Package Approval Workflow for NuGet.org

Are you deploying applications to production with packages downloaded *directly* from NuGet.org? Many organizations start this way—it's the default behavior of most NuGet clients. But while convenient, this approach can introduce serious long-term risks.

Packages from NuGet.org are not vetted for quality, unwanted licenses, or vulnerabilities. Requiring developers to constantly make decisions on which packages are acceptable is not only inefficient, but will lead to unwanted code in production. This is where a package review and approval process come in.

In this chapter, we'll look at why using raw packages directly from NuGet.org isn't a good approach, what a package approval workflow is, why your team should use one, and how to easily set it up in a tool like ProGet.

3 Risks of Using Packages Directly from NuGet.org

Anyone can publish a package to NuGet.org; they aren't vetted or endorsed by Microsoft, and using them without properly vetting them can lead to all sorts of issues.

⚠ Security Risks and Vulnerabilities

Many packages contain known vulnerabilities that attackers can exploit; some even have malicious code. Not all packages are insecure, but unless you look for security issues, your applications and data could be compromised.

⚠ Unwanted Licenses

Nearly all NuGet packages have a license agreement that will legally bind your company when you use the package. Some of these licenses (like GPL-3) are not a good fit for most applications and can put your company at serious legal risk.

⚠ Low Quality Packages

There are a lot of great packages on NuGet.org, but some are simply not suitable for production code. They can introduce bugs or performance issues into your application. This results in wasted time and effort to resolve, as you may need to troubleshoot problems caused by using these packages.

Use Only Approved NuGet Packages

How? A Package Approval Workflow

Think of a Package Approval Workflow as "code review" but for NuGet packages. And like code review, the process will vary from team to team. Primarily, you'll need to vet packages from NuGet.org before they are used in the application.

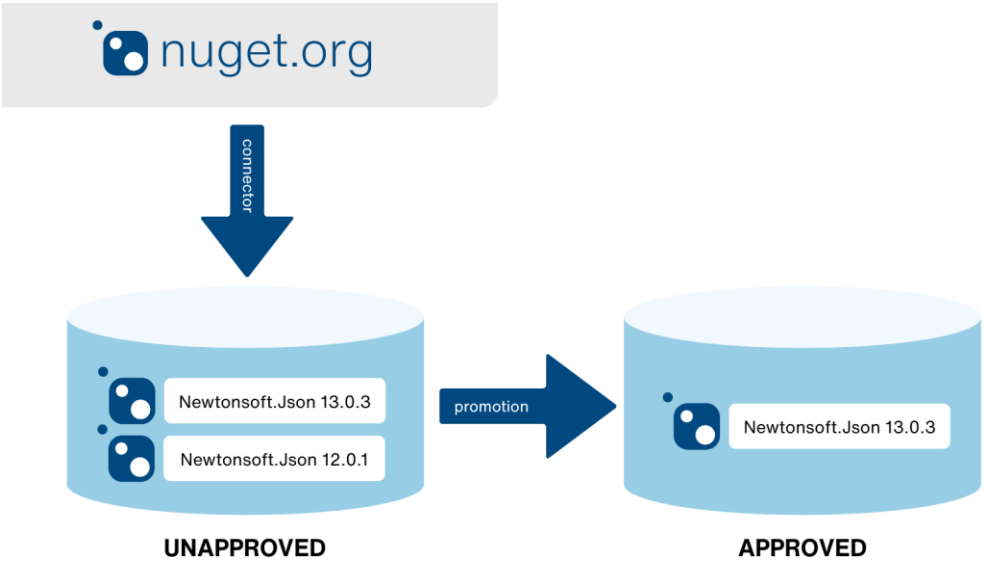
This means checking that each package has:

- ☑ Low/no Vulnerabilities or Security Risks
- ☑ Acceptable License Agreements
- ☑ Suitable Quality for Production

Finding the right process is important. If it's too complex, you'll risk developers bypassing the process. If it's not thorough enough, unwanted packages will make their way in your application.

Implementing A Package Approval Workflow

A package approval workflow is typically built with two feeds. One of the feeds is used to store unapproved packages from NuGet.org. After the packages are vetted, they are "promoted" to the other feed for production use.



What are the Benefits of a Package Approval Workflow?

A package approval workflow ensures that developers can only use packages that have been vetted.

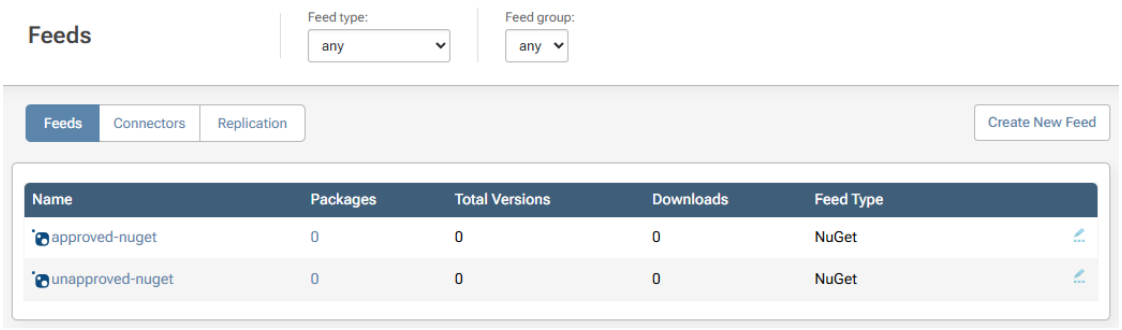
It's a simple and effective way to reduce the day-to-day decisions that developers need to make. They won't need to constantly consider what packages they should or shouldn't be using.

How to Create a Package Approval Workflow for NuGet

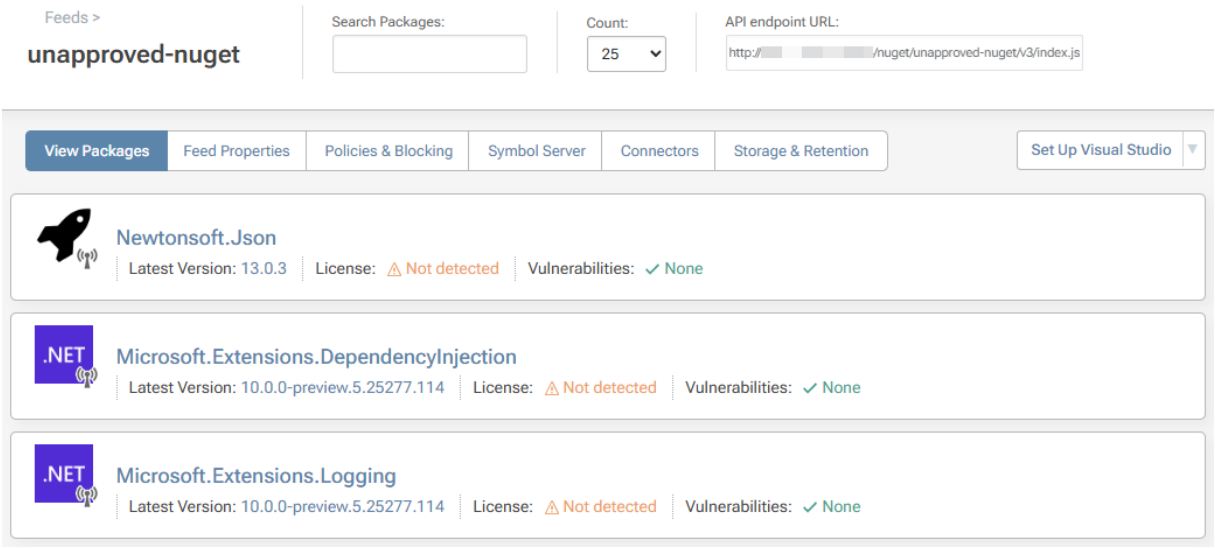
Creating a package approval flow for your NuGet packages is pretty straightforward and can be done with two feeds in a private NuGet server.

We'll use ProGet here, but other NuGet servers are conceptually similar.

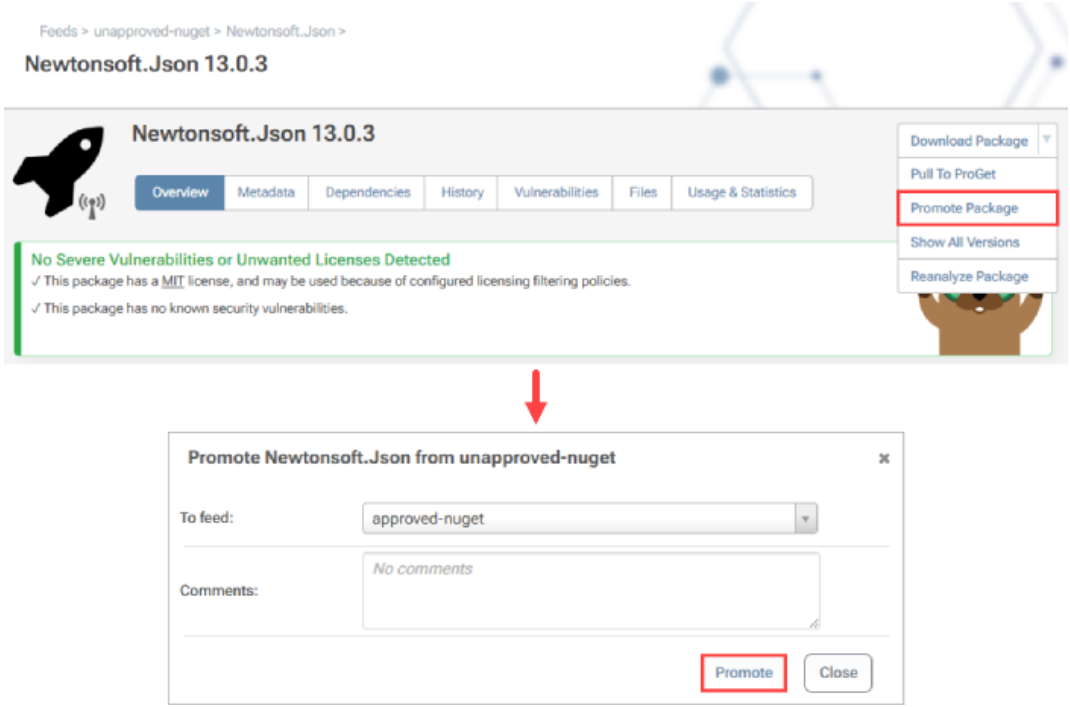
First, create two NuGet feeds, one for unapproved packages and one for approved packages that developers can download packages from. If you already have an existing unapproved NuGet package feed, create a second feed for promoting your approved packages.



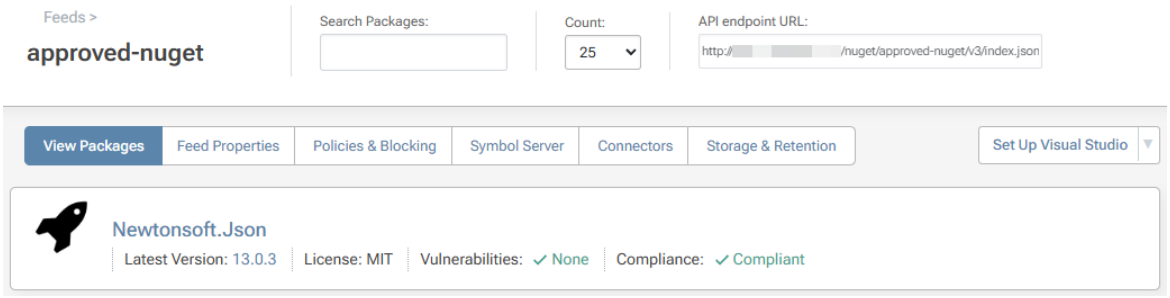
You'll want to "connect" the unapproved feed to NuGet.org. This will populate the feed with remote NuGet packages from the public repository. They can be downloaded through ProGet and are available for promotion once vetted.



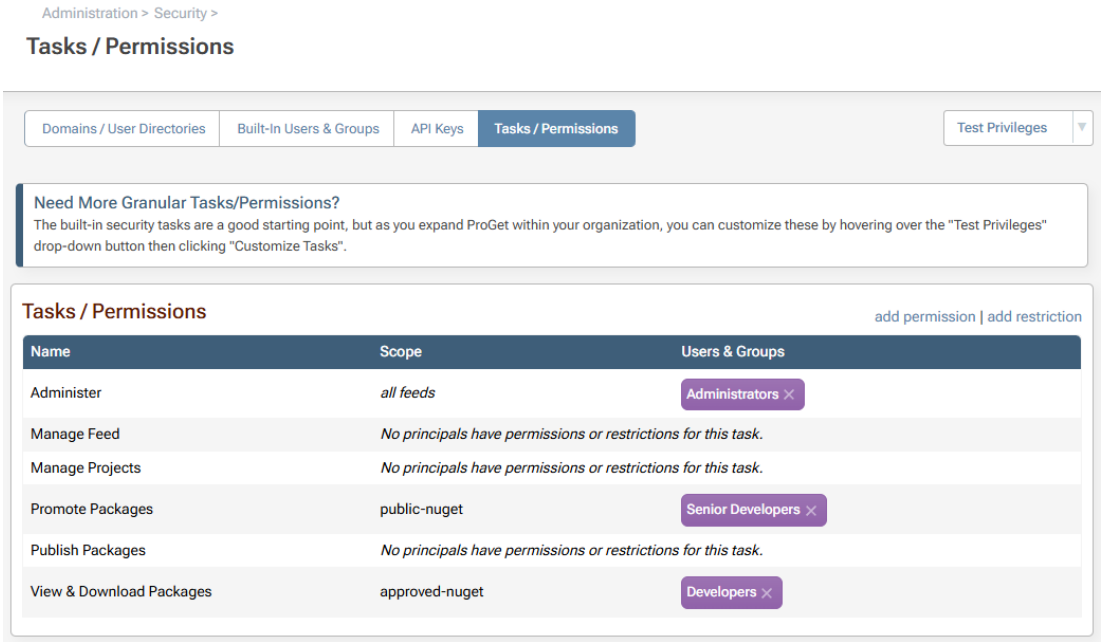
After vetting a package for quality, acceptable licenses, and vulnerabilities, you can promote it to the approved feed.



Now, the package will be in the approved feed, ready for developers to download and use.



You may also want to configure permissions so developers can only view or download packages from the approved feed.



Read a detailed breakdown in our How To: Approve and Promote Open-Source Packages guide.

Avoid Risks With an Approval Workflow

Using packages directly from NuGet.org exposes your projects to unnecessary risk. Vulnerabilities, restrictive licenses, and low-quality code can sneak in unnoticed—putting your security and compliance on the line. And with no clear approval process, developers are left deciding what's safe to use.

By implementing a package approval workflow, you catch vulnerabilities and license issues early—before they impact your projects. With ProGet, you can manage separate feeds for unapproved and approved packages, promoting only those that pass your review and are ready for production.

How to Filter Unwanted Packages from NuGet.org

Every NuGet package you install pulls someone else's code into your software. And with it—risks. Vulnerabilities, license issues, bad code. Not just once, but every time it updates. One bad change can lead to security breaches, compliance failures, or broken builds—without you even noticing.

The simplest way to manage NuGet risk at scale? Block bad packages at the source. Set a filter once, and automatically stop anything with security risks, incompatible licenses, or poor-quality code—before it ever reaches production. No more rechecking dependencies across dozens—or hundreds—of projects. Just clean, secure code.

This chapter covers how filters work, how they support a package approval workflow, and how to set them up in ProGet—so you can implicitly allow or block NuGet packages, keeping your builds secure and compliant.

What is a Connector Filter?

When managing NuGet packages from external sources—whether that's public registries like NuGet.org or other third-party feeds—filters help you control which packages are allowed into your projects. This matters because every new package or update carries potential risks—security vulnerabilities, license conflicts, or unstable code—that can quietly slip in and compromise your builds.

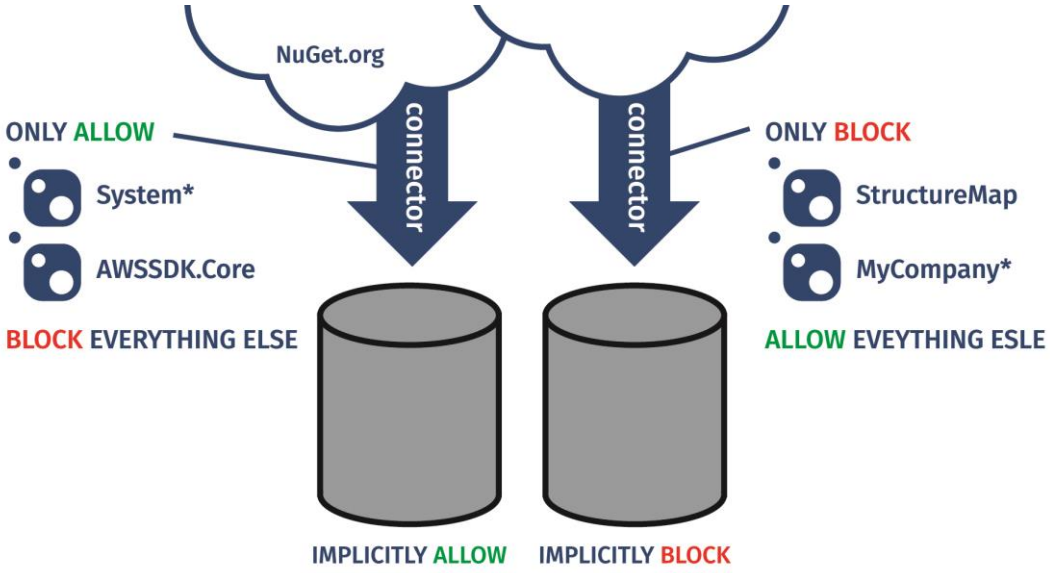
Conceptually, a package filter is straightforward: you specify package names (using exact names or wildcards like "*") and set rules to either allow or block matching packages. These filters let you implicitly allow or block:

- All versions of a specific NuGet package
- All versions of all packages sharing a certain prefix (for example, all packages starting with your organization's name)

With filters in place, you can automatically allow NuGet packages from trusted sources, so you don't have to verify or scan them every time they're updated. For example, a package like AWSSDK.Core is updated frequently—sometimes twice a week. Without filters, your team risks overlooking a vulnerable or unstable update, or spend a lot of time manually reviewing every version. Filters also help block packages you know are untrustworthy, reducing risks such as dependency confusion attacks that can silently introduce compromised code into your software supply chain.

If your team already uses a manual package approval process, filters make a great compliment. Manual approvals **explicitly** allow or block single versions of packages, whereas filters **implicitly** apply those decisions across all versions—or entire namespaces—helping prevent risky packages from slipping through with every update.

For instance, you could create a filter to allow all System.* packages, which are part of .NET's core libraries and regularly updated, or block every version of a deprecated package like StructureMap, preventing its use entirely.



Filters are a powerful way to control what gets into your feed—but they're most effective when paired with a package approval workflow. Together, they give you stronger control and flexibility—helping you stay ahead of the ongoing risks without the need for constant manual checks.

How Filters Fit Into a Package Approval Workflow

You don't *need* a full-on package approval process to start using filters—but having one seriously levels up your protection against all the random risks that come with NuGet packages and their updates. Think of it like a package version of a code review. It could be as chill as a quick "hey, is this cool to use?" or something more official like a team review meeting.

Having a clear workflow makes sure every package your team uses actually meets your quality, security, and licensing standards *before* it hits production. Plus, there's less back-and-forth, faster access to approved packages, and no cutting corners on safety.

If your team's more laid-back or just hasn't set up a formal process yet, there are basically two ways you can still manage the risk:

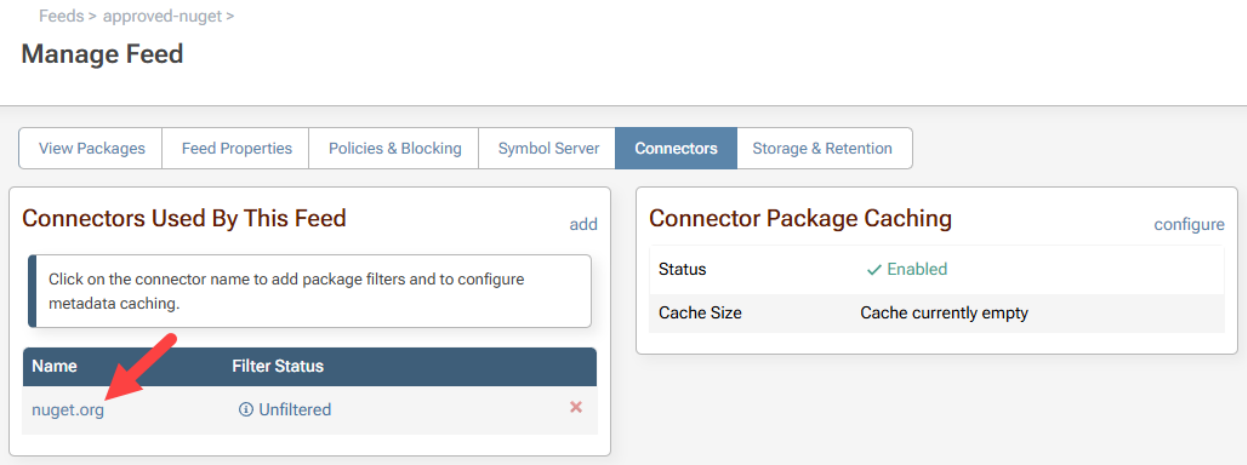
1. Create a highly-restrictive private NuGet feed that only has packages from trusted prefixes (like Microsoft.* or System.*)
2. Create a more open feed that only blocks packages from specific sources (like MyCompany.* or deprecated libraries)

Filters are a powerful tool in this process—but they are not a replacement for a solid package approval workflow. Managing a large number of filters can get messy over time, and since filters apply across *all* versions of a package, rules may need regular updates as packages evolve.

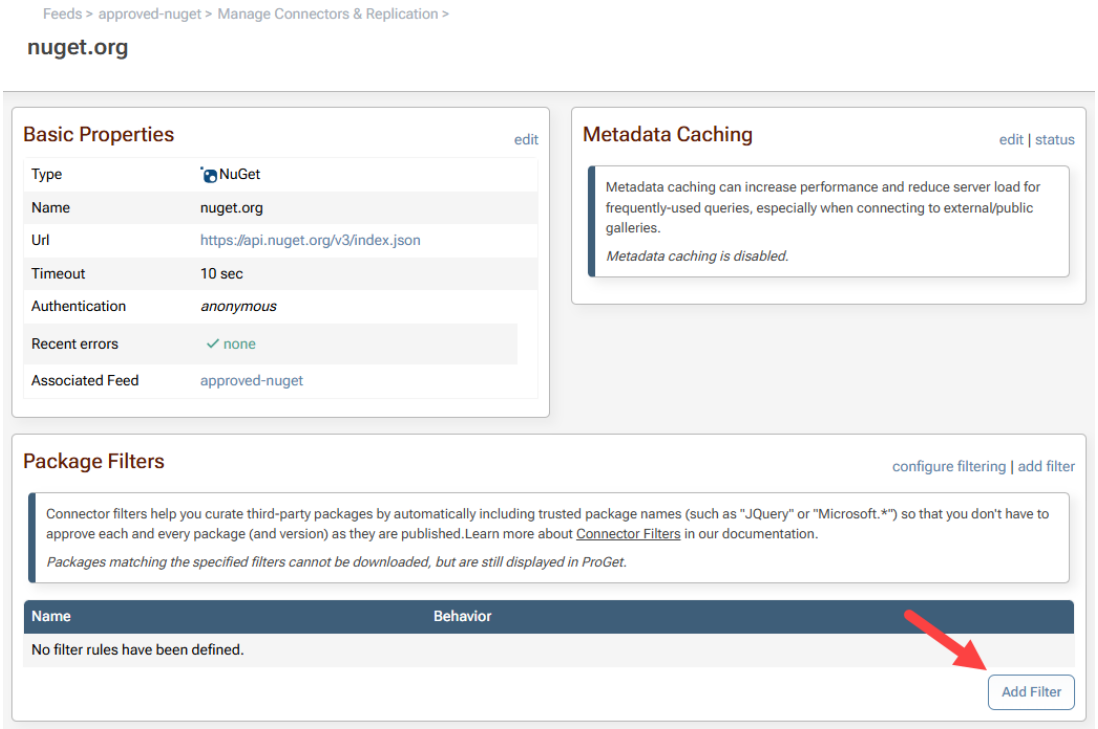
Used together, filters and a simple approval workflow give you a scalable way to manage ongoing risks—helping you maintain control without adding too much overhead. Tools like ProGet simplify this by making it easy to create and manage filters while integrating them into your team's approval process.

Setting Up a Connector Filter in ProGet

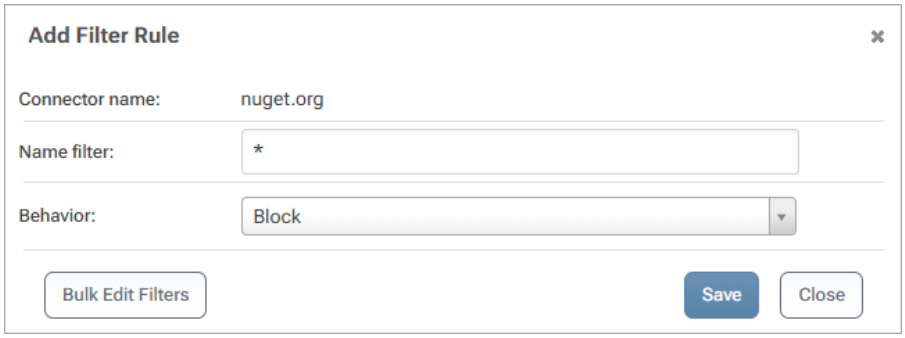
With ProGet, you can run your own NuGet server and decide exactly which packages get in. One powerful feature? Connector Filters. Here's how to set one up. First, navigate to your NuGet feed in ProGet and click on "Connectors".



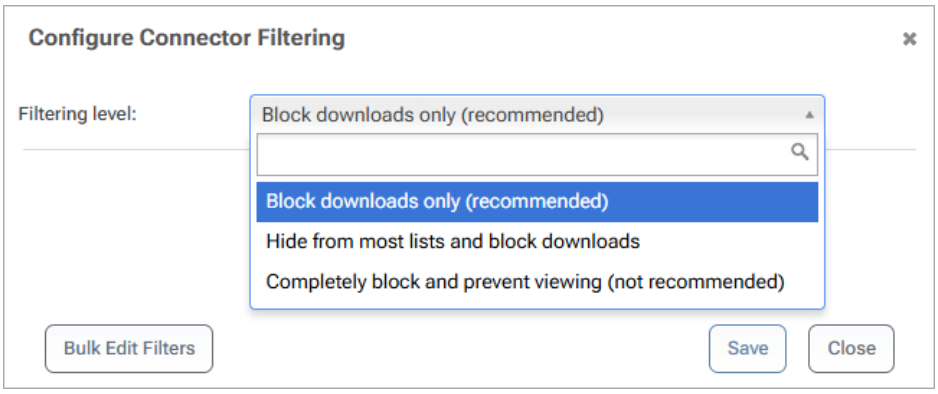
Any current rules this feed may have will be shown at the bottom of this page. To create a new allow/block rule click "Add Filter."



By default, ALL packages are allowed. So, in order to only allow packages you want, you must first create a rule that blocks all packages.



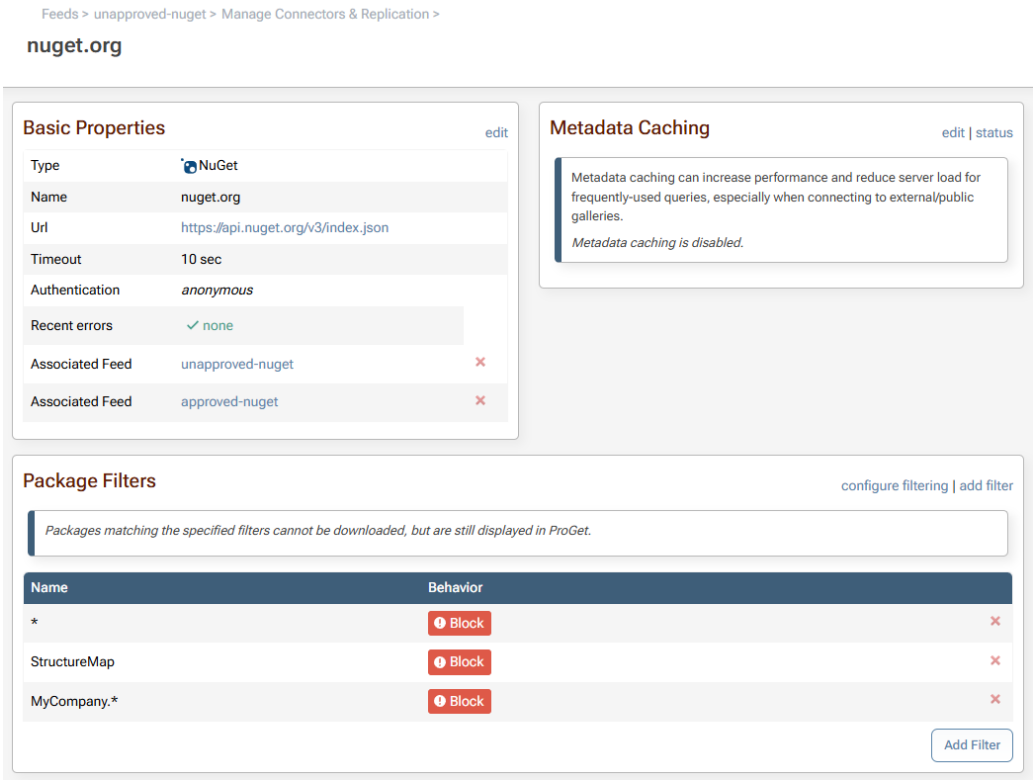
You can also configure what it means when a package is “blocked” by clicking “configure filtering.” In all cases, a blocked package can’t be downloaded, but you can partially hide it (i.e. you can navigate to the package in the UI directly by URL), or totally hide it (you can’t even browse to it).



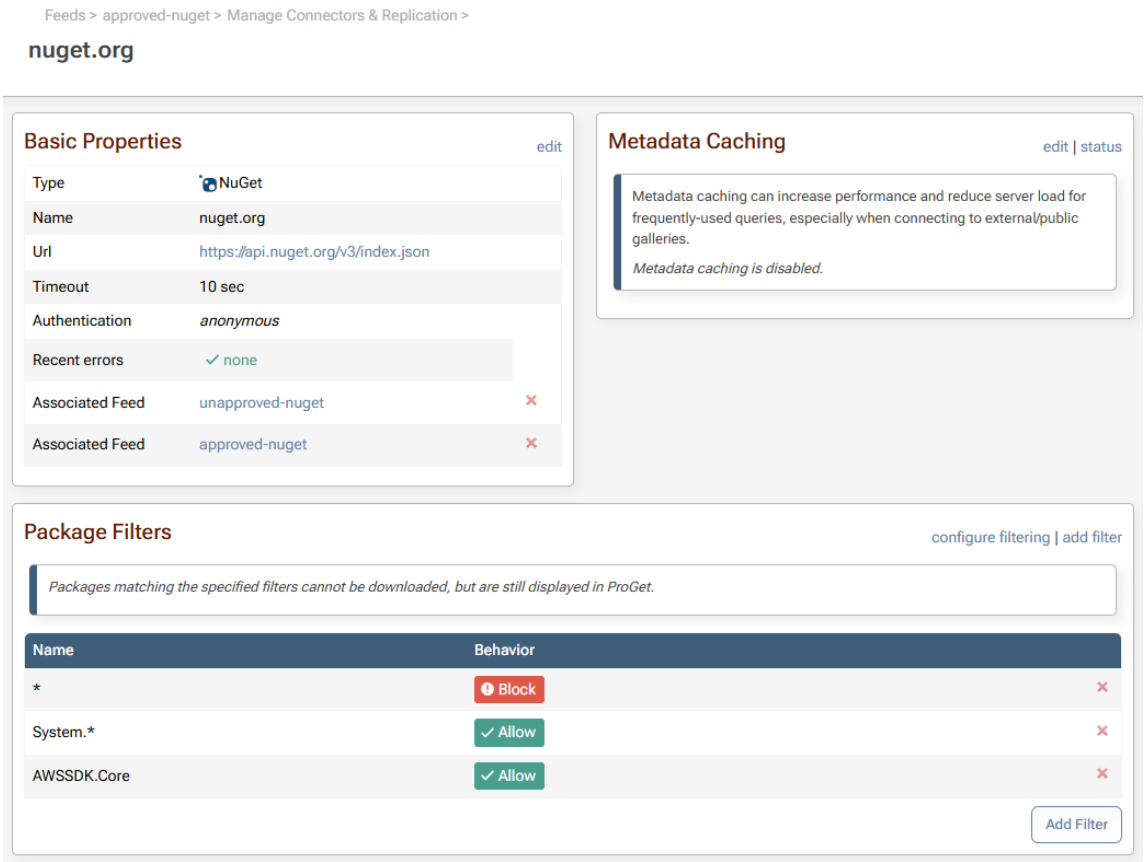
Filters enable implicit approval so you can implicitly allow or prohibit:

- All versions of a specific package
- All versions of all packages with a certain prefix (i.e. organization)

Using Connector Filters in conjunction with an approval workflow allows you to filter the connector on your "unapproved-nuget" feed with packages that will **never** be approved (e.g. StructureMap, MyCompany.*).



You are also able to add a filtered connector to your "approved-nuget" feed with packages that will **always** be approved (e.g. System.*, AWSSDK.Core).



Package Filter Best Practices

💡 Block Your Company's Prefix (e.g. MyCompany.*):

All the packages in your organization should start with the name of your company (e.g. MyCompany.*), and that means there should be no legitimate package that starts with "MyCompany." on NuGet.org.

At best, it could be a mistake. At worst, it might be a dependency confusion attack or another way to get seemingly legit packages snuck into your supply chain.

💡 Don't Add Rules for Infrequently Updated Packages (e.g. Newtonsoft.Json):

`Newtonsoft.Json` is another package engineers need access to. However, since it's only updated twice a year, it's much more reasonable to have it manually reviewed.

💡 Allow Integral Packages That are Frequently Updated (e.g. AWSSDK.Core)

`AWSSDK.Core` is an integral package engineer need access to and is updated up to twice a week. That would be a lot of manual approval. Using a Connector Filter is the perfect way to balance package verification and an approver's schedule.

💡 Allow Packages with Trusted Prefixes (e.g. System.*):

With packages hosted at NuGet.org. Organizations can apply for an ID Prefix Reservation. Once approved, only that organization can publish packages with that prefix. "System." is reserved by Microsoft, and it refers to a package integral to .NET and therefore can be trusted.

Side Note About Prefixes: Orgs can now reserve an ID prefix, which means only they can use it for their packages once it's approved. This is a newer thing, though, so there are some older packages that got "grandfathered in" and still use names that technically break the rule. Changing them now would just cause chaos, so NuGet leaves them alone.

For example, System.Linq.Dynamic.Core is by zzzprojects, not Microsoft even though Microsoft now owns the "System" prefix. This could be a good case for implicitly blocking that name.

Filter Your NuGet Stress Away

Relying on open-source NuGet packages means taking on constant, invisible risks. What starts as a simple install can quietly open the door to vulnerabilities, license conflicts, or unstable code—especially as packages evolve over time. Without safeguards, it's far too easy for those risks to creep into your builds unnoticed.

That's where filters come in. They give you a simple, automated way to control what gets through—blocking problematic packages before they ever become a problem. Combined with a package approval workflow, filters help you maintain secure, compliant, and reliable builds—without the overhead of constant manual reviews. Easily managed with ProGet, filters let your team build securely without slowing down.

Manage NuGet Dependencies with Lock Files and Package Consumers

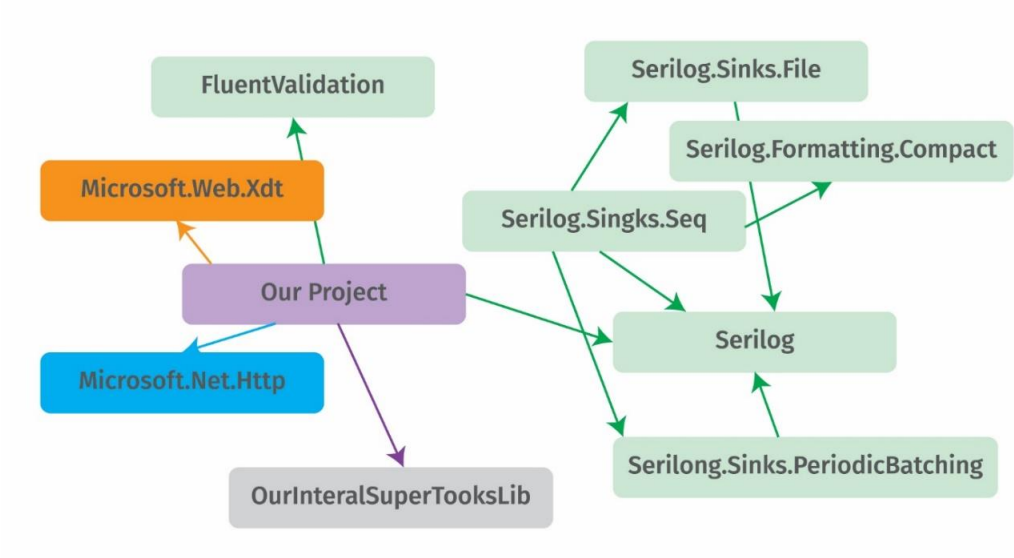
It worked when I built it—or the classic—*well, it works on my machine*. You've probably heard (or said) these yourself, usually right before an endless spiral of debugging and frustrated sighs. More often than we'd like, NuGet packages and their dependencies are the sneaky culprits behind these headaches.

NuGet package dependencies might seem simple, but under the hood, they can get real messy, real fast. If you're not keeping an eye on things, NuGet might resolve them in a way that leads to **unwanted NuGet packages** being pulled into your code, maybe even into production.

In this chapter, I'll explain how unwanted NuGet packages can sneak into your code, how to use Lock Files to ensure that your applications only use your approved NuGet packages, and how to use Package Consumers to quickly identify which applications are using a specific package so you can update them when necessary.

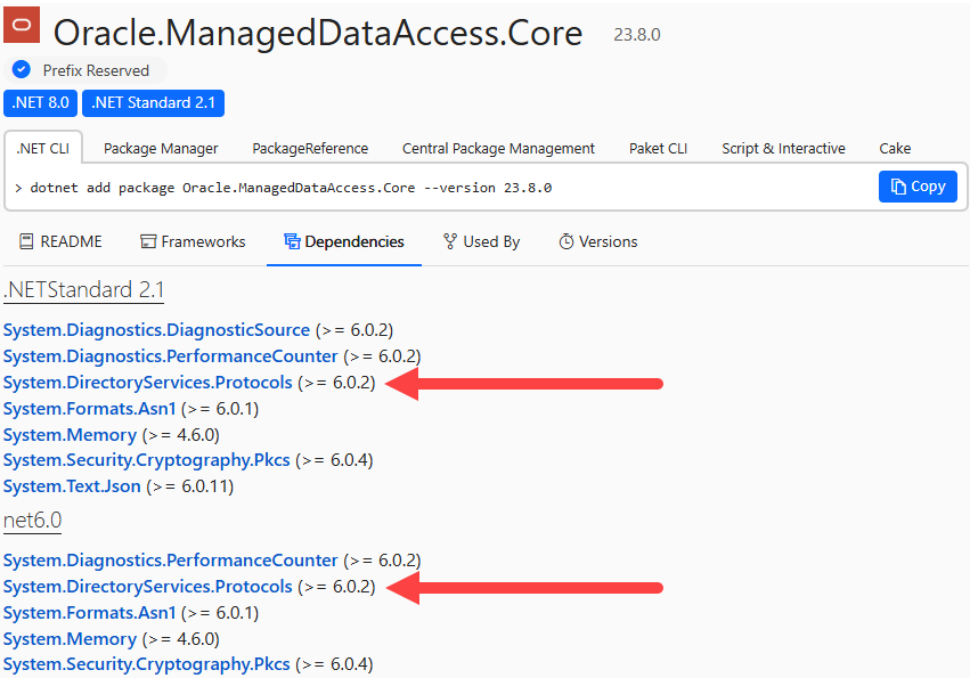
The Unintended Side Effects of Dependency Resolution

Most NuGet packages depend on other packages, meaning every time you install one, you're also growing a whole dependency tree that must be resolved for your project to build.



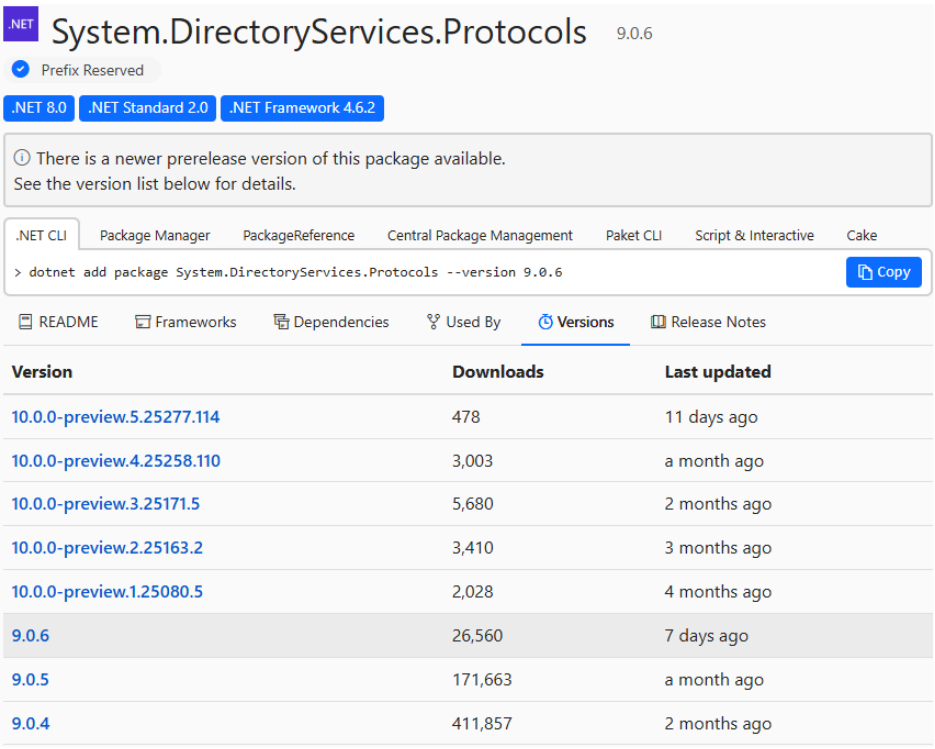
As you can see, things can get tricky fast—especially since even the wrong version of the right package will prevent dependency resolution. This is typically handled using **version ranges**.

Version ranges let you define a range of versions your package will accept. Super helpful for sorting out dependency trees—but they can come with some unintended side effects if you're not careful.



Let's say you are using the package Oracle.ManagedDataAccess.Core 23.8.0. This package is dependent on the System.DirectoryServices.Protocols package. To resolve this dependency, a version range is used to tell `Oracle.ManagedDataAccess.Core` to accept any version of `System.DirectoryServices.Protocols` that is greater than or equal to version 6.0.2.

Pretty convenient, right? It is. But it comes at a cost!



Take a look at the version history for the `System.DirectoryServices.Protocols` package. Version 9.0.6 dropped seven days ago (as of the time of writing), with version `10.0.0` already in the works, and version `6.02` a lot further down the list. Since the version range for `Oracle.ManageDataAccess.Core` is set to accept any version north of `6.0.2` inclusive, your application would automatically include a new package—perhaps, an unwanted one.

Letting in unexpected third-party packages opens the door to two common issues:

1. Builds that suddenly break and leave you saying, "But it worked yesterday!"
2. Unsafe packages sneaking into your code.

Version ranges definitely make dependency resolution easier, but they can introduce chaos if you're not careful. The good news? You can keep things under control with lock files.

Best Practice: Use Lock Files for Repeatable Builds

Lock files basically "lock down" every package version in your whole dependency tree at the moment you create the file. They give you a full snapshot of all the NuGet packages your app is using.

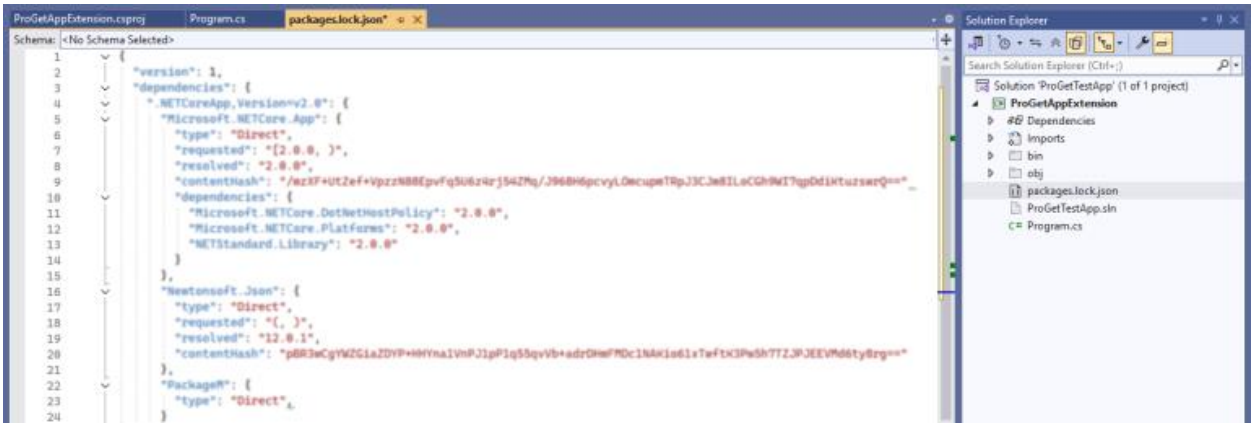
Why is this so helpful?

When the NuGet client installs dependencies at build time, it grabs the latest version it can find. That might not match what you originally intended when you were developing. If you don't lock those versions down, your build could end up different the time you—or someone else on your team—builds it.

To enable the use of lock files with NuGet, set the MSBuild property `RestorePackagesWithLockFile` in your project file:

```
<PropertyGroup>
<RestorePackagesWithLockFile>true</RestorePackagesWithLockFile>
</PropertyGroup>
```

If this property is set, NuGet `restore` will generate a lock file – `packages.lock.json` file at the project root directory. Besides locking in the NuGet package versions, the lock file will also show all the packages your application depends on.



Lock files solve the massive headache of unpredictable NuGet package versions sneaking into production… but they don't eliminate the problems with dependencies completely.

Use Lock Files in .NET 9 for Increased Reliability

And get this—in .NET 9, lock files become even more powerful thanks to an updated NuGet restore algorithm. This new resolver was completely rewritten to better handle complex networks of dependencies, building dependency graphs more efficiently and deterministically to significantly improve restore performance and memory usage—basically, you get **faster, identical restores every time**—*and that's especially valuable for your enterprise-scale apps and project-crammed repositories.*

Now locked-mode restores are super reliable, since running `RestorePackageWithLockFile` along with the `--locked-mode` flag, the NuGet CLI sticks to dependency versions specified in your lock file, failing anything that's out of sync. This prevents unexpected changes from sneaking into your builds, making lock files a must-use feature in .NET 9 for scaling teams and projects.

Watch out for New Dependencies

Remember, dependencies can—and often will—have dependencies. And these dependencies can change over time, meaning that when you update, you can find yourself with totally different

packages than before. And yeah—those packages might bring along some license issues, vulnerabilities, and quality risks.

The best way to mitigate this risk is by setting up a package approval workflow that only allows approved packages to be used by developers and build servers.

Watch out for New Vulnerabilities

Even after you've got NuGet lock files in place and a solid workflow to stick to approved packages, there's always a chance a new vulnerability pops up in one of your dependencies.

This vulnerability could then impact any applications that use it. So what can you do?

You could manually inspect the lock files of all your applications to see which ones are using the unwanted dependency. That's a lot of repositories to check out and a lot of searching.

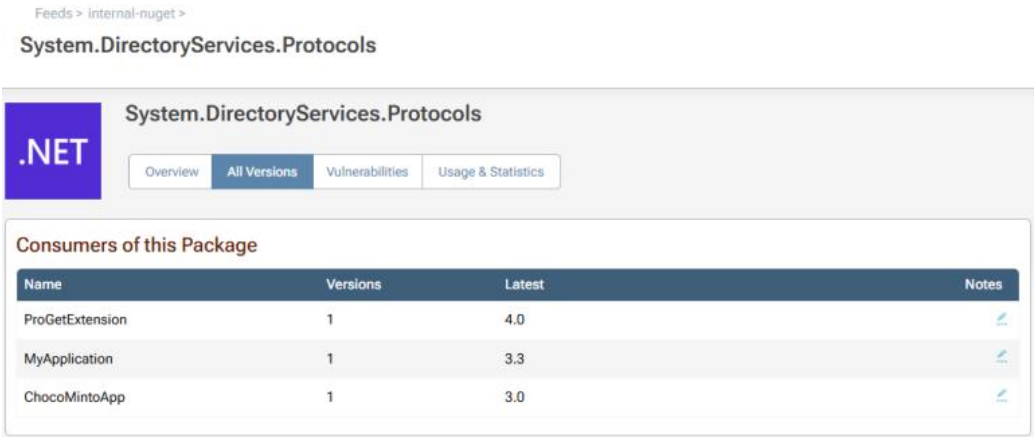
Another option that pairs well with lock files is NuGet Audit. Introduced in .NET 8 and improved in .NET 9, this feature scans your project's dependencies—including transitive ones—for known vulnerabilities, pulling data from the GHSA. Since lock files provide an exact snapshot of every package and version, using them makes NuGet Audits results far more reliable. With a lock file in place, you'll know exactly which vulnerable packages are being flagged, without worrying about inconsistencies after restores.

But you can also use **ProGet's** Package Consumer Feature to do this automatically.

Best Practice: Use Package Consumers to Track Dependencies

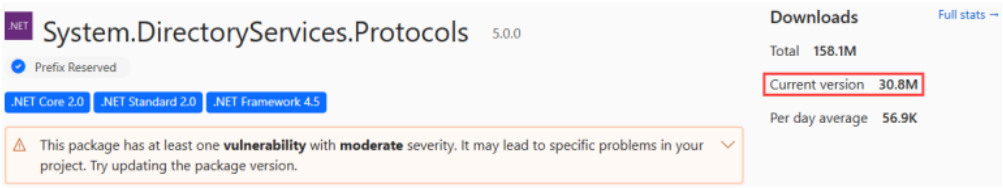
ProGet's Package Consumers feature shows all the applications that are "consuming" or using a specific package. So, let's say the package dependency in question is *System.DirectoryServices.Protocols 9.0.6*.

After building your application, ProGet's Package Consumers feature kicks in, using pgscan to scan the build output, track down exactly which package versions your app is using, and then publish that data—plus your app's name and version—back to ProGet.



With Package Consumers, we can see that `System.DirectoryServices.Protocols 9.0.6` is being used in the applications `ProGetExtension`, `MyApplication`, and `ChocoMintoApp`. Now you know exactly which apps will be impacted—and can jump in to make the fixes, fast!

By the way... If you were wondering about that `System.DirectoryServices.Protocols` package, it's not just an example for theoretical purposes. Back when `System.DirectoryServices.Protocols` was in version `5.0.0`, it was downloaded around **17.6M** times before it was discovered to have a vulnerability, and the download number has risen to **30.8M** since then!



Anyone with an app using a package dependent on version 5.0.0 now has a vulnerable package in their code, and anyone in that position definitely wants to know which applications make use of this package as quickly as possible. Of course, that's exactly why we created **package consumers** 😉.

NuGet in the Enterprise

NuGet packages and dependency issues go hand in hand—an endless network of packages within packages almost guarantees a conflict somewhere, leading to broken apps and teams of frustrated developers.

Utilizing NuGet lock files will reduce dependency conflicts, but it's not enough, since versions ranges mean vulnerable packages can still slip through. Luckily, **ProGet's** package consumers let you track which of your apps are using those packages, letting you clean up problems fast.

How to Debug NuGet Packages with Symbols and Source Link Painlessly

Debugging private NuGet packages is a serious headache. Honestly, it's one of the biggest reasons dev teams hesitate to break up their monolithic .NET solutions. Unlike a regular referenced library project, you can't easily step-into a NuGet package's code and start debugging. And if you need to just tweak a method? Yep—time to rebuild the whole package. At scale, this slows down development and increases friction in the inner loop.

Luckily, there's a simple solution to this problem: **NuGet Symbol Packages**. To make debugging your NuGet packages less of a nightmare, the real game-changer is setting up a Symbol Server, and optionally Source Link. With `.snupkg` files—or symbol-enabled `.nupkg` files— pushed to a symbol server, you can pull `.pdb` files and source maps, to step-through your package's code in Visual Studio without the usual hurdles.

In this chapter, we'll break down Symbols and Source Serving (Source Link) and how to create NuGet Symbol Packages with Source Link. We'll also look at Legacy Symbol formats and packages and cover configuring Visual Studio to debug Symbol packages.

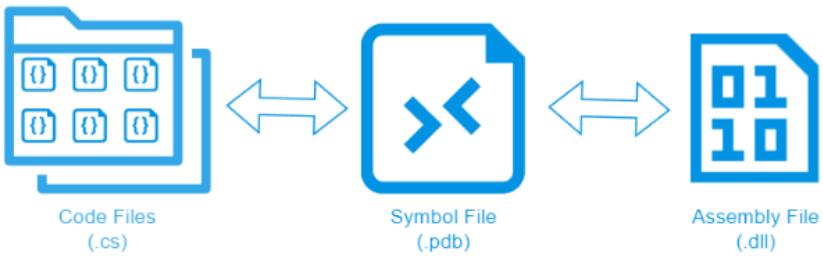
Demystifying Symbols & Source Serving (Source Link)

The secret sauce behind debugging NuGet packages? It's all about Symbols and Source serving. There's actually four different technology "layers" working together to make this happen.

But I won't lie—*it's complicated*. It took me a little while to wrap my head around how everything fits, especially after reading a bunch of outdated (and sometimes just plain wrong) articles. In any case, I'll do my best to walk you through each piece and explain how they all connect!

What the heck are Symbols, anyway?

You've likely noticed the `.pdb` files in your bin directory after building your project—right next to your `.dll` or `.exe` files. These are your **Symbol files**, and they basically map compiled assemblies (i.e., the `.dll` and `.exe` files) to the original source code (i.e., the `.cs` files) that were used to build them.



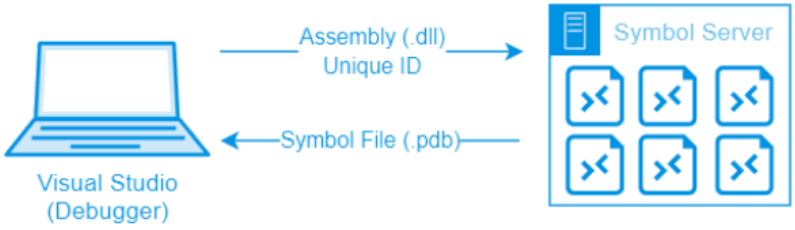
Visual Studio uses these .pdb files to let you step-through your code while it's running in debug mode on your machine.

But .pdb files are useful even outside of interactive debugging, providing way more context in stack traces when something crashes—like showing you the full disk path to the original source code, even when your application errors on a different machine.

What is a Symbol Server?

Fun fact: Symbol files actually predate NuGet—and .NET as a whole—by at least a few decades. Designed in the era when disk space was at a premium, this meant Symbols were really shipped with software, which was great for saving space, but terrible for debugging.

That's where **Symbol Servers** came in. When you build a library, the compiler embeds a unique identifier (basically a GUID) into both the `.dll` and the `.pdb` files. If you upload the `.pdb` to a Symbol Server, tools like Visual Studio can later fetch it using that unique ID embedded in the `.dll` file.



That's how a debugger like Visual Studio can load up Symbol files when your app crashes—you don't need to have every single `.pdb` file sitting on your machine. Visual Studio just grabs the right one on demand from the Symbol Server, using that embedded ID.

What are NuGet Symbol packages?

A **symbol package** is basically just a NuGet package that contains symbols (`.pdb` files) instead of binaries (`.dll` files) and some different metadata. They also have a different file extension `.snupkg` instead of `.nupkg`.

The nuget.exe or dotnet CLI creates a Symbol package with your normal NuGet packages when you specify the `SymbolPackageFormat` argument. For example:

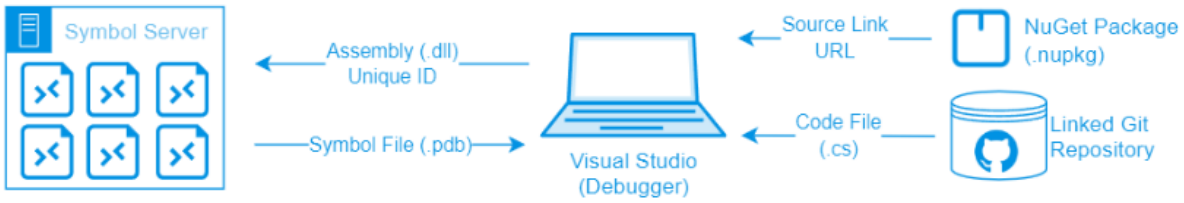
```
nuget pack MyPackage.csproj -Symbols -SymbolPackageFormat snupkg
dotnet pack MyPackage.csproj -p:IncludeSymbols=true
-p:SymbolPackageFormat=snupkg
```

Running one of these commands creates `MyPackage.nupkg` and `MyPackage.snupkg` in your working directory, one with the `.dll` files and one with the `.pdb` files. Your regular .nupkg can then be pushed to a NuGet feed, with your Symbols `.snupkg` package being pushed to a Symbol Server.

What is Source Link?

Remember how .pdb files map your compiled assemblies (`.dll` and `.exe`) back to the original source code (.cs files)? Well, that "map" only works if the debugger can find the exact version of the source code used when the assembly was built. So if you're using a symbol server... where do the code files come in?

This is where Source Link comes in. When you build a NuGet package with Source Link enabled, a Git repository URL and commit ID will be embedded right into the package metadata. That way, when you're debugging, Visual Studio knows where to locate the required code files.



Enabling Source Link in your own .NET project means setting a few properties and then adding a NuGet package specific to your NuGet repository (e.g. `Microsoft.SourceLink.GitHub`).

Legacy Symbol Format & Packages

Believe it or not, NuGet Symbol and Source Serving was *even messier* a few years back, and if you're poking around older packages, you might run into some of those legacy formats, so I'll give you a quick rundown to help them make sense.

Legacy Symbol Packages (.symbols.nupkg)

These days, `.snupkg` Symbol packages are the norm—but you might still come across the older `.symbols.nupkg` format. These legacy Symbol packages usually bundle up the `.pdb` files, source code, and the compiled `.dlls`... which defeats the point of separating things cleanly.

Microsoft PDB (Legacy Symbol Files)

Not all .pdb files are created equal. There are actually two kinds—Microsoft PDB (aka Windows PDB) and Portable PDB. They might look the same, but under the hood, they're totally different file formats.

Microsoft PDBs go way back—like early '90s old. They've been used for everything from native Windows drivers to classic .NET Framework assemblies, but it only works on Windows and was never intended with cross-platform (Linux) debugging in mind.

Portable PDBs, on the other hand, are a fresh rewrite from the .NET team. As the name suggests, they're portable—so they work across platforms like Linux and Windows. The catch? They only work with .NET libraries, which means if you're building native (C++) libraries for Windows, you'll be sticking to the Microsoft PDB format.

These days, if you're working with NuGet packages, .NET 9.0, or .NET Core or later, you likely won't come across too many Microsoft PDB Symbol files.

Legacy Source Server

Before Source Link became the go-to for NuGet packages, an older system called Microsoft Source Server was used to pull the exact version of the source code used to build a library. These days, it's mostly obsolete, built for a *very* different time when Git and web-based source control weren't a thing. But if you're using TFVC or SVN, it's basically your only option.

How to Debug NuGet Packages with Visual Studio

So making NuGet Symbol packages is pretty easy, since you just specify the `SymbolPackageFormat` when running the nuget.exe or dotnet CLI:

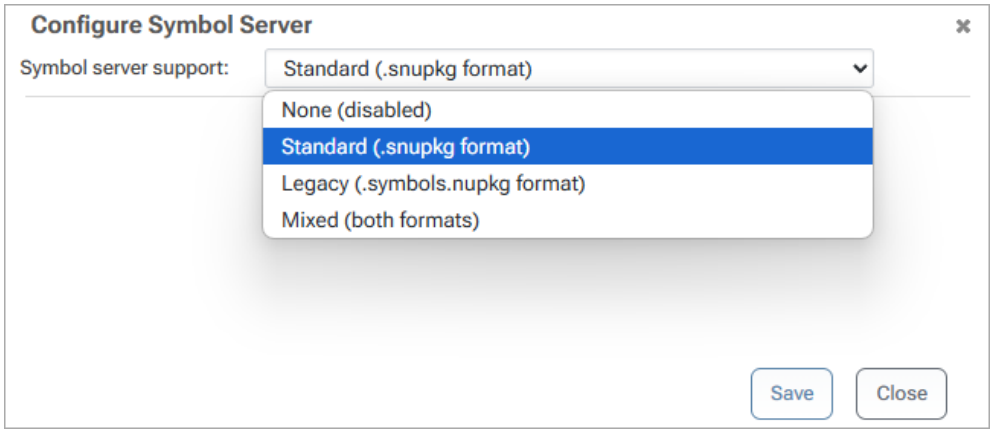
```
dotnet pack MyPackage.csproj -p:IncludeSymbols=true
-p:SymbolPackageFormat=snupkg
```

From here, you just need to publish your `.nupkg` package to a private NuGet feed and your `.snupkg` to a Symbol Server. Of course, you'll need a capable private NuGet Server.

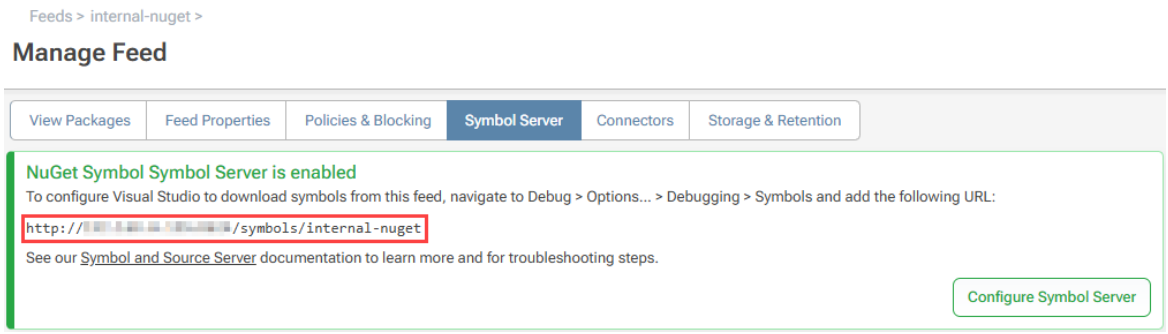
I'll use ProGet to show how this works, but the concepts are similar in other tools.

Configuring your NuGet Feed for Symbol Serving

ProGet has an integrated Symbol Server on its NuGet feeds, and you can choose the type of Symbol packages (**modern** and **legacy**) supported.

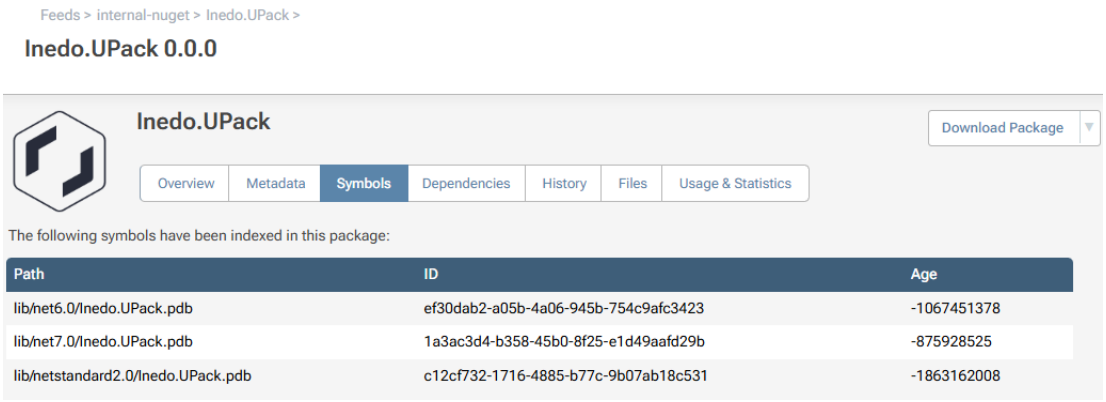


Once set up, you'll get a Symbol Server URL to configure Visual Studio:



Verifying Indexed Symbols

Now, you'll want to verify that your Symbol files were properly indexed. You can find this on the Symbols tab of a NuGet package in ProGet:

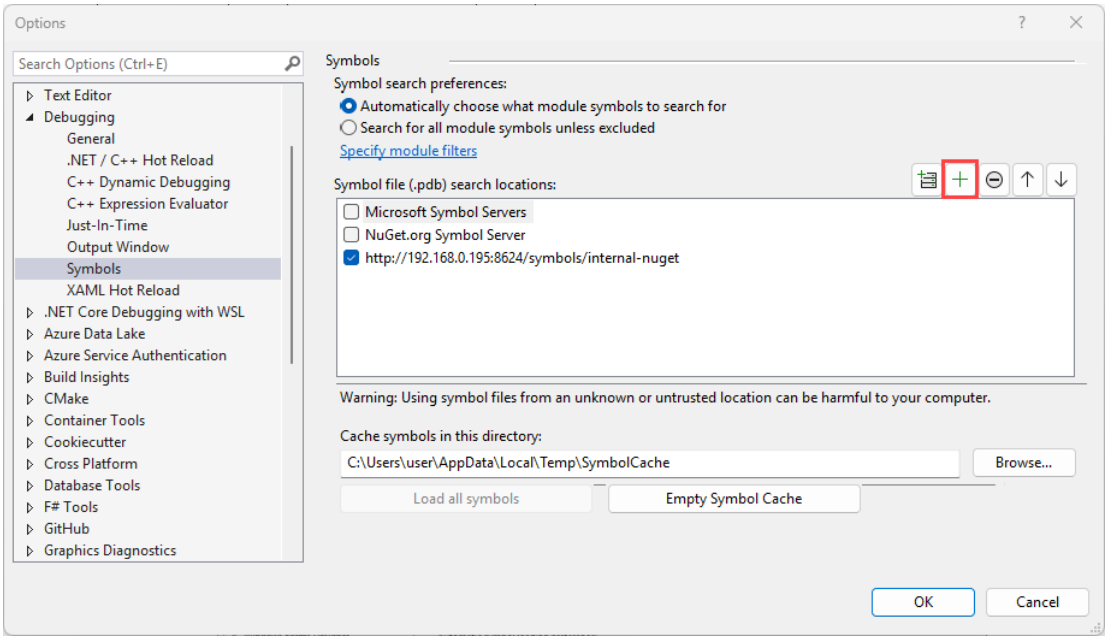


Now you can see the unique identifiers (**ID** and **Age**) Visual Studio uses to retrieve a `.pdb` file.

Configuring Visual Studio to work with a Symbol Server

To debug NuGet package libraries, Visual Studio needs to use the Symbol Server, which is where your Server URL from earlier comes in.

In Visual Studio, select **Debug > Options**... from the menu bar, then navigate to **Debugging > Symbols** from the tree menu. Then add (**+**) your ProGet Symbol Server URL and specify a Symbol Cache Directory. By default, Visual Studio will use `%LOCALAPPDATA%\Temp\SymbolCache`, but you may specify any path.



And that's it! For devs, it'll be as easy as configuring a private NuGet feed in Visual Studio.

Alternatively: Debugging NuGet Packages Without Symbols

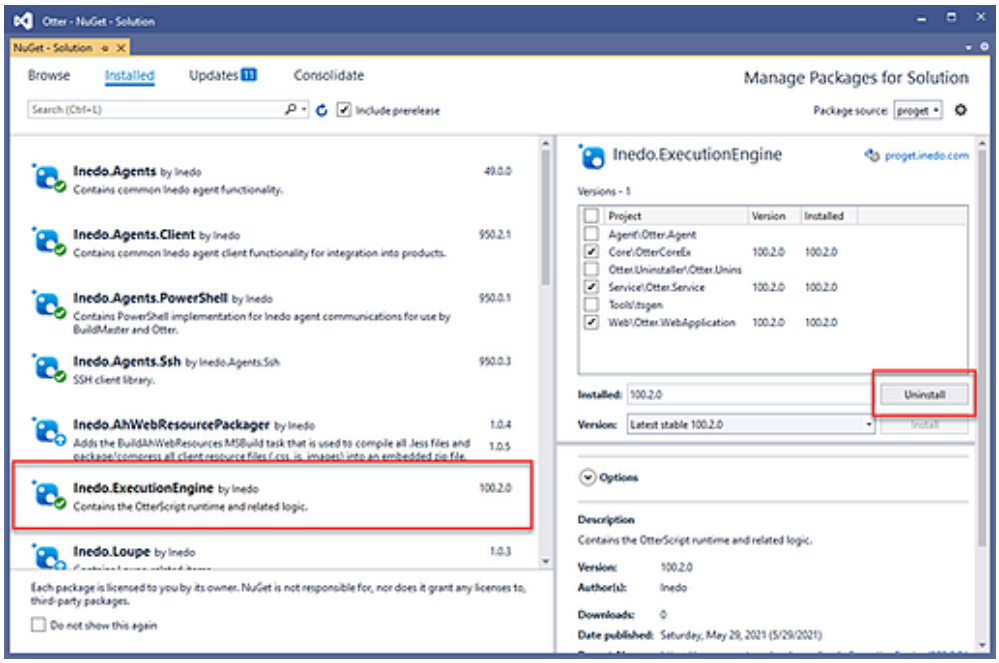
While NuGet Symbol packages offer an efficient, integrated solution for debugging and development, something I call **"munging"** project files is a pretty simple alternative solution.

Munging project files means temporarily incorporating the code from the library project into the app you're gonna debug, especially useful for situations where you want to write or edit the library code.

This can be done in a few simple steps, and I'll show you an example where I need to debug/edit the `Inedo.ExecutionEngine` library, but I want to do so while using it in an application **Otter**.

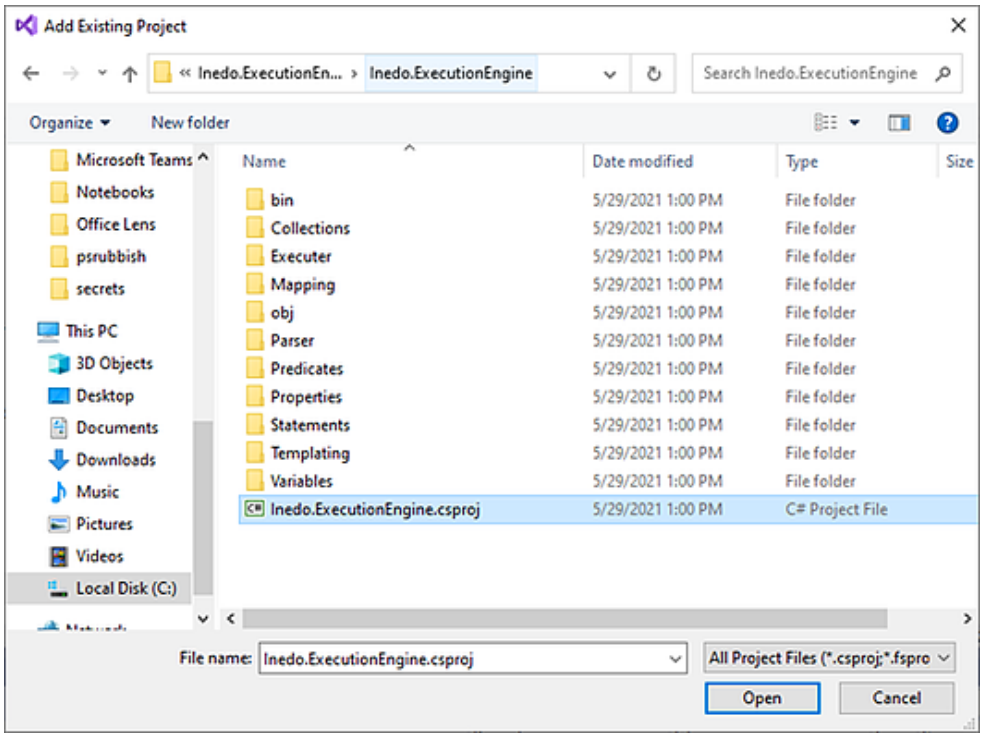
Step 1: Uninstall the Packages Reference

In the Otter solution, `Inedo.ExecutionEngine` was installed in three different projects, so I right-clicked on my Solution, then Manage NuGet Packages, selected the library, and uninstalled it from the projects.

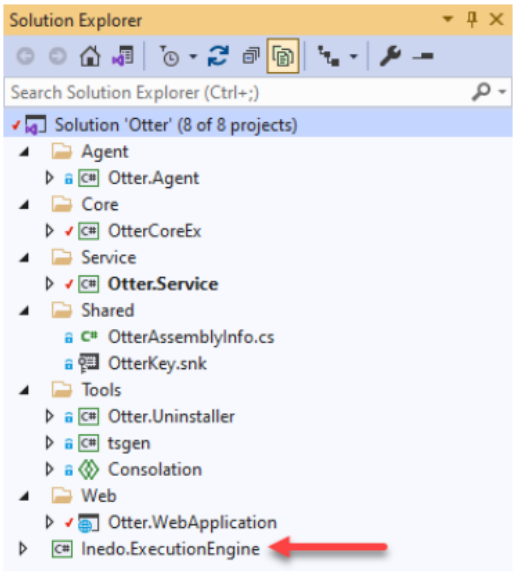


Step 2: Add Library Project

I right-clicked on my Solution, and selected **Add Existing Project.**

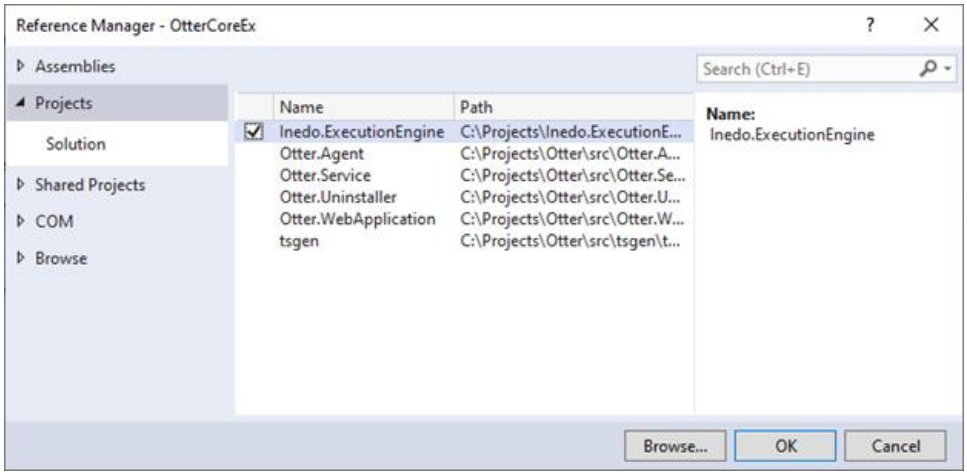


After adding the project to the solution, it will show up at the bottom like this:



Step 3: Add Project References

On each of the three projects, I uninstalled `Inedo.ExecutionEngine` project reference, I added a project reference to my newly added library project (`Inedo.ExecutionEngine`).

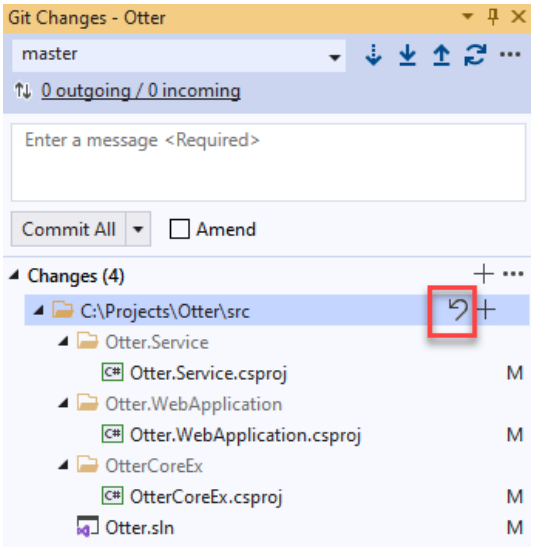


Step 4: Debug & Edit Code As Normal

And that's that! `Inedo.ExecutionEngine` can just be debugged and edited like all other projects on my solution! Note that you won't be able to commit source control changes to the munged library in the same instance of Visual Studio. But you can always commit your library changes later.

Step 5: Undo Project "Munging"

⚠ Once you're done, make sure to undo the project "munging" to your library. Accidentally committing these changes will lead to broken builds and a headache for the rest of your team.



Project Munging with Tools & PowerShell

If you don't do this very often, it's not that big of a deal to click this many times. However, a lot of steps and clicks become a little too bothersome, there's a Visual Studio Extension that can do some of the work, but it's outdated and doesn't work on newer project types.

It's really simple to just write a very basic PowerShell script that uses the dotnet CLI.

```
# Munge-InedoLib.ps1

$pkgName = "Inedo.ExecutionEngine"
$pkgProjFile = "C:\Projects\Inedo.ExecutionEngine\Inedo.ExecutionEngine\
Inedo.ExecutionEngine.csproj"

$slnFile = "C:\Projects\Otter\src\Otter.sln"
$projFilesToMunge = @( `
       "C:\Projects\Otter\src\Otter.Service\Otter.Service.csproj", `
       "C:\Projects\Otter\src\Otter.WebApplication\Otter.WebApplication.csproj", `
       "C:\Projects\Otter\src\OtterCoreEx\OtterCoreEx.csproj" `
)

dotnet sln $slnFile add $pkgProjFile
foreach ($projFile in $projFilesToMunge) {
       dotnet remove $projFile package $pkgName
       dotnet add $projFile reference $libProjFile
}
```

Debug NuGet Packages With ProGet

Debugging NuGet packages is anything but intuitive. Using the solutions outlined in this article can help your developers to more easily integrate debugging with NuGet.

We recommend using NuGet Symbol Packages, especially if you're debugging library code. That said, **project munging** is a good alternative. ProGet supports these solutions so your developers can debug NuGet packages as painlessly as possible.

How to Make SBOMs and Why You Need Them

Software Bills of Materials (SBOMs) are becoming something every developer needs to deal with. If your organization uses NuGet packages you've probably heard of them, but finding guidance that actually applies to NuGet can be a bit of a pain, since most resources focus on other ecosystems like npm or Maven. For many teams, SBOMs feel like just another checkbox. They're easy to brush off or only half-do, especially when deadlines are tight and your backlog never ends.

But SBOMs aren't just busywork. When done right, they're a practical way to see what's actually in your code, track dependencies, and protect your software supply chain. That's especially important with NuGet, where even a single package can pull in dozens of transitive dependencies. Knowing exactly what's included and being able to document it can save you a lot of headaches during audits, security reviews, or when a critical vulnerability pops up.

In this chapter, we'll break SBOMs down from the ground up, explain why they matter, and walk you through how to create one for your NuGet packages. I'll talk about different approaches, why you might pick each one, and show exactly how to do it using tools like CycloneDX and ProGet.

What Is an SBOM and Why You (Will) Need It

Trying to figure out what's inside your software without an SBOM is like trying to remember every ingredient in a recipe after it's been cooked. It's nearly impossible without a clear list. If you're developing with NuGet packages, this is especially true. A single package can pull in dozens of transitive dependencies, each with its own version, license, and potential risks.

An SBOM is a detailed inventory that lists every component in your software, including all packages and their dependencies. This lets you know exactly what's in your project. It's usually built from a mix of open-source libraries, commercial components, and internal modules. That tangled web of dependencies can bring security vulnerabilities, license headaches, or regulatory challenges if you don't track it carefully. An SBOM helps answer key questions:

- *What's in our software?*
- *Where did it come from?*
- *Is it safe to use?*

SBOMs are also increasingly becoming a requirement. Policies like the U.S. Executive Order 14028 and frameworks such as NIST's Secure Software Development Framework (SSDF) expect organizations to provide accurate SBOMs for security and compliance.

What Does an SBOM Actually Look Like?

At a technical level, an SBOM is usually an `.xml`, `.json`, or `.yaml` file that lists all components, their versions, licenses, and dependencies. For example, here's a simplified CycloneDX SBOM for a NuGet project that includes a package with several transitive dependencies:

```
<?xml version="1.0" encoding="UTF-8"?>
<bom xmlns="http://cyclonedx.org/schema/bom/1.3" version="1">
  <components>
    <component type="library" bom-ref="pkg:nuget/Microsoft.AspNetCore.App@2.2.0 >
      <name>Microsoft.AspNetCore.App</name>
      <version>2.2.0</version>
      <licenses>
        <license>
          <name>MIT</name>
        </license>
      </licenses>
      <dependencies>
        <dependency ref="pkg:nuget/Microsoft.AspNetCore.Http@2.2.0"/>
        <dependency ref="pkg:nuget/Microsoft.Extensions.Logging@2.2.0"/>
        <dependency ref="pkg:nuget/Newtonsoft.Json@12.0.3"/>
      </dependencies>
    </component>
    <component type="library" bom-ref="pkg:nuget/Newtonsoft.Json@12.0.3">
      <name>Newtonsoft.Json</name>
      <version>12.0.3</version>
    </component>
  </components>
</bom>
```

Like the `Microsoft.AspNetCore.App` package above, NuGet packages often bring in a whole bunch of transitive dependencies. Even if you only install one package, it can pull in many others, each with its own version, license, and potential vulnerabilities.

This is a big part of what makes SBOMs important. They make it clear which NuGet packages are in your project, including the ones you don't add directly. There are a few different ways to actually generate an SBOM, each with its own trade-offs, and understanding the options makes it easier to pick the right approach for your NuGet projects.

What Are Your Options When Creating SBOMs?

To generate SBOMs for NuGet projects, you have a few options to choose from. Each solves the problem in a slightly different way, depending on your needs.

CycloneDX (Good for Quick, One-Off SBOMs)

CycloneDX is a CLI tool that scans your project's dependencies and generates a standards-compliant SBOM file. Once the file is created, you're done; there's nothing more to manage. This makes it good for small projects or situations where you just need a quick SBOM to share with an auditor or security team. It's fast, simple, and doesn't require any ongoing setup.

ProGet (Good for Tracking, Analysis and Compliance)

ProGet's SBOM features are good if you're looking for something more comprehensive. It works with NuGet to automatically scan all your packages every time you build or release, gathers all the details, and stores everything in one place. This makes it easy for your team to track what's in your software and catch any security or compliance issues early.

It also offers built-in Software Composition Analysis (SCA), helping you identify security vulnerabilities and license issues of any packages listed in a project's SBOM. This makes it a great choice for organizations that want to maintain up-to-date SBOMs, improve supply chain visibility, and automate much of the work involved in managing dependencies.

Manual Creation (Not Recommended)

Technically, you *could* create an SBOM by hand, listing every single package, version, license, and all those pesky transitive dependencies. But good luck typing out 100 packages and their dependencies without losing your mind, or making a mistake. Manual SBOMs are mostly only useful for tiny projects or just playing around. For anything real, you're almost always better off using CycloneDX or ProGet.

The easiest way to handle SBOMs for NuGet is to automate them as part of your build. Use tools that track every package, including transitive dependencies, and capture key details like versions and licenses. CycloneDX works well for quick, one-off SBOMs, while ProGet helps teams automate and maintain them long-term. Once you've seen what these tools can do, creating an SBOM for your NuGet project feels a lot more straightforward.

How to Create an SBOM For Your NuGet Projects

Now we've covered our options, here's how to actually generate SBOMs for your NuGet projects, whether you're using CycloneDX or ProGet. Before you generate an SBOM, **make sure you've built your project** so that all dependencies are restored.

Creating an SBOM with Cyclone DX

If you haven't already installed CycloneDX you'll need to start with that by running:

```
dotnet tool install --global CycloneDX
```

Then it's just a case of navigating to the root directory of the NuGet project you want to scan and running this command to create an SBOM in `.xml` format:

```
cyclonedx dotnet -o sbom.xml
```

This command scans your project, including all transitive dependencies, and outputs a structured SBOM file called `sbom.xml`. This will make sure even transitive dependencies are captured, so you get a complete view of your project.

Once the file is generated, you're done, but note that if your dependencies change, you'll need to regenerate the SBOM.

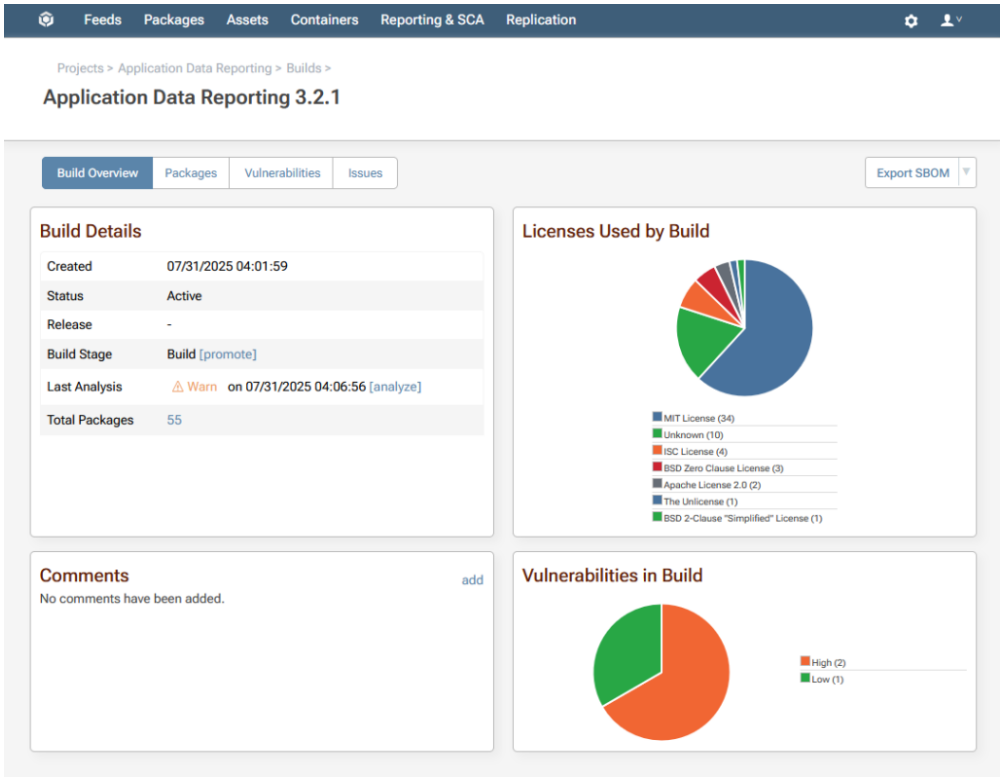
Creating an SBOM in ProGet

ProGet makes it easier to implement automated SBOM workflows in alignment with standards such as the OWASP CycloneDX specification and federal guidance like Executive Order 14028. SBOMs can be generated as part of the build process using Inedo's pgutil CLI tool and the pgutil builds scan command, which audits components and automatically uploads the SBOM to a centralized instance. For example:

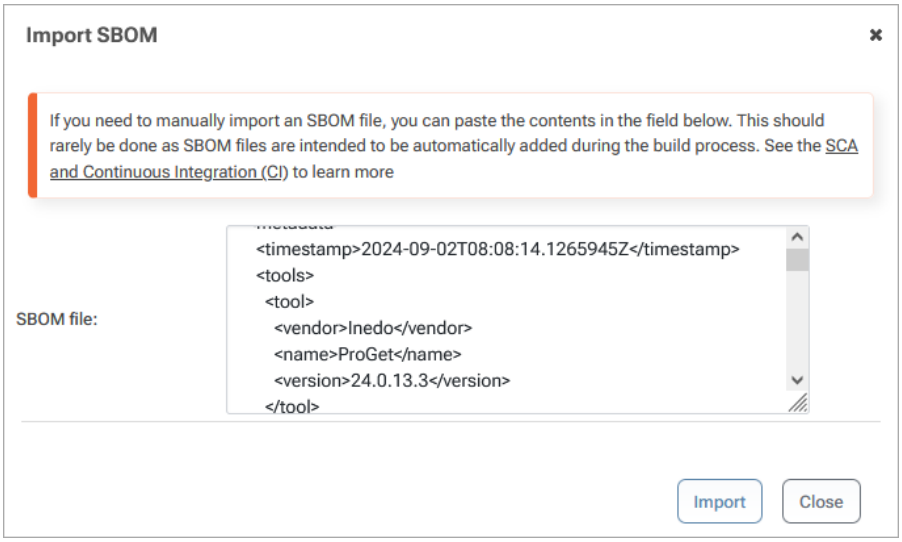
```
pgutil builds scan --project-name="Web Data Tool" --version=1.2.3
```

This command will capture all dependencies (even the transitive ones) and upload a lightweight SBOM to ProGet. From there, ProGet fills in the gaps with extra details like licenses, vulnerabilities, and author info, checking everything against your organization's policies. From here, you can:

- ⭐ Create or update Projects and Releases based on component metadata
- ⭐ Add all relevant packages (including transitive dependencies) to each release
- ⭐ Store the SBOM document alongside the release for audit purposes
- ⭐ Audit components for license issues and retain results for ongoing compliance tracking



You're not limited to pgutil. If your build system already generates SBOMs (including CycloneDX), you can upload them to ProGet via the UI or API. ProGet then fills in extra details like author info and release notes, runs audits to catch compliance issues, and gives you centralized traceability and real-time visibility. Exported SBOMs are still useful for reports, but connecting everything to ProGet gives you the full picture.



Best Practices for Managing SBOM

Before I leave you, it's worth mentioning a few things you'll want to keep in mind if you want to effectively manage your SBOMs.

💡 **Keep SBOMs up-to-date**: Dependencies change all the time. Regenerate SBOMs whenever packages are added, updated, or removed. ProGet can automate this, while CycloneDX requires running a new scan.

💡 **Integrate SBOMs into your workflow**: Treat SBOMs like any other artifact. Store them in source control or CI/CD pipelines so they're always available and versioned. With ProGet, you don't even have to worry about this, as SBOMs are stored automatically and ready whenever you need them.

💡 **Use SBOMs for risk management, not just compliance**: It's all too easy to generate an SBOM, tick the checkbox, and then forget it exists. The real value comes from actively using it to keep a tab on vulnerabilities, outdated packages, or license conflicts, and act on the findings rather than letting them sit on a shelf.

💡 **Merge SBOMs when multiple ecosystems are involved**: Many projects mix different types of dependencies. For example, you might have a NuGet backend alongside an npm-based frontend, each producing its own SBOM. Keeping them separate can make it hard to get a complete view of your software supply chain.

Merging them though, will make sure you have a complete, accurate inventory of everything your project depends on, no matter which package ecosystem it comes from. With **CycloneDX**, you can merge multiple SBOM files into a single SBOM using the CLI:

```
cyclonedx merge -i sbom1.xml sbom2.xml -o merged-sbom.xml
```

This creates one SBOM that combines the components and dependencies from the input files. With **ProGet**, SBOMs are automatically merged within a project, giving you a unified view of all dependencies without extra work.

Making SBOMs Work for NuGet

SBOMs are becoming an important and common way of tracking what's in your code. But when you have to keep track of not only the packages you're installing, but also the dozens of hidden dependencies that come along with them, it's easy to see why NuGet SBOMs might feel a bit too much to get your head around.

Good news is that generating and managing SBOMs doesn't have to be painful. CycloneDX lets you quickly create one-off SBOMs for individual projects, while ProGet keeps everything up-to-date, automatically merges dependencies, and gives you visibility across multiple projects. With these tools integrated into your workflow, SBOMs start being a practical way to stay on top of your dependencies, security, and compliance.

NuGet Private Package Manager Comparison Guide

So, you set up a local, private NuGet repository on a network file share for your small team, and it worked great... *At first.* But as an organization scales and more packages (and more developers) join the mix, things start to slow down. And let's be honest: using network share feels clunky when everything is web-based, especially over a VPN.

Local private NuGet repositories are simple—it's just a file share! But at some point, you'll need to upgrade to a real NuGet package manager. The tricky part? There's a lot of options out there, and not every team needs an enterprise-level solution. Sometimes, you just want something "a little better" that won't overcomplicate your setup, and while the simplest choice is to go with ProGet, it's not the only option.

In this chapter, we'll explore the **five** different types of NuGet package managers out there, the differences between them, and how to use this info to decide which server is best for you and your team.

What can a Private NuGet Server do?

Before diving into the five different types of NuGet package managers, take a moment to think about the features your team actually needs. Even the most basic NuGet manager can do a lot more than a simple local feed—so it's worth knowing what tools are out there:

☑ **Restricted feeds** allow administrators to control who can view, publish or promote packages.

☑ **Proxy / Mirror NuGet.org**: Sits between your team and NuGet.org (and other sources) to act as a sort of proxy or mirror for packages.

☑ **Open-source Package Verification** automatically scans and blocks packages that have vulnerabilities or unwanted licenses.

☑ **Host Non-NuGet Packages**: As your team expands beyond .NET, you may have a need for packages like npm / Node.js.

☑ **Package promotion**: Integrated workflows that help standardize the package review/approval process, and your software development lifecycle.

Keep these features in mind as you look around for a private NuGet package manager, and think about which ones you're willing to compromise on (and which ones you're definitely not) before committing to a new server.

Type 1: NuGet.Server-based Servers

A lot of teams lean towards NuGet.Server as a replacement for their local feed. It is a lightweight NuGet package you can use to build a web app that hosts a private NuGet feed. It's a small step up from a local feed.

Microsoft has clear docs for building a NuGet package manager, and there's tons of community posts to help if you get stuck. Pre-built servers can be bought from NuGetServer.net for a few bucks to save you the hassle of making it yourself. *But...*

⚠ NuGet.Server Isn't Recommended ⚠

With so many free alternatives available, NuGet server can't be recommended anymore. It comes with too many limitations, and *none* of the features other NuGet package managers have.

The limitation that sticks out is the lack of **restricted feeds**. At best, you can set up a feed to be read-only. A restricted feed is important since it lets admin choose who can view, publish, and more to your feeds, a basic feature that keeps package quality consistent in growing teams.

Type 2: Community/Hobby Projects

Community or hobby project servers are open-source NuGet feeds that are all still actively being developed—*and they're open to anyone*. The big plus here is the *community* part. You can check out the project's GitHub, pitch in with contributions, scan posts for troubleshooting tips, or see how other devs optimized the NuGet package manager's functions.

That said, these projects are usually side gigs for someone (or a small group), so there's no commercial support, but they're a great fit for individuals or small teams.

✓ **Sleet**	A *simple static NuGet package feed generator*, Sleet is serverless, which means you can create feeds directly on AWS for example... But, it's static, so it's read-only, and you can't publish packages to it.
✓ **BaGetter**	Pronounced just like the bread but better, BaGetter is a *lightweight NuGet and Symbol Server* forked from BaGet, still actively maintained with continued features and improvements, with its latest update in Feb 2025.
⚠ **SlimGet** (Inactive)	A *lightweight implementation of a NuGet and Symbol Server, powered by ASP.NET Core 2.2, designed to be ran in Docker*. Currently seems like it's no longer updated, with the last commit made back in 2020.

While none of the community projects support restricted feeds, **BaGetter** *does* support NuGet.org proxying and mirroring, allowing you handy access to one hundred percent of the packages you need with no direct contact with the site.

That being said, these projects are really just meant for hosting your internal packages. Once you start using more open-source packages from NuGet.org, you'll start to feel the limitations—especially when it comes to integrated license and vulnerability scanning, critical features when using third-party packages.

Skipping these license checks can land you in hot water—and I mean serious legal trouble. Likewise, shipping a package with security vulnerabilities is only a matter of time without an automated scanning solution.

Type 3: Package Hosting Services

Just like the name suggests, these are options that host packages—basically "Packages as a Service." Unlike local or community options, these are fully managed by the hosting service, so setup is quick and easy. Right now, there's only three major players offering hosted NuGet feeds:

MyGet	A hosted universal package registry that *integrates with your existing source code ecosystem and enables end-to-end package management*. They also offer Build Services, which is continuous integration for your packages.
Cloudsmith	A *cloud-native, hosted package management service* that helps you manage, trace and control the software used within your development and deployment pipelines, or when you distribute it to your customers.
Feedz.io	A *Package Hosting and Distribution* that lets you store and distribute your private NuGet and npm packages, with no user limit, and transparent pricing.
Bytesafe	A relatively new contender offering a *security-focused NuGet registry with support for SBOMs, license and vulnerability scanning, and dependency firewalling*. It can act as both a private feed and a proxy to NuGet.org, and supports integration with GitHub repos.

MyGet, **Bytesafe**, and **Cloudsmith** support NuGet.org proxying and mirroring, allowing your team to pull NuGet packages with no direct contact with the NuGet site itself. Even better, they both support automated vulnerability and license scanning for those external packages, which is super useful for when you're working with open-source libraries and need to cover your legal and security bases.

They also come with some other handy features: **restricted feeds** and the ability to **host non-NuGet packages**.

But while these package hosting services are the easiest to get up and running, they're not always the ideal choice for scaling teams or larger enterprises. Everything is stored in the cloud, which might not be the best environment for the code you're working with, and this also means the pricing gets confusing fast, depending on how much space or what features you use.

Type 4: Built-in Package Registries

A **Built-in Package Registry** is a feed built into another tool—usually a **monolithic platform**—and tightly integrated with the projects you build there. Monolithic platforms are all-in-one services that handle everything from project management, issue tracking, build automation, and deployment.

A built-in feed is kind of like the scissors on a Swiss Army Knife—**super convenient,** but not always the best tool for the job. And unlike a real Swiss Army Knife, the package registry feature often feels like an afterthought (along with most of the other features in the platform). There are currently five built-in package NuGet package registries out there:

- GitHub Packages
- GitLab Packages
- Azure Artifacts
- TeamCity as a NuGet Feed
- Gitea

These tools might be powerful, but they're not **really built for packages**. They're complex, expensive, and unless you're planning on using them for other stuff, they're just not worth it. Even organizations that do use them often find the built-in feed to be **too complex and limited**, especially since monoliths focus on building your software, not pulling open-source NuGet packages. Proxying and package validation usually aren't included.

Type 5: Enterprise-Grade Repositories

Designed to be the central hub for all your software assets, packages, and Docker containers, this type of NuGet package manager crams in a ton of package-focused features, like high availability and replication across servers.

There are a few big names on the market—Artifactory, Nexus, and ProGet—all with free and paid versions available. But when it comes to hosting private NuGet feeds, **ProGet is hands down the best option**. And I'm not just saying that because I work for Inedo.

☆ ProGet Is Recommended ☆

ProGet was always designed around NuGet. Take Source and Symbol Serving, a must-have feature for package development, from day one, right out of the box. Compare this to Artifactory, which only incorporated Symbol Servers after eight years of it being a requested feature, and doesn't support legacy formats like .symbols.nuget, and **Microsoft PDBs**, unlike ProGet.

Not to mention, ProGet's features built around developing NuGet packages using CI/CD pipelines, tracking which applications use which packages, and creating a package review process.

So why not just drive straight into ProGet? It's free and easy to install, but it has *a lot more* features than most teams are looking for right now. That can feel overwhelming—but if you're like me, it's hard not to use a feature if I see it, even if building processes around these new features takes time.

And then there's the matter of system requirements. Enterprise-grade repositories like ProGet need a bit more server resources than a basic NuGet.Server feed, but in return, they'll scale *way* better when your team usage grows.

The Right NuGet Package Manager for You

This chapter covered a lot—various types of NuGet servers and all the options they come with. From NuGet.server to Enterprise-Grade, there's a NuGet private package manager that will work for you.

For teams operating in the enterprise, ProGet offers features such as package workflows, license detection, vulnerability scanning, restricted feeds, repackaging, and integrated debugging. Together in ProGet, you and your team can have greater control over your private NuGet servers, minimizing risk and increasing security for your projects.

Appendix

Now you know how to manage NuGet packages at scale, these practical mini chapters will guide you through applying that knowledge in real-world environments, focusing on key ProGet tasks like onboarding, CI/CD security, and governance.

Contents:

NuGet (.NET) Packages and Feeds in ProGet

A NuGet feed allows client tools like Visual Studio to consume and publish NuGet (the package manager for .NET) and symbol packages.

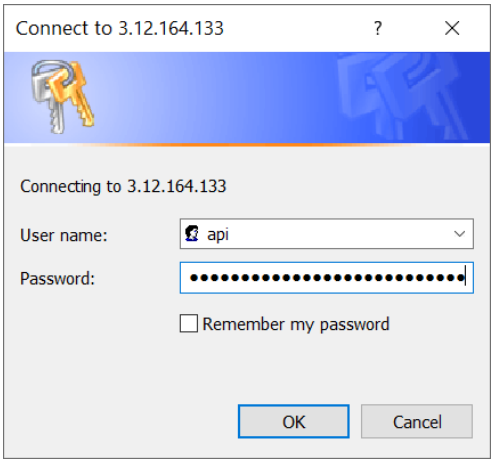
You can also use a NuGet Feed as a proxy for NuGet.org, both to block improper licenses and security vulnerabilities, and to cache packages in case the internet or NuGet.org goes down.

Authenticating to NuGet Feeds

By default, NuGet feeds in ProGet do not need to be authenticated to when consuming packages. However, if your feed requires authentication to view and download packages, then you'll need to configure an *authenticated* source when adding the feed.

> **Best Practices: Use API Keys for Authenticated Feeds**
>
> Instead of using your ProGet username/password for a NuGet feed, we recommend Creating a ProGet API Key to authenticate. You can enter `api` as the username and your key as the password.

When an authenticated feed is added to Visual Studio, you'll be prompted for a username and password. You can enter `api` for the username and your API key as the password:



Or, you can use the `--username` and `--api` arguments in the NuGet CLI:

```
$ dotnet nuget add source --name "Internal NuGet" --username api --password
abcdef12345 https://proget.corp.local/nuget/internal-nuget
```

See the NuGet CLI sources command to learn more.

Authentication for Publishing Packages

If you use the NuGet CLI to push packages to ProGet, you'll need to also specify an `--api-Key` argument.

You can also add an API key to a source using the `setApiKey` command:

```
$ dotnet nuget setApiKey «api-key» -Source «feed-url»
```

The NuGet CLI only uses this setting for pushing packages; it will be ignored when viewing/downloading packages, which means you'll also need to add an authenticated source.

Creating and Publishing NuGet Packages

The NuGet package format is well-documented, and you can create packages in a number of ways, including directly from Visual Studio or using the NuGet CLI.

However, many users find it easiest to publish packages using pgutil. This requires some minor configuration before use, but allows you to more easily specify different feeds.

Example:

To upload the package `My.Package.1.0.0.nupkg` stored at `bin\publish` to your `internal-nuget` feed you would enter:

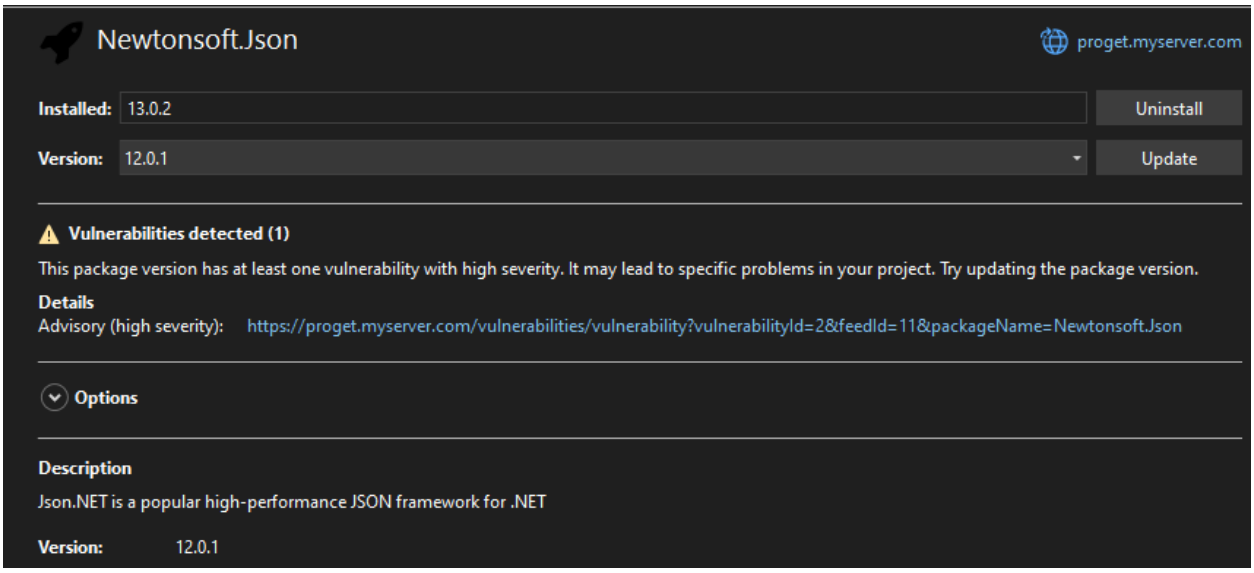
```
$ pgutil packages upload --feed=internal-nuget
--input-file=bin\publish\My.Package.1.0.0.nupkg
```

Your package will then be uploaded to the `internal-nuget` feed. To learn more about creating and publishing packages, see HOWTO: Publish NuGet Packages to a Private Repository in ProGet.

Vulnerability Integration

Beginning in ProGet 2023.5, we have added support to show Vulnerability Information directly in Visual Studio and the NuGet CLI. You will see the severity of your vulnerability, along with a link to see the vulnerability details in ProGet.

When selecting a package version with a vulnerability in Visual Studio, you will see the severity with a clickable link to the vulnerability in ProGet.



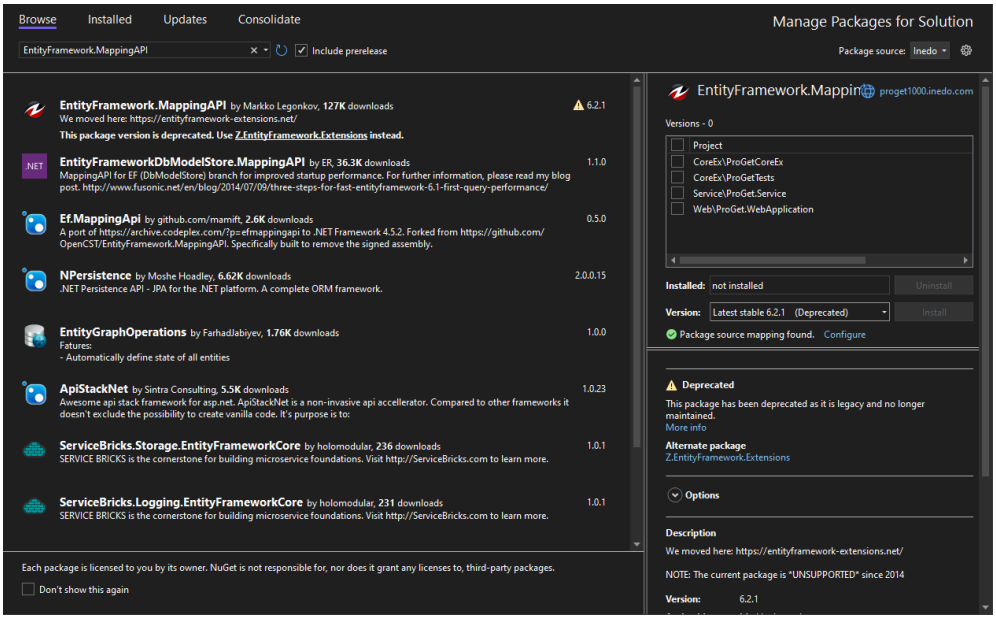
When running `dotnet list package --vulnerable` in the NuGet CLI, you will see a list of vulnerable packages with their severity and a link to the vulnerability in ProGet.

⚠ ProGet Free edition limits vulnerability information shown in Visual Studio and the NuGet CLI.

Managing Package Deprecation and Unlisting

ProGet will show deprecation information directly in Visual Studio and the NuGet CLI. You will see if a specific version of the package was deprecated along with the specified reason. This works for both packages that come from a NuGet connector and packages that were deprecated using the ProGet UI.

When selecting a package version that has been deprecated in Visual Studio, you will see a warning that the package has been deprecated. If the package has been installed in your project, you will also see a shield in the package list.



When running `dotnet list package --deprecated` in the NuGet CLI, you will see a list of deprecated packages installed in your project along with the deprecation reason.

```
>dotnet list package --deprecated

The following sources were used:
   https://proget1000.inedo.com/nuget/NuGetLibraries/v3/index.json
   C:\Program Files (x86)\Microsoft SDKs\NuGetPackages\

Project `VatCompWeb` has the following deprecated packages
   [net6.0]:
   Top-level Package               Requested   Resolved   Reason(s)   Alternative
   > EntityFramework.MappingAPI    6.2.1       6.2.1      Legacy      Z.EntityFramework.Extensions >= 0.0.0
```

Visual Studio and the NuGet CLI heavily caches this information and may take time to show this information after a package has been marked as deprecated. Due to this caching, parts of Visual Studio (version drop-down, version details, and package list) may show the deprecation information at different times.

To force the NuGet CLI to update its deprecated packages, you will need to first clear the NuGet HTTP Cache using the command:

```
`dotnet nuget locals http-cache --clear`
```

Then the next time you run `dotnet list package --deprecated`, the deprecation information will show immediately.

To force Visual Studio to update its deprecated packages, you will need to navigate to:

Tools > Options > NuGet Package Manager > General > Clear All NuGet Storage

This will clear the entire NuGet cache (including cached packages) from the machine, and the deprecated information will show immediately.

In addition to marking packages as deprecated, ProGet allows you to unlist or delete packages from your feed:

Unlisting Packages

Unlisted packages will no longer appear in most search results or Visual Studio. However, these packages can still be installed if the exact version number is specified, making it a useful option for deprecated or outdated packages you want to limit to advanced users.

To unlist a package you can use the `packages status unlisted` command in pgutil. For example, if unlisting version `1.2.3` of the package `MyPackage` from the feed `internal nuget` you would enter:

```
$ pgutil packages status unlisted --feed=internal-nuget --package=MyPackage --version=1.2.3 --state=listed
```

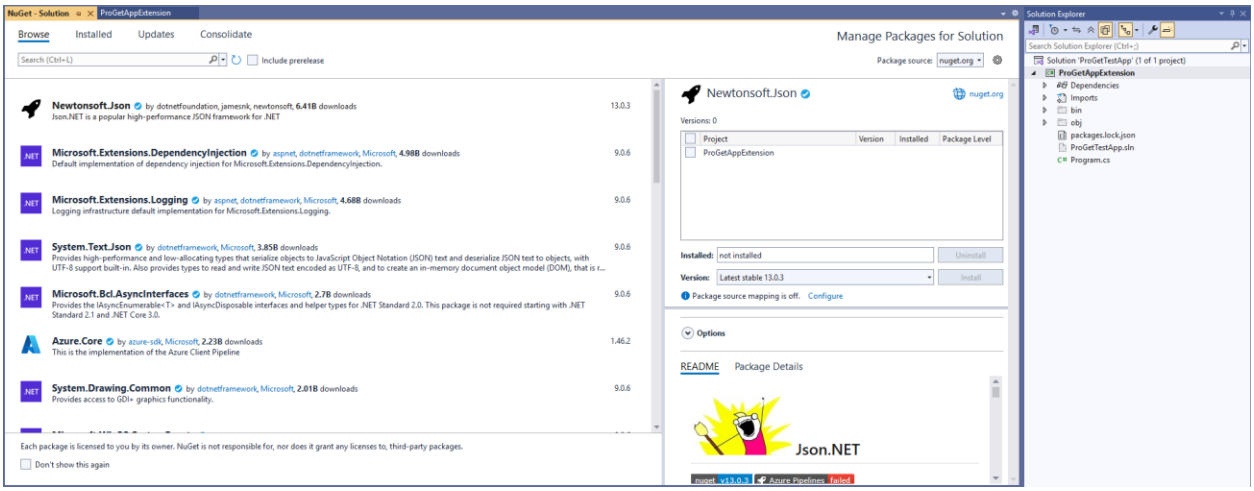
How to Install / Proxy NuGet Packages in Visual Studio

NuGet's integration with Visual Studio means you're connected to NuGet.org by default. Here's a quick look at how it all works—and what you should keep in mind.

You can browse NuGet packages through the Package Manager. Navigate to:

Tools > NuGet Package Manager > Manage NuGet Packages for Solution...

Then use the **Browse** tab to search for packages from the public repository.



Next, just select the package you're looking for, check its details, and click **Install** to add it to your project. Accept any license agreements to complete the installation.

Connecting directly to NuGet.org might work well for your personal projects, but for larger-scale teams... not so much, with limited control over:

⚠ Vulnerabilities

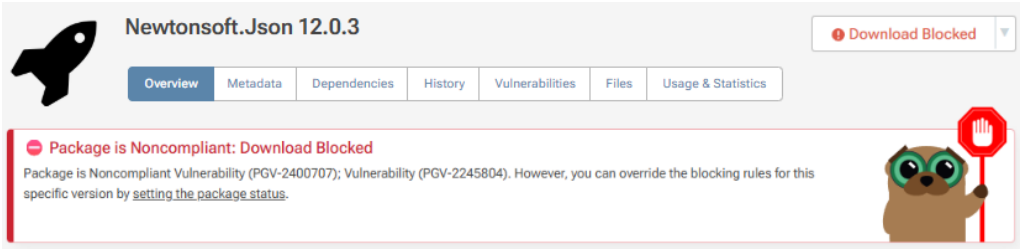
⚠ Licensing

⚠ Deprecated packages

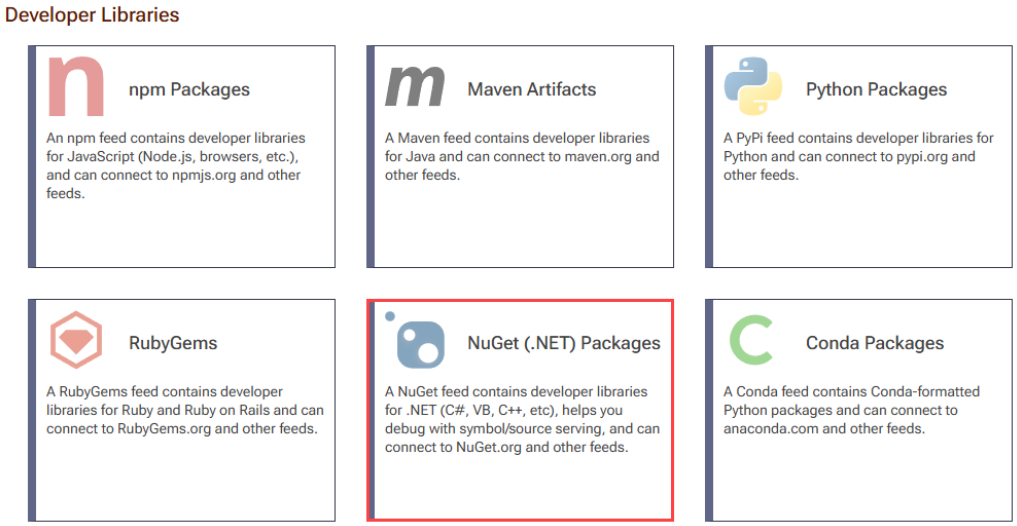
These risks highlight the need for better oversight in package management, and that's where a proxy package manager like ProGet can help.

Using a Proxy Package Manager with NuGet Packages

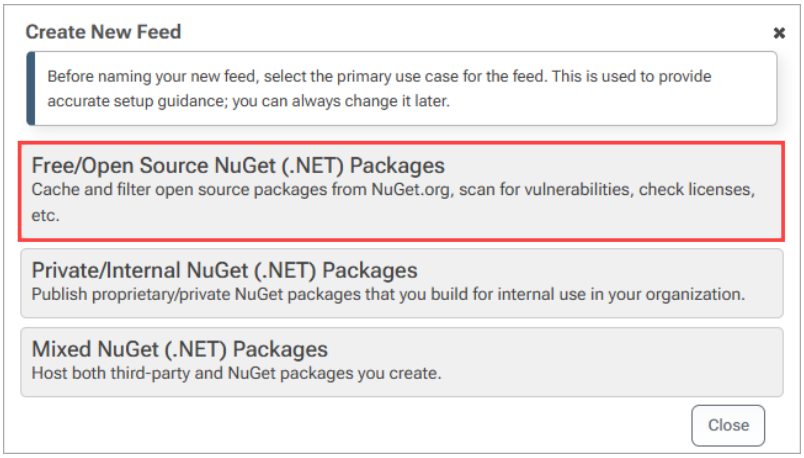
In **ProGet** you can *proxy* packages from NuGet.org, without a direct connection, set up approval flows, detect vulnerabilities with vulnerability scanning, block or permit these packages, and create license rules for automatically detected licenses, limiting developers to pulling production-safe packages.



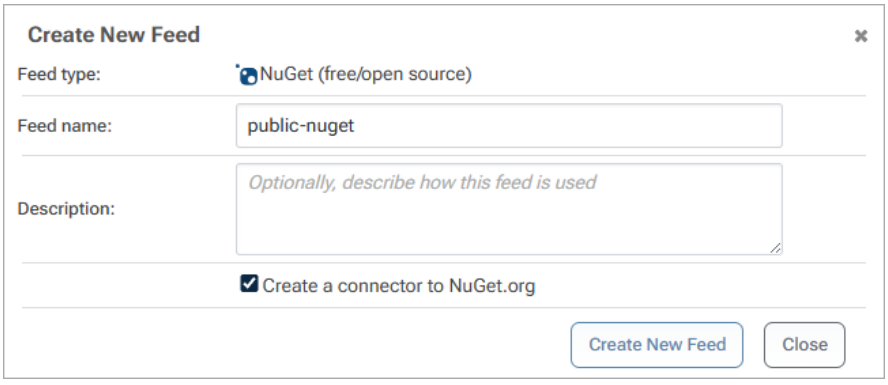
Begin by creating a private NuGet feed in ProGet to host your internal packages. Navigate to **Feeds > Create New Feed**, then choose **NuGet (.NET) Packages**.



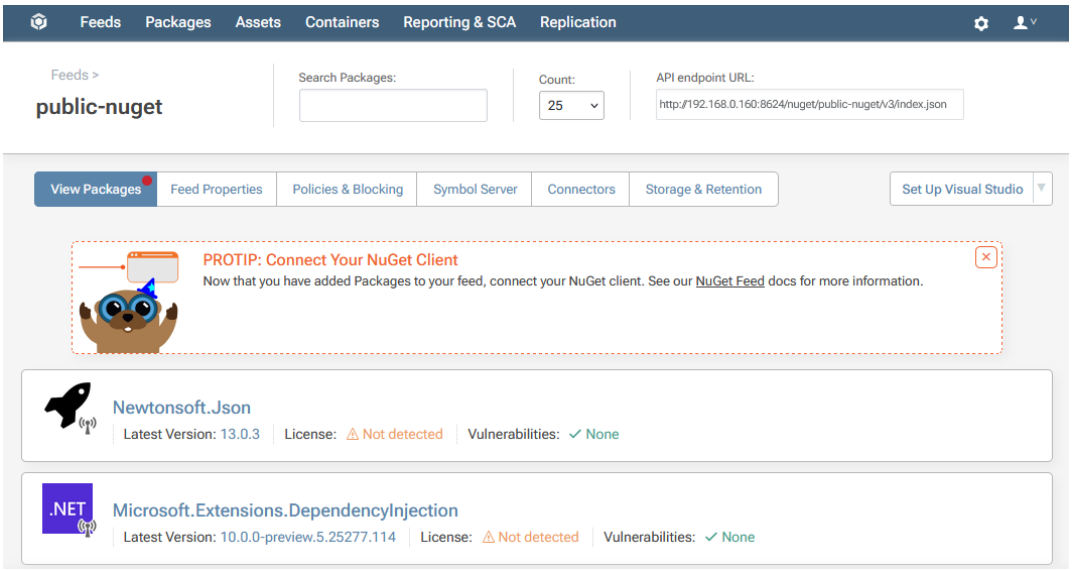
Then select **Free/Open Source NuGet (.NET) Packages** which will allow you to proxy and cache packages from NuGet.org.



Next, name your feed–for example, **public-nuget,** and click **Create Feed**.



ProGet will create your `public-nuget` feed, which will now be populated with packages proxied from NuGet.org.

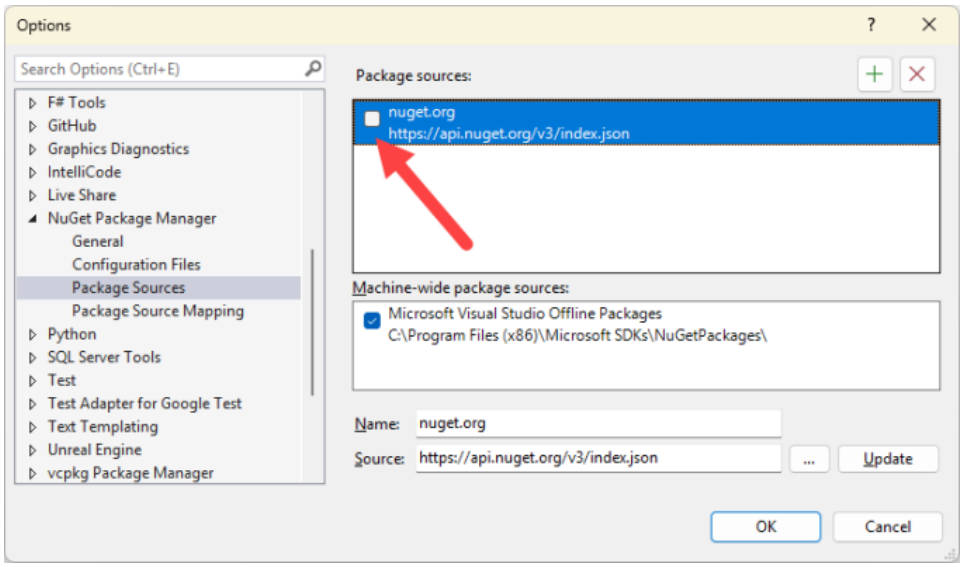


Adding ProGet Feeds as a Source in Visual Studio

Once your ProGet feeds are ready, it's super easy for developers to add them as a source in Visual Studio. Navigate to:

Tools > NuGet Package Manager > Package Manager Settings > Package Sources

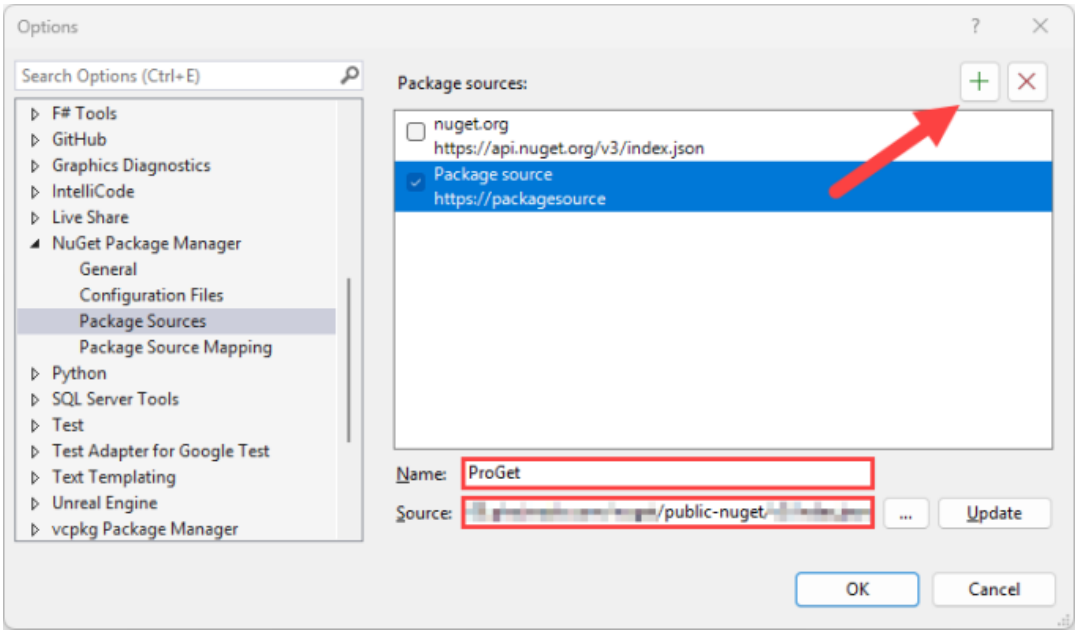
Then, deselect the "nuget.org" checkbox to stop Visual Studio scanning NuGet.org and ProGet for packages. Limiting package pulls to ProGet avoids licensing issues, vulnerable packages, dependency conflicts, and other common problems when using public sources.



⚙ NuGet Feed Endpoint URL

```
https://«proget-server»/nuget/«feed-name»/v3/index.json
```

Next, create a new package source by clicking the **+** button at the top right. Give your new source a name and enter your ProGet feed URL. Finally, click **OK** to save your changes.



Be sure to click "Update" prior to clicking "OK"
If you click "Ok" without clicking "Update" your package source configuration will not be saved in Visual Studio.

Visual Studio and ProGet will now be connected. To confirm the connection in Visual Studio, right click on a project in the solution explorer and select **Manage NuGet Packages...** from the menu. In the **Package Manager** window under **Browse**, you should see a window populated with packages from your ProGet NuGet feed.

Integrating the NuGet CLI

You can also use the NuGet CLI to add your approved ProGet NuGet feed as a source:

```
$ dotnet nuget add source --name "Internal NuGet" https://«proget-
server»/nuget/«feed-name»
```

Once you've added the NuGet feed as a source, you'll be able to install and upgrade NuGet packages.

Using Other NuGet Clients

ProGet can also be added as a source in a number of other popular clients including VS Code and JetBrains Rider.

In VS Code

To add your NuGet feed in ProGet as a source, add it to a `nuget.config` in your project. The config should look like this:

```
<?xml version="1.0" encoding="utf-8"?>
  <configuration>
    <packageSources>
    <add key="proget" value="https://«proget-server»/nuget/«feed-
name»/v3/index.json" />
  </packageSources>
</configuration>
```

In JetBrains Rider

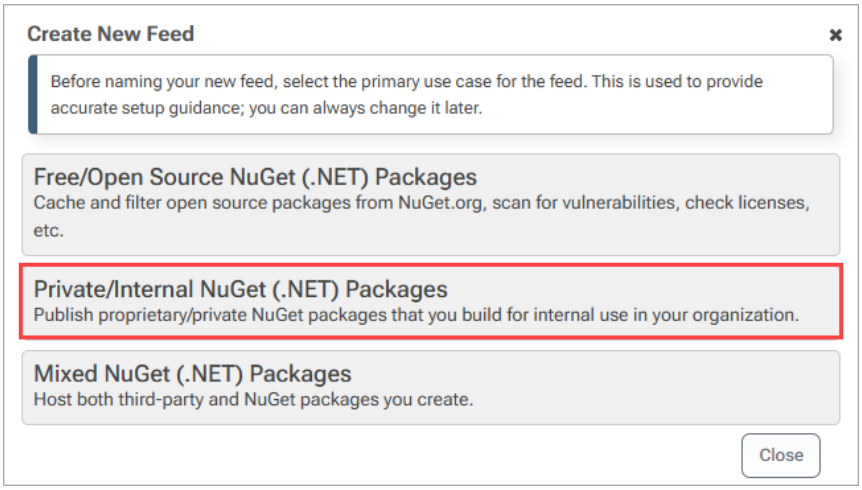
To add your NuGet feed in ProGet as a source, open "NuGet Settings" by navigating to "File" > "Settings", and then "Build, Execution, Deployment" > "NuGet". Next, under the "Package Sources" tab, click on the + (Add) button to create a new package source.

In the `Name` field, enter a name for your source (e.g., `internal-nuget`), and then in the URL field, enter the URL of your feed.

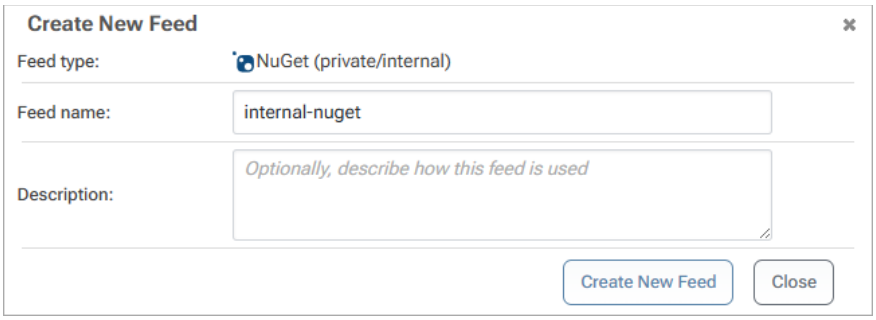
Creating and Uploading NuGet Packages to a Private ProGet Repository

ProGet lets you set up private NuGet repositories to publish, store, and share your internal NuGet packages within your organization.

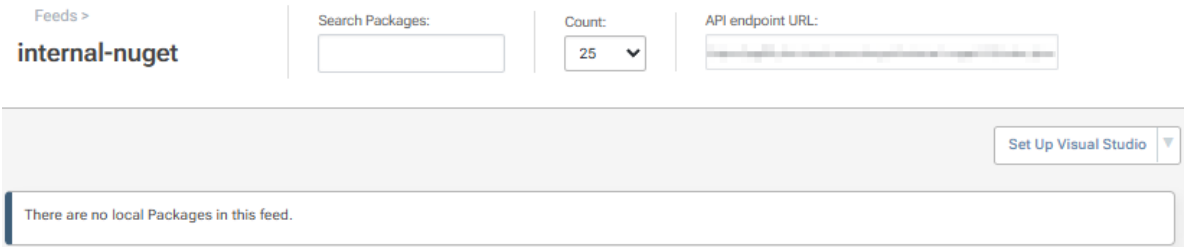
As shown earlier, begin by following the steps to create a NuGet feed. This time, select **Private/Internal NuGet (.NET) Packages**, since this feed will only host private packages.



Next, name your private feed–for example, `internal-nuget`–and select **create feed**.



ProGet will create your `internal-nuget` feed, which will currently be empty.



Creating a NuGet Package

In Visual Studio, you can create a NuGet package by following these steps:

1. Create a class library project by selecting the Class Library template, choosing a suitable framework, then building the project to ensure it's set up correctly.
2. Configure the NuGet package properties by accessing Project Settings, ensuring the Package ID is unique, and other details are filled out.
3. Use the **Pack** command in **Release Configuration** to generate a `.nupkg file`, checking the Output Window for its location.

For more details, see Microsoft's Official Documentation.

Creating a project in dotnet is easier to demonstrate visually, and shows the individual files that make up a NuGet package. To start, create the project in the dotnet CLI by entering:

```
$ dotnet new classlib -n MyPackage
$ cd MyPackage
```

In the project folder, create a `.nuspec` file, that will contain the package metadata. For example:

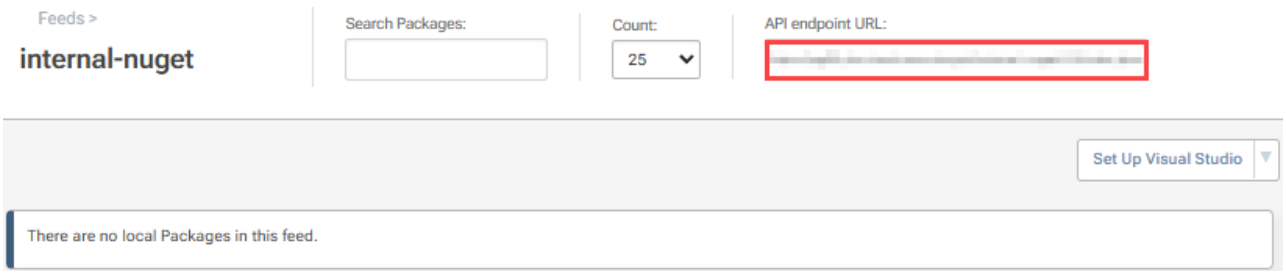
```
<?xml version="1.0"?>
<package >
  <metadata>
   <id>MyPackage</id>
   <version>1.0.0</version>
   <authors>Your Name</authors>
   <owners>Your Organization</owners>
   <description>A simple example NuGet package.</description>
   <language>en-US</language>
   <tags>example</tags>
  </metadata>
</package>
```

Then run the following command to create the package:

```
$ dotnet pack --configuration Release
```

Add Your Internal Feed as a Source

Adding your `internal-nuget` feed as a source in Visual Studio, the nuget CLI, or other clients like VS Code and JetBrains Rider, requires your `internal-nuget` feed URL found at the top right of your feed page.



Then, simply follow the steps shown earlier, to connect your ProGet feed to Visual Studio.

Pushing Your Package to Your NuGet Feed

You can push your NuGet package using either Visual Studio or the NuGet CLI—just ensure your feed is added as a source and authentication is configured.

To push your NuGet package using Visual Studio, open the Package Manager Console by navigating to **Tools > NuGet Package Manager > Package Manager Console**. Then, run the following push command:

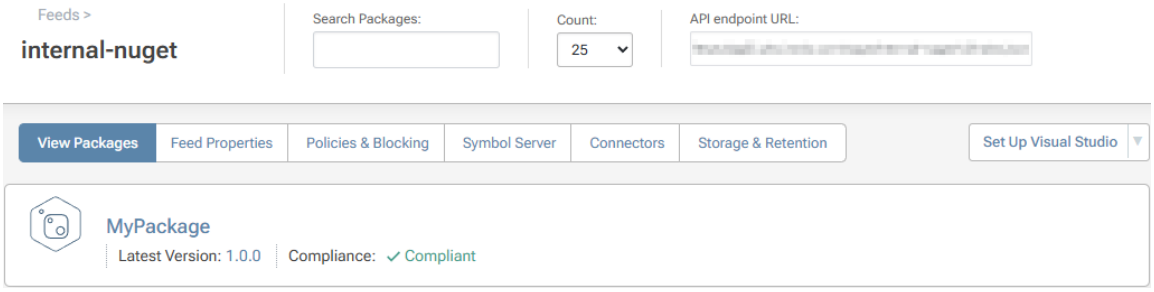
```
$ nuget push «path-to-package» -Source «source-name» -ApiKey api:«apikey
```

Example:

Pushing the package MyPackage to a configured source named internal-nuget, you would enter:

```
$ nuget push bin\Release\MyPackage.1.0.0.nupkg -Source internal-nuget -
ApiKey api:abc12345
```

Your package will then be uploaded and listed in your NuGet feed.

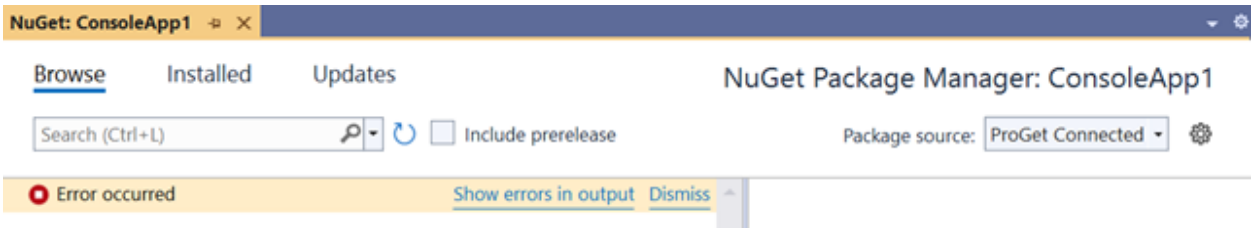


(Optional) Repackaging NuGet Packages in CI/CD

After publishing, consider repackaging pre-release versions into stable, production-ready packages to improve your CI/CD workflow. This also maintains an audit trail showing who performed the operation and when. For details, see Repackaging NuGet Packages.

Troubleshooting (Authentication Error)

An error may occur when trying to browse the authenticated NuGet feed in Visual Studio.

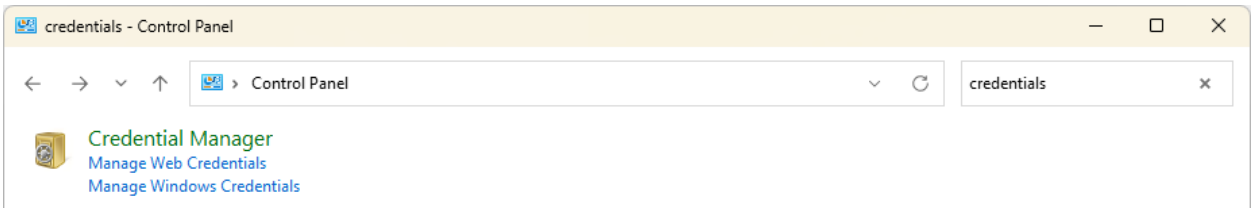


The window does not fill with packages and the error list says "API Key ... does not exist."

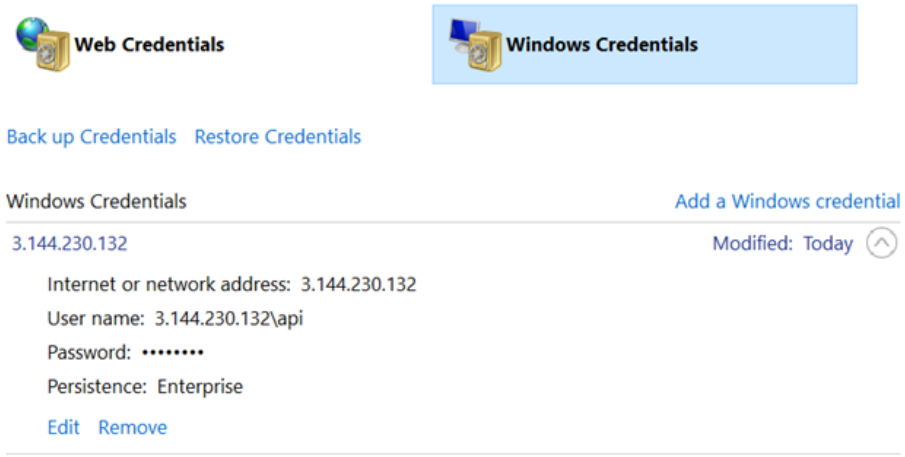
There may have been an error entering the personal API key while connecting to the server, or the API key may have been deleted in ProGet.

To resolve, in Visual Studio, close all your instances.

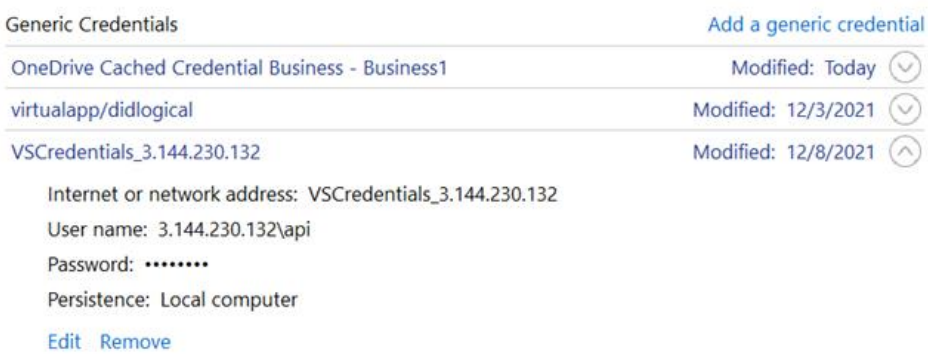
Then, in **Windows**, open the **Control Panel** and navigate to **Credential Manager**.



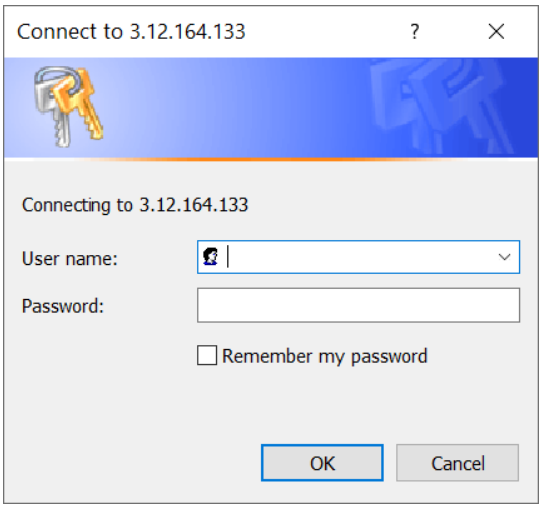
Under Windows Credentials, delete the one named as the ProGet host (in this example, 3.144.230.132):



Under **Generic Credentials**, find and remove the one named "VSCredentials..." (in this example, `VSCredentials_3.144.230.132`):



Now, in Visual Studio open your instance and navigate to the **Package Manager**. You will be prompted again to enter a Personal API key: `api` for the username, and your API key as the password.



NuGet Package Versioning (SemVer2, Legacy)

All NuGet packages have version numbers–`4.2.3`, `3.1.2.50012`, `6.2.0-beta1`, etc. The format of these version numbers is called a *version number scheme*, and NuGet has had several different schemes over the years.

NuGet SemVer2 Package Versioning

Modern versions of ProGet (V5 and later) as well as NuGet (i.e. 4.3/2018+) support SemVer2 versioning for packages. This is a standardized versioning scheme across many packages and has become intuitive to many software developers.

Best Practices: Use SemVer2

Use SemVer2 compatible versioning for your NuGet packages. We recommend it, Microsoft recommends it, and your future developers will appreciate it.

A version number in SemVer2 format consists of three integer parts (major, minor, and patch) plus an optional suffix pre-release. Each component has a specific meaning:

- Major: Breaking changes
- Minor: New features, but backward compatible
- Patch: Backward compatible bug fixes
- Pre-release: A non-stable package such as alpha, beta, etc.

Example versions include: `1.0.1, 6.11.1231, 4.3.1-rc, 2.2.44-beta.1.`

See Microsoft's NuGet Package Versioning Documentation to learn more about how to use version numbers in your packages.

Build Metadata

There is a fifth, rarely used component in SemVer2: the build metadata. It starts with a plus sign (+) and may contain any of the characters that are allowed to contain suffixes for pre-release versions. Build metadata is ignored when comparing version numbers, so `1.2.3+foo` and `1.2.3+bar` are considered the same version.

In addition, NuGet generally ignores build metadata when it requests packages (e.g., if you specify `1.2.3+foo`, NuGet only asks for `1.2.3`); thus, ProGet generally doesn't require build metadata when it requests packages. We say "in general" because there are bugs and quirks in different versions of NuGet (and also of ProGet).

Best Practices: Avoid Build Metadata in NuGet Packages

It's not very useful in NuGet, causes confusion, and you can just add additional metadata to your .nuspec if you need it.

Legacy NuGet Version Numbers

Prior to supporting SemVer2 versioning, NuGet used a 4-digit versioning scheme (e.g. `3.4.1.5`) that is a combination of .NET's System.Version class and SemVer1 (an older, obsolete specification). This format isn't very well documented.

ProGet considers these "legacy version numbers" and defines them as:

- Four decimal integers (digits 0-9), separated by periods (a)
- Optionally a pre-release indicator, i.e. a hyphen (-) followed by a sequence of letters (a-z, A-Z), digits (0-9), and hyphens (-). Pre-release indicators must be at least one character long and begin with a letter.

Modern versions of ProGet and NuGet still fully support these types of version numbers.

Quirky Version Numbers

Ancient versions of NuGet (i.e. prior to 3.4/2016) had very few rules, and the rules changed from version to version of NuGet. Versions could be 1,2,3, or 4 parts. In addition, the equality rules were a bit strange. For example, `1.0, 1.000`, and `1.0.0` were considered different version numbers in most cases... except when they weren't. Likewise, a `-6.4` and `6.04` were almost always different versions, depending on how you requested them from NuGet.

We call these "quirky version numbers" and fortunately they have not been allowed on NuGet.org since 2016. Unfortunately, there are many packages with "quirky version numbers" created before 2016 that people still want to use.

A great example of a quirky package is Owin. There's a single version (1.0), and it hasn't changed since 2012. It's considered a "quirky version" because it's only a two-part number.

NuGet.org and Quirky Versions

It's not possible to upload packages in quirky versions to NuGet.org, but the old still exist.

NuGet.org now only shows a "normalized", three-part version number for quirky packages. Even if the package version (as specified in the `.nuspec` file) is `1.0`, NuGet.org will return `1.0.0` as the version number in the API.

You'll notice that Owin is listed as `1.0.0`, even though the actual version is `1.0`. The only way to see the actual version is to download the file and look in the .nuspec file:

```
<metadata>
  <id>Owin</id>
  <version>1.0</version>
  ...snip...
</metadata>
```

However, you still request `Owin-1.0` by name directly, NuGet.org will return the one (and only) version of the `Owin` package.

NuGet Client Support for Quirky Versions

Since not all NuGet servers behave like NuGet.org, the NuGet client (Visual Studio) has to do some interesting workarounds, including "version dancing", to make sure it can find a requested package.

For example, if a package has a dependency on `Owin-1.0`, then you'll see NuGet make multiple requests to the package source: first `1.0.0`, then `1.0.0.0`, then `1.0`, and finally `1`. This allows the NuGet client to work with both NuGet.org, and legacy NuGet servers.

ProGet Support for Quirky Versions

You can still upload a "quirky version" of packages to ProGet in many cases, and these quirky packages will work in most cases. However, they can cause headaches, especially when it comes to connectors and operations around local packages (like promotion and repackaging).

For example, `Owin` will show up as `1.0.0` via a connector to NuGet.org, because that's what NuGet.org is reporting. In most cases, it will work just fine, even though it's really `1.0` behind the scenes. However, when you pull `Owin` ProGet, ProGet will use the "true" version number (`1.0`). It will *also* show `Owin 1.0.0`.

Old versions (before `5.3`) had a "Legacy (Quirks) NuGet" feed that could handle these oddities, but those feeds couldn't handle SemVer queries properly because queries like `1.0` were ambiguous (should `1.0` or `1.0.0` be returned, which are both valid versions, etc.), so it was a big compromise. They have been completely removed in ProGet 5.3.

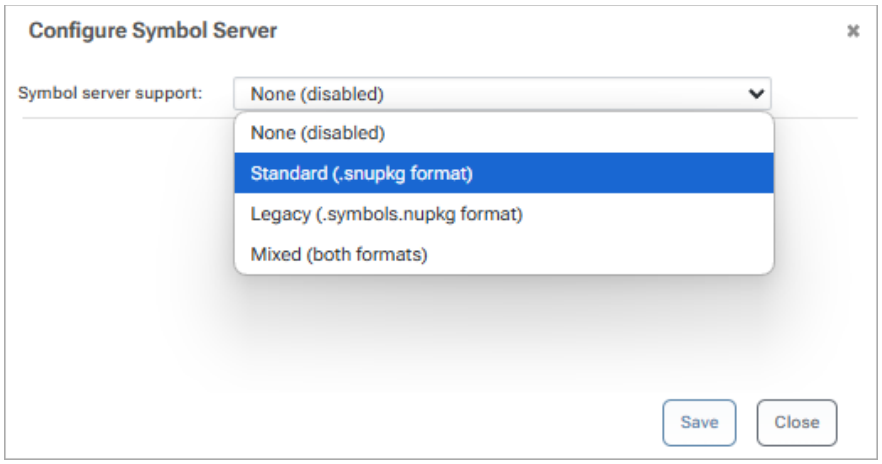
Because there are very few packages with quirks like this, we recommend simply downloading the package, editing the `.nuspec` file, and replacing it with a semantic version.

Symbol and Source Server

Symbols and Source Packages

Debugging in NuGet packages can be tedious, but you can make it easier by configuring your NuGet feeds to work as a Symbol/Source server compatible with debuggers like Visual Studio and WinDbg.

A NuGet feed in ProGet may be configured as a Symbol/Source server compatible with debuggers such as Visual Studio and WinDbg. This will allow you to "step through" code in your NuGet package as if it were in the project.



Configuring this is a bit more involved than simply enabling that feature on your feed and requires creating special NuGet symbol packages and then configuring Visual Studio to use those packages. See Symbols and Source Code in ProGet to learn more.

This section will walk through:

- Creating Symbol Packages
- Configuring ProGet for Symbol Serving
- Configuring Visual Studio
- Testing Configuration
- Troubleshooting Tips

Looking for the Symbol Server URL for Visual Studio?

If you're trying to Configure Visual Studio and are looking for the URL, it will follow this pattern:

```
http://«proget-server»/symbols/«feed-name»
```

Creating Symbol Packages

There are two types of NuGet symbol packages: standard (`.snupkg`) and legacy (`.symbols.nupkg`). As of v2022.20, ProGet supports both package formats, but earlier versions support only the legacy format. See Symbol Package Formats for more information about these formats; in general, we recommend using standard symbol packages when possible.

Pushing Symbol Packages to ProGet

By default, when you use the `nuget push` command and there is a standard format (`.snupkg`) symbol package next to the main NuGet package, the symbol package is transferred by default.

If you are using a legacy NuGet Symbol package, your main NuGet package will also be your symbol package. To ensure that you create a legacy symbol package so that it can be processed by ProGet, see Symbol Package Formats.

After configuring ProGet for symbol serving, you can push a package and its symbols to ProGet by simply adding the `--source` argument to `dotnet nuget push` with the feed's Endpoint URL.

For example:

```
dotnet nuget push kramericalib.4.1.2.nupkg -- source
https://proget.kramerica.corp/nuget/internal-nuget/index.json
```

PDB Formats and Source Serving

ProGet can index and serve both `.pdb` file formats: portable and Windows PDB (sometimes referred to as Microsoft PDB). Portable PDB files are cross-platform and an open specification, and are generated by default with modern .NET build systems. Portable PDB files have many advantages

over the older format, but the main advantage for ProGet users is how they provide source files to debuggers.

Source Link (Portable PDB)

Portable PDB files can use Source Link to embed source code repository information directly, allowing debugging environments like Visual Studio to request the exact version used to produce the `.pdb` file. Portable PDB files with Source Link are indexed and served by ProGet's symbol server. Once the debugger has downloaded the `.pdb` file, it will make any requests it needs to the source repository directly.

Source Server (Windows PDB)

Windows PDB files are a much older format and do not have a mechanism like Source Link to provide a direct link back to the source. Instead, these files contain a list of the full absolute paths of the source files used in the build process. Tools like pdbstr must be used to rewrite the `.pdb` file with paths that point to a specially formed URL instead.

When using the source server feature, ProGet performs this rewrite for each Windows PDB file in a NuGet package, provided that the package contains an 'src/' folder containing the source files used to build the package.

This system has several disadvantages:

- The NuGet package itself must contain the source code used to build the package, which increases the size of the package and build complexity.
- PDB rewriting is an error-prone process that relies on very old and unmaintained tools and can be difficult to configure.
- The rewriting tooling requires ProGet to be running on Windows.

For these reasons, we recommend using Portable PDB files whenever possible. ProGet accepts both types of files, but Portable PDB files provide much better support for source code debugging.

Configuring ProGet for Symbol Serving

To enable Symbol Sever support for a NuGet feed in ProGet refer to Configuring your NuGet Feed for Symbol Serving.

There are four Symbol Server modes for a NuGet feed:

Disabled: Symbol Server support for this feed is disabled.

Standard (.snupkg): Symbol Server is enabled, but only standard (.snupkg) format symbol packages are indexed.

Legacy: Symbol Server is enabled, but only legacy symbol packages are indexed, Standard (.snupkg) symbol files are not accepted.

Mixed: Symbol Server is enabled, and both standard and legacy symbol packages are indexed. If a package contains both legacy and standard symbols, only the standard symbols are used.

Legacy Symbol Server Options

When legacy or mixed mode is enabled, additional checkboxes are available to control legacy symbol package-specific options:

Strip symbol files: Legacy symbol packages store symbols directly in the "base" NuGet package. If you check this box, ProGet will remove these files from the package when the package is downloaded.

Strip source code: Removes everything in the 'src/' folder of a legacy symbol package when the package is downloaded.

Remove signature file: If the symbol or source code is removed from a package based on the previous two settings, the package signature (if it has one) becomes invalid. Select this check box to remove this signature in such cases.

Note that these settings have no effect on standard (`.snupkg`) files.

Verifying Indexed Symbols

When Symbol Server is enabled for a NuGet feed and a package has symbols indexed in ProGet, the Symbols tab is available on the package details page:

This page can be used to verify that ProGet has detected all the symbols associated with the package, whether they are stored in the package itself (in the case of a legacy symbol package) or in a separate `.snupkg` file.

The `Id` and `Age` fields are used only to uniquely identify the PDB file by the debugger. Portable PDB files use a slightly different identifier scheme, so the Age field of these PDB files typically show as negative, but this is normal.

Configuring Visual Studio

Enable Symbol Server Support

To debug NuGet package libraries, Visual Studio must be configured to use ProGet as the symbol server as shown in an earlier section of this guide.

Enable Source Server Support

> **Legacy Symbol Packages Only**
>
> Source Server is only used for Windows PDB files contained in legacy NuGet symbol packages. We recommend using Source Link with Portable PDB files instead.

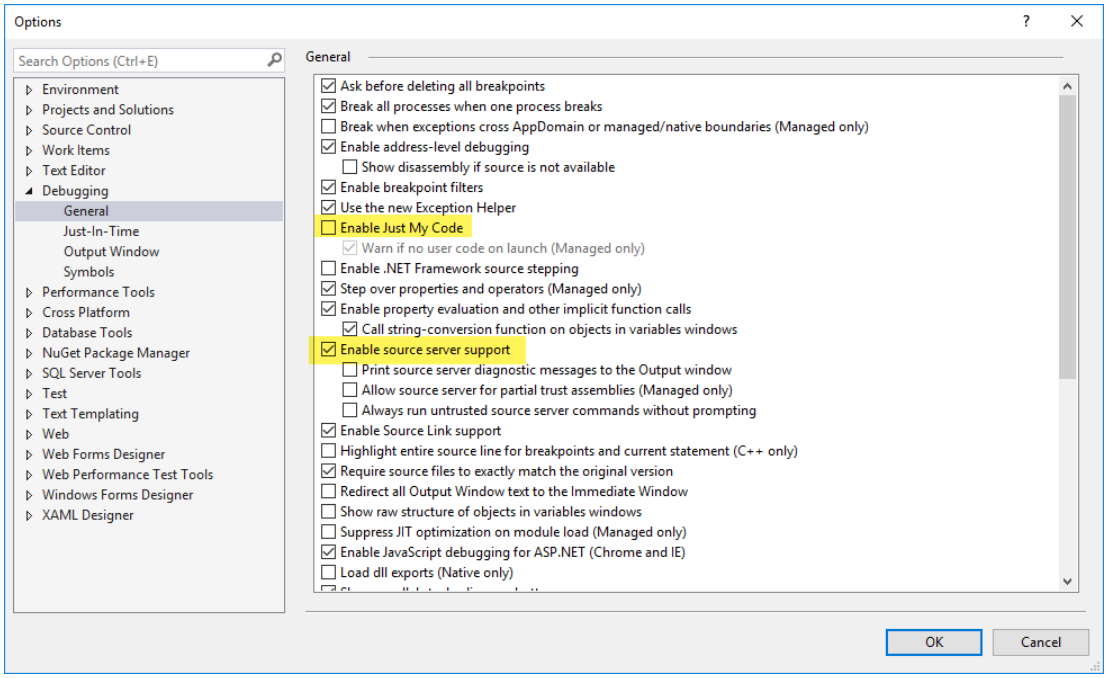
To configure source server support, go to **Debugging > General** in the debugging options tree, and make sure the following settings are enabled/disabled as follows:

- ☐ Enable Just My Code
- ☑ Enable source server support

Additionally, you may have to uncheck:

- ☐ Enable .NET Framework Source Stepping

The settings should look like the following:

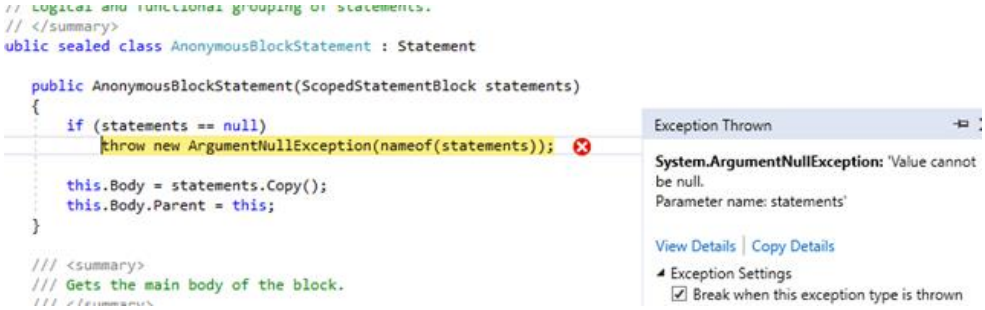


Testing the Configuration

An easy way to test the configuration is to create a console application that consumes the NuGet package with symbols, write some throwaway code that you know will throw an exception, and then click the Start button in Visual Studio to begin debugging:

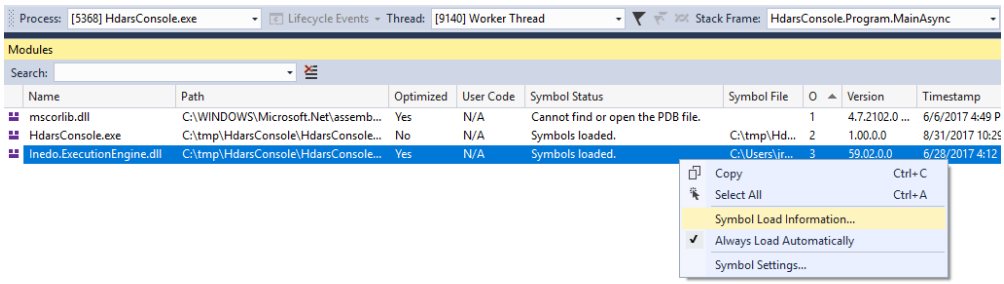
```
Internal class program
{
  Static void Main(string[] args)
  {
    var test = new Inedo.ExecutionEngine.AnonymousBlockStatement null ;
  }
}
```

If everything is configured correctly, Visual Studio tries to load the symbols locally, then queries the ProGet symbol server if they cannot be found, and the exact line that throws the exception is highlighted.

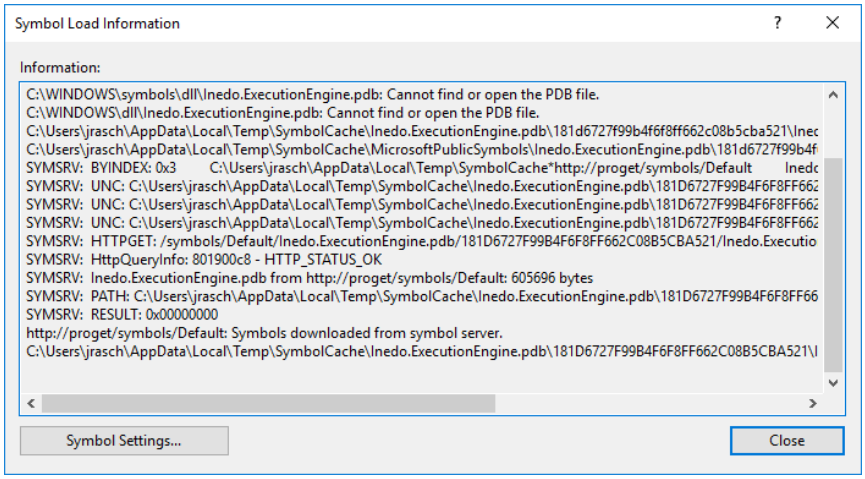


Troubleshooting

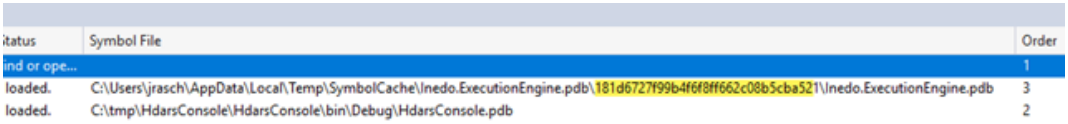
In some cases, the symbol server fails for various reasons, the most common being misconfiguration. To display the PDBs that Visual Studio has loaded, select the **Debug > Windows > Modules** menu option during debugging and locate the desired DLL in the list. The symbol status is displayed, along with the version and path to the DLL and the downloaded symbols.



Whether or not the symbols have been loaded, you can right-click on the DLL and select **Symbol Load Information...** to view the diagnostic messages associated with the symbol server for that library.



The hex string in the file path should also start with the GUID listed in ProGet.



The GUID will be found under the **Symbols** tab on the NuGet package.

Common Errors

The most common errors (based on previous support inquiries) include:

- Using the wrong URL, e.g. http://«proget-server»/**nuget**/«feed-name» instead of the correct http://«proget-server»/**symbols**/«feed-name».
- Pushing both NuGet packages (with and without symbols) when using legacy symbol packages.
- Trying to consume symbols for connector packages (which is not supported, a workaround is to pull packages locally or use a separate feed for symbols).
- Not including source files under the `/src/` directory at the root of the `.nupkg` file (legacy symbol packages only).

About the Author

Hey 👋, I'm Crista, Solutions Architect at Inedo, longtime developer, and someone who accidentally fell in love with internal tooling and package management. I've spent years helping teams make sense of their NuGet setup, from one-person projects to enterprise-scale chaos.

This book came out of those experiences. The late-night Slack messages, the broken CI builds, the "wait, which version are we using?" mysteries. I wanted to create something that feels practical and honest, not just another slide deck pretending everything is simple. Whether you're just getting started or trying to wrangle an existing NuGet mess, I hope what's in here helps you scale with a little more confidence... and maybe fewer headaches.

Solutions Architect at Inedo

Crista Perlton

[Connect with me on LinkedIn]

NUGET AT SCALE VER 2.1.0

An enterprise guide to managing NuGet package approvals, handling vulnerabilities, versioning and license compliance, setting up a solid CI/CD pipeline, and everything else growing teams need to scale with confidence.

Berea, OH, United States

inedo